



Introduction

- Import the Libraries

```
In [1]: import pandas as pd
import numpy as np
import os
from pathlib import Path
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
from scipy.stats import kstest, norm
import scipy.stats as stats
from scipy.fft import fft, fftfreq
from scipy.signal import find_peaks
from scipy.stats import kruskal
from scipy.stats import shapiro
from scipy import stats
from scipy.fft import rfft, rfftfreq
from scipy.stats import zscore
import random
from collections import Counter
from sklearn.neighbors import NearestNeighbors
from sklearn.preprocessing import StandardScaler
import seaborn as sns

from sklearn.metrics import (
    confusion_matrix,
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    classification_report
)

# from scipy.ndimage import gaussian_filter1d
```

2 - Download the Data

- Download the data

```
In [2]: BASE_PATH = os.path.join(os.getcwd(), "FORTH_TRACE_DATASET-master/FORTH_TRACE_

def load_data(participant_id, device_id, base_path=BASE_PATH):
    filepath = os.path.join(base_path, f"part{participant_id}", f"part{partici
    try:
        data = np.genfromtxt(filepath, delimiter=',')
        return data
    except FileNotFoundError:
```

```
print(f"Error:File not found at {filepath}")  
return None
```

- Loading the data from only device 1 for each individual
- [column 1: device ID,
column 2: accelerometer x,
column 3: accelerometer y,
column 4: accelerometer z,
column 5: gyroscope x,
column 6: gyroscope y,
column 7: gyroscope z,
column 8: magnetometer x,
column 9: magnetometer y,
column 10: magnetometer z,
column 11: TimeStamp,
column 12: Activity Label]
- Each individual has different number of data (individuals[i])
- Number of individuals: 15
- Each individual has their respective TimeStamps start and end

```
In [3]: individuals = []  
for i in range(15):  
    data = load_data(i, 1)  
    individuals.append(data)  
  
print("Nmber of arrays: ",len(individuals),"\n")  
print("Arrays: ",individuals)
```

Nmber of arrays: 15

```
Arrays: [array([[ 1.0000e+00, -1.8650e+00,  9.3890e+00, ...,  1.0134e+00,
 5.0589e+02,  1.0000e+00],
 [ 1.0000e+00, -1.7963e+00,  9.3742e+00, ...,  1.0000e+00,
 5.2542e+02,  1.0000e+00],
 [ 1.0000e+00, -1.8696e+00,  9.3000e+00, ...,  1.0156e+00,
 5.4495e+02,  1.0000e+00],
 ...,
 [ 1.0000e+00, -2.1324e+00,  9.3112e+00, ...,  5.4343e-01,
 1.0379e+06,  1.0000e+00],
 [ 1.0000e+00, -2.1327e+00,  9.3110e+00, ...,  5.5234e-01,
 1.0380e+06,  1.0000e+00],
 [ 1.0000e+00, -2.1334e+00,  9.2982e+00, ...,  5.4120e-01,
 1.0380e+06,  1.0000e+00]], shape=(53120, 12)), array([[ 1.0000e+00,
-1.1173e+00,  9.6762e+00, ...,  4.6771e-02,
 8.7930e+02,  1.0000e+00],
 [ 1.0000e+00, -1.2130e+00,  9.7239e+00, ...,  5.7906e-02,
 8.9883e+02,  1.0000e+00],
 [ 1.0000e+00, -1.1892e+00,  9.7119e+00, ...,  6.4588e-02,
 9.1837e+02,  1.0000e+00],
 ...,
 [ 1.0000e+00, -1.3065e+00,  9.7499e+00, ...,  5.1448e-01,
 1.0433e+06,  1.0000e+00],
 [ 1.0000e+00, -1.3065e+00,  9.7137e+00, ...,  5.2339e-01,
 1.0433e+06,  1.0000e+00],
 [ 1.0000e+00, -1.3184e+00,  9.7137e+00, ...,  5.2561e-01,
 1.0434e+06,  1.0000e+00]], shape=(53376, 12)), array([[ 1.0000e+00,
-2.1166e+00,  9.5323e+00, ...,  6.4811e-01,
 6.5375e+02,  1.0000e+00],
 [ 1.0000e+00, -2.1394e+00,  9.5213e+00, ...,  6.6147e-01,
 6.7328e+02,  1.0000e+00],
 [ 1.0000e+00, -2.1398e+00,  9.4244e+00, ...,  6.6147e-01,
 6.9281e+02,  1.0000e+00],
 ...,
 [ 1.0000e+00, -1.9381e+00,  9.6166e+00, ...,  6.1470e-01,
 1.0406e+06,  1.0000e+00],
 [ 1.0000e+00, -1.9257e+00,  9.4966e+00, ...,  6.2138e-01,
 1.0406e+06,  1.0000e+00],
 [ 1.0000e+00, -1.9502e+00,  9.5079e+00, ...,  6.1470e-01,
 1.0406e+06,  1.0000e+00]], shape=(53248, 12)), array([[ 1.0000e+00,
-3.8686e+00,  8.3008e+00, ...,  9.5100e-01,
 6.6046e+02,  1.0000e+00],
 [ 1.0000e+00, -3.9050e+00,  8.3242e+00, ...,  9.5991e-01,
 6.7999e+02,  1.0000e+00],
 [ 1.0000e+00, -3.9167e+00,  8.3365e+00, ...,  9.5768e-01,
 6.9952e+02,  1.0000e+00],
 ...,
 [ 1.0000e+00, -3.5183e+00,  8.5349e+00, ...,  6.4143e-01,
 1.0410e+06,  1.0000e+00],
 [ 1.0000e+00, -3.4937e+00,  8.5477e+00, ...,  6.4143e-01,
 1.0410e+06,  1.0000e+00],
 [ 1.0000e+00, -3.5171e+00,  8.6084e+00, ...,  6.5256e-01,
 1.0410e+06,  1.0000e+00]], shape=(53268, 12)), array([[ 1.0000e+00,
```

```

-2.2433e+00, 9.4040e+00, ..., 8.0401e-01,
    9.7601e+04, 1.0000e+00],
[ 1.0000e+00, -2.2197e+00, 9.4399e+00, ..., 8.0624e-01,
    9.7620e+04, 1.0000e+00],
[ 1.0000e+00, -2.1962e+00, 9.4397e+00, ..., 7.8619e-01,
    9.7640e+04, 1.0000e+00],
...,
[ 1.0000e+00, -2.1902e+00, 9.6626e+00, ..., 5.0111e-01,
    1.1353e+06, 1.0000e+00],
[ 1.0000e+00, -2.2247e+00, 9.6277e+00, ..., 4.9443e-01,
    1.1353e+06, 1.0000e+00],
[ 1.0000e+00, -2.2001e+00, 9.5682e+00, ..., 5.0557e-01,
    1.1353e+06, 1.0000e+00]], shape=(53120, 12)), array([[ 1.0000e+00,
-2.3432e+00, 9.5200e+00, ..., 3.8085e-01,
    1.5095e+03, 1.0000e+00],
[ 1.0000e+00, -2.3908e+00, 9.5320e+00, ..., 3.9198e-01,
    1.5290e+03, 1.0000e+00],
[ 1.0000e+00, -2.3306e+00, 9.5328e+00, ..., 3.9644e-01,
    1.5486e+03, 1.0000e+00],
...,
[ 1.0000e+00, -2.3194e+00, 9.4718e+00, ..., 1.7372e-01,
    1.0339e+06, 1.0000e+00],
[ 1.0000e+00, -2.3432e+00, 9.4718e+00, ..., 1.7149e-01,
    1.0339e+06, 1.0000e+00],
[ 1.0000e+00, -2.4034e+00, 9.4711e+00, ..., 1.6481e-01,
    1.0339e+06, 1.0000e+00]], shape=(52861, 12)), array([[ 1.0000e+00,
-3.7502e+00, 8.9632e+00, ..., 8.0846e-01,
    7.7936e+02, 1.0000e+00],
[ 1.0000e+00, -3.8095e+00, 8.9514e+00, ..., 8.0178e-01,
    7.9889e+02, 1.0000e+00],
[ 1.0000e+00, -3.7971e+00, 8.9278e+00, ..., 7.8842e-01,
    8.1842e+02, 1.0000e+00],
...,
[ 1.0000e+00, -3.7296e+00, 8.9239e+00, ..., 6.4143e-01,
    1.0482e+06, 1.0000e+00],
[ 1.0000e+00, -3.7534e+00, 8.9359e+00, ..., 6.3920e-01,
    1.0482e+06, 1.0000e+00],
[ 1.0000e+00, -3.7531e+00, 8.9241e+00, ..., 6.6147e-01,
    1.0483e+06, 1.0000e+00]], shape=(53632, 12)), array([[ 1.0000e+00,
-8.4302e-01, 9.3027e+00, ..., 1.0913e+00,
    1.9184e+03, 1.0000e+00],
[ 1.0000e+00, -8.6780e-01, 9.3138e+00, ..., 1.0935e+00,
    1.9380e+03, 1.0000e+00],
[ 1.0000e+00, -9.1345e-01, 9.3157e+00, ..., 1.0980e+00,
    1.9575e+03, 1.0000e+00],
...,
[ 1.0000e+00, -1.1540e+00, 9.4460e+00, ..., 4.6102e-01,
    1.0269e+06, 1.0000e+00],
[ 1.0000e+00, -1.1904e+00, 9.4453e+00, ..., 4.8552e-01,
    1.0269e+06, 1.0000e+00],
[ 1.0000e+00, -1.1780e+00, 9.4819e+00, ..., 4.7216e-01,
    1.0269e+06, 1.0000e+00]], shape=(52480, 12)), array([[ 1.0000e+00,
-6.6783e-01, 9.5163e+00, ..., 9.7327e-01,
    3.6050e+02, 1.0000e+00],

```

```

[ 1.0000e+00, -6.9162e-01, 9.5043e+00, ..., 9.7550e-01,
 3.8004e+02, 1.0000e+00],
[ 1.0000e+00, -7.2684e-01, 9.5289e+00, ..., 9.7105e-01,
 3.9957e+02, 1.0000e+00],
...,
[ 1.0000e+00, -9.6678e-01, 9.5030e+00, ..., -3.0067e-01,
 1.0374e+06, 1.0000e+00],
[ 1.0000e+00, -9.9084e-01, 9.5268e+00, ..., -2.9621e-01,
 1.0374e+06, 1.0000e+00],
[ 1.0000e+00, -1.0141e+00, 9.5273e+00, ..., -2.9176e-01,
 1.0374e+06, 1.0000e+00]], shape=(52992, 12)), array([[ 1.0000e+00,
-1.9803e+00, 9.5256e+00, ..., 5.2116e-01,
 3.5955e+04, 1.0000e+00],
[ 1.0000e+00, -1.9808e+00, 9.5010e+00, ..., 5.1448e-01,
 3.5975e+04, 1.0000e+00],
[ 1.0000e+00, -1.9567e+00, 9.5013e+00, ..., 5.0557e-01,
 3.5994e+04, 1.0000e+00],
...,
[ 1.0000e+00, -1.4198e+00, 9.5988e+00, ..., 5.9020e-01,
 1.0609e+06, 1.0000e+00],
[ 1.0000e+00, -1.4084e+00, 9.6104e+00, ..., 5.9911e-01,
 1.0609e+06, 1.0000e+00],
[ 1.0000e+00, -1.4196e+00, 9.5991e+00, ..., 5.9911e-01,
 1.0609e+06, 1.0000e+00]], shape=(52480, 12)), array([[ 1.0000e+00,
-2.4716e-01, 9.4721e+00, ..., 9.9555e-01,
 4.1565e+01, 1.0000e+00],
[ 1.0000e+00, -3.0546e-01, 9.5216e+00, ..., 1.0089e+00,
 6.1096e+01, 1.0000e+00],
[ 1.0000e+00, -2.3356e-01, 9.4859e+00, ..., 1.0156e+00,
 8.0627e+01, 1.0000e+00],
...,
[ 1.0000e+00, 2.3483e-01, 9.6227e+00, ..., 6.6592e-01,
 1.0418e+06, 1.0000e+00],
[ 1.0000e+00, 2.8366e-01, 9.5757e+00, ..., 6.7483e-01,
 1.0418e+06, 1.0000e+00],
[ 1.0000e+00, 3.0601e-01, 9.5622e+00, ..., 6.6592e-01,
 1.0418e+06, 1.0000e+00]], shape=(51456, 12)), array([[ 1.0000e+00,
-2.7330e+00, 8.9200e+00, ..., 8.7528e-01,
 1.5372e+03, 1.0000e+00],
[ 1.0000e+00, -2.7328e+00, 8.9203e+00, ..., 8.8641e-01,
 1.5567e+03, 1.0000e+00],
[ 1.0000e+00, -2.6970e+00, 8.8359e+00, ..., 8.9310e-01,
 1.5762e+03, 1.0000e+00],
...,
[ 1.0000e+00, -2.6073e+00, 8.9992e+00, ..., 5.2116e-01,
 1.0520e+06, 1.0000e+00],
[ 1.0000e+00, -2.6187e+00, 8.9876e+00, ..., 5.1448e-01,
 1.0520e+06, 1.0000e+00],
[ 1.0000e+00, -2.6313e+00, 8.9387e+00, ..., 5.1670e-01,
 1.0520e+06, 1.0000e+00]], shape=(52864, 12)), array([[ 1.0000e+00,
-2.0018e+00, 9.3711e+00, ..., 8.8196e-01,
 7.4036e+02, 1.0000e+00],
[ 1.0000e+00, -2.0382e+00, 9.3584e+00, ..., 8.8641e-01,
 7.5989e+02, 1.0000e+00],

```

```
[ 1.0000e+00, -2.0620e+00, 9.3704e+00, ..., 8.8641e-01,
 7.7942e+02, 1.0000e+00],
...,
[ 1.0000e+00, -1.9153e+00, 9.3743e+00, ..., -2.9621e-01,
 1.0382e+06, 1.0000e+00],
[ 1.0000e+00, -1.8429e+00, 9.3511e+00, ..., -2.9621e-01,
 1.0382e+06, 1.0000e+00],
[ 1.0000e+00, -1.8201e+00, 9.3622e+00, ..., -2.8062e-01,
 1.0382e+06, 1.0000e+00]], shape=(53120, 12)), array([[ 1.0000e+00,
2.8373e-02, 9.6427e+00, ..., 7.2606e-01,
 7.8070e+02, 1.0000e+00],
[ 1.0000e+00, -1.8983e-02, 9.6430e+00, ..., 7.1492e-01,
 8.0023e+02, 1.0000e+00],
[ 1.0000e+00, -6.5862e-02, 9.6678e+00, ..., 7.1269e-01,
 8.1976e+02, 1.0000e+00],
...,
[ 1.0000e+00, -2.2962e+00, 9.4106e+00, ..., 4.6771e-02,
 1.0462e+06, 1.0000e+00],
[ 1.0000e+00, -2.2729e+00, 9.3861e+00, ..., 6.6815e-02,
 1.0463e+06, 1.0000e+00],
[ 1.0000e+00, -2.2491e+00, 9.3861e+00, ..., 5.7906e-02,
 1.0463e+06, 1.0000e+00]], shape=(52992, 12)), array([[ 1.0000e+00,
-2.8247e+00, 9.1527e+00, ..., 6.7261e-01,
 1.7156e+03, 1.0000e+00],
[ 1.0000e+00, -2.8243e+00, 9.1773e+00, ..., 6.7483e-01,
 1.7351e+03, 1.0000e+00],
[ 1.0000e+00, -2.8002e+00, 9.1655e+00, ..., 6.7483e-01,
 1.7546e+03, 1.0000e+00],
...,
[ 1.0000e+00, -2.8658e+00, 9.2196e+00, ..., -1.9154e-01,
 1.0417e+06, 1.0000e+00],
[ 1.0000e+00, -2.7702e+00, 9.2683e+00, ..., -1.9376e-01,
 1.0417e+06, 1.0000e+00],
[ 1.0000e+00, -2.7935e+00, 9.2929e+00, ..., -2.1381e-01,
 1.0417e+06, 1.0000e+00]], shape=(53248, 12))]
```

- Each array has different sizes, these is expected because they used continuos sensors

```
In [4]: for i in range(len(individuals)):
        print(f"Lenght of array {i}: ",len(individuals[i]))
```

Lenght of array 0: 53120
Lenght of array 1: 53376
Lenght of array 2: 53248
Lenght of array 3: 53268
Lenght of array 4: 53120
Lenght of array 5: 52861
Lenght of array 6: 53632
Lenght of array 7: 52480
Lenght of array 8: 52992
Lenght of array 9: 52480
Lenght of array 10: 51456
Lenght of array 11: 52864
Lenght of array 12: 53120
Lenght of array 13: 52992
Lenght of array 14: 53248

3 - Outliers Analysis and Handling

3.1 - Boxplot for each activity

- Calculation of each vector module
- Column 2: accelerometer x
- Column 3: accelerometer y
- Column 4: accelerometer z
- Column 5: gyroscope x
- Column 6: gyroscope y
- Column 7: gyroscope z
- Column 8: magnetometer x
- Column 9: magnetometer y
- Column 10: magnetometer z
- Column 12: Activity Label

```
In [5]: activities = {1:[],2:[],3:[],4:[]}

for i in range(len(individuals)):
    data = individuals[i]
    for row in data:
        label = int(row[11])
```

```

    if label in activities:
        accelerometer = np.sqrt(row[1]**2 + row[2]**2 + row[3]**2)
        gyroscope = np.sqrt(row[4]**2 + row[5]**2 + row[6]**2)
        magnetometer = np.sqrt(row[7]**2 + row[8]**2 + row[9]**2)

        activities[label].append((accelerometer,gyroscope,magnetometer))

print("Activities Structure: ",len(activities))

```

Activities Structure: 4

- Each activity has different data sizes

```

In [6]: for key in activities:
        print(f"Number of activity {key} data: ",len(activities[key]))

```

```

Number of activity 1 data: 162125
Number of activity 2 data: 85632
Number of activity 3 data: 83584
Number of activity 4 data: 141342

```

- Boxplot of sensors/activity

```

In [7]: accelerometer_activity = []
        gyroscope_activity = []
        magnetometer_activity = []

        for act in range(1, 5):
            acceloremeter_values = [x[0] for x in activities[act]]
            gyroscope_values = [x[1] for x in activities[act]]
            magnetometer_values = [x[2] for x in activities[act]]

            accelerometer_activity.append(acceloremeter_values)
            gyroscope_activity.append(gyroscope_values)
            magnetometer_activity.append(magnetometer_values)

        print("Lenght of Accelerometer data: ",len(accelerometer_activity))
        print("Lenght of Gyroscope data: ",len(gyroscope_activity))
        print("Lenght of Magnetometer data: ",len(magnetometer_activity))

```

```

Lenght of Accelerometer data: 4
Lenght of Gyroscope data: 4
Lenght of Magnetometer data: 4

```

- Each row of each activity for the accelerometer_activity has the (ax,ay,az) coordinates but all are consecutive, so i need to access three in three to be like activities dictionary

```

In [8]: lenghts_acc= []
        for i in range(len(accelerometer_activity)):

```

```
lengths_acc.append(len(accelerometer_activity[i]))

print(lengths_acc)
```

[162125, 85632, 83584, 141342]

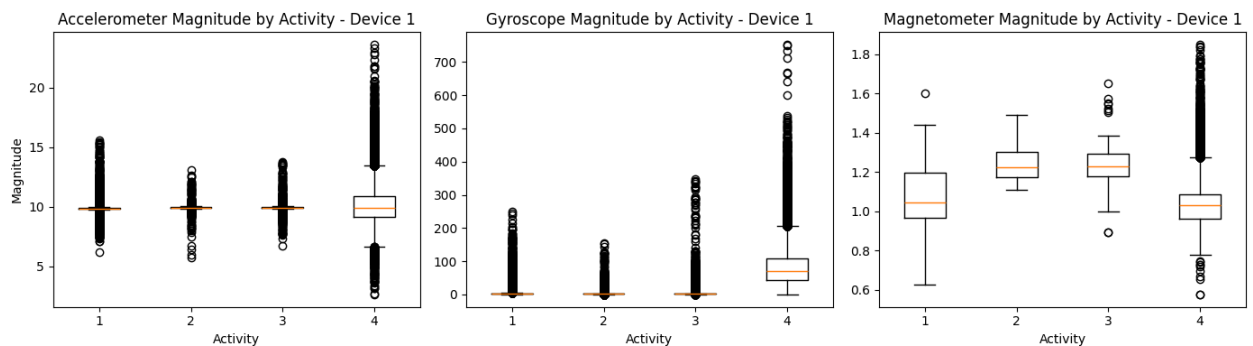
```
In [9]: plt.figure(figsize=(14, 4))

plt.subplot(1, 3, 1)
plt.boxplot(accelerometer_activity)
plt.title("Accelerometer Magnitude by Activity - Device 1")
plt.xlabel("Activity")
plt.ylabel("Magnitude")

plt.subplot(1, 3, 2)
plt.boxplot(gyroscope_activity)
plt.title("Gyroscope Magnitude by Activity - Device 1")
plt.xlabel("Activity")

plt.subplot(1, 3, 3)
plt.boxplot(magnetometer_activity)
plt.title("Magnetometer Magnitude by Activity - Device 1")
plt.xlabel("Activity")

plt.tight_layout()
plt.show()
```



3.2 - Analyse and Comment the Density of Outliers

- IQR Implementation
- Sort the lists of each sensor

```
In [10]: accelerometer_activity_sorted = []

for i in range(len(accelerometer_activity)):
    sorted_values = sorted(accelerometer_activity[i])
    accelerometer_activity_sorted.append(sorted_values)

gyroscope_activity_sorted = []
```

```

for i in range(len(gyroscope_activity)):
    sorted_values = sorted(gyroscope_activity[i])
    gyroscope_activity_sorted.append(sorted_values)

magnetometer_activity_sorted = []

for i in range(len(magnetometer_activity)):
    sorted_values = sorted(magnetometer_activity[i])
    magnetometer_activity_sorted.append(sorted_values)

```

- Length of each sensor list

```

In [11]: len_accelerometer = []

for i in range(len(accelerometer_activity_sorted)):
    len_accelerometer.append(len(accelerometer_activity_sorted[i]))

len_gyroscope = []

for i in range(len(gyroscope_activity_sorted)):
    len_gyroscope.append(len(gyroscope_activity_sorted[i]))

len_magnetometer = []

for i in range(len(magnetometer_activity_sorted)):
    len_magnetometer.append(len(magnetometer_activity_sorted[i]))

print("Accelerometer: ", len_accelerometer)
print("Gyroscope: ", len_gyroscope)
print("Magnetometer: ", len_magnetometer)

```

```

Accelerometer: [162125, 85632, 83584, 141342]
Gyroscope: [162125, 85632, 83584, 141342]
Magnetometer: [162125, 85632, 83584, 141342]

```

```

In [12]: Q1_acc = []
Q3_acc = []

for i in range(len(accelerometer_activity_sorted)):
    values = accelerometer_activity_sorted[i]
    n = len(values)
    pos_Q1 = 0.25 * (n + 1)
    pos_Q3 = 0.75 * (n + 1)

    lower_Q1 = int(pos_Q1) - 1
    upper_Q1 = lower_Q1 + 1
    lower_Q3 = int(pos_Q3) - 1

```

```

upper_Q3 = lower_Q3 + 1

Q1_val = values[lower_Q1] + (pos_Q1 - int(pos_Q1)) * (values[upper_Q1] - v
Q3_val = values[lower_Q3] + (pos_Q3 - int(pos_Q3)) * (values[upper_Q3] - v

Q1_acc.append(Q1_val)
Q3_acc.append(Q3_val)

Q1_gyr = []
Q3_gyr = []

for i in range(len(gyroscope_activity_sorted)):
    values = gyroscope_activity_sorted[i]
    n = len(values)
    pos_Q1 = 0.25 * (n + 1)
    pos_Q3 = 0.75 * (n + 1)

    lower_Q1 = int(pos_Q1) - 1
    upper_Q1 = lower_Q1 + 1
    lower_Q3 = int(pos_Q3) - 1
    upper_Q3 = lower_Q3 + 1

    Q1_val = values[lower_Q1] + (pos_Q1 - int(pos_Q1)) * (values[upper_Q1] - v
    Q3_val = values[lower_Q3] + (pos_Q3 - int(pos_Q3)) * (values[upper_Q3] - v

    Q1_gyr.append(Q1_val)
    Q3_gyr.append(Q3_val)

Q1_mag = []
Q3_mag = []

for i in range(len(magnetometer_activity_sorted)):
    values = magnetometer_activity_sorted[i]
    n = len(values)
    pos_Q1 = 0.25 * (n + 1)
    pos_Q3 = 0.75 * (n + 1)

    lower_Q1 = int(pos_Q1) - 1
    upper_Q1 = lower_Q1 + 1
    lower_Q3 = int(pos_Q3) - 1
    upper_Q3 = lower_Q3 + 1

    Q1_val = values[lower_Q1] + (pos_Q1 - int(pos_Q1)) * (values[upper_Q1] - v
    Q3_val = values[lower_Q3] + (pos_Q3 - int(pos_Q3)) * (values[upper_Q3] - v

    Q1_mag.append(Q1_val)
    Q3_mag.append(Q3_val)

```

```

In [13]: print("Q1 (acc): ",Q1_acc," Q3 (acc): ",Q3_acc)
          print("Q1 (gyr): ",Q1_gyr," Q3 (gyr): ",Q3_gyr)
          print("Q1 (mag): ",Q1_mag," Q3 (mag): ",Q3_mag)

```

```
Q1 (acc): [np.float64(9.819646410640253), np.float64(9.901181647157069), np.float64(9.883672002095173), np.float64(9.179633085362125)] Q3 (acc): [np.float64(9.870302200687677), np.float64(9.969197738113133), np.float64(9.940777034517975), np.float64(10.89105599130694)]
Q1 (gyr): [np.float64(1.3965113863838314), np.float64(1.1622591618481655), np.float64(1.2700557673988524), np.float64(42.23248137752593)] Q3 (gyr): [np.float64(2.6834860928961377), np.float64(1.5489879470143055), np.float64(1.9141148821648422), np.float64(107.80765056800097)]
Q1 (mag): [np.float64(0.9656947954601289), np.float64(1.1714641708767508), np.float64(1.1757712458930947), np.float64(0.9603195538311597)] Q3 (mag): [np.float64(1.1962697774540487), np.float64(1.301677157093859), np.float64(1.2948422088894886), np.float64(1.0854070396053244)]
```

- IQR (lower and upper interval) for each sensor and each activity

```
In [14]: k = 1.5

IQR_acc = [Q3_acc[i] - Q1_acc[i] for i in range(len(Q1_acc))]
lower_acc = [Q1_acc[i] - k * IQR_acc[i] for i in range(len(IQR_acc))]
upper_acc = [Q3_acc[i] + k * IQR_acc[i] for i in range(len(IQR_acc))]

IQR_gyr = [Q3_gyr[i] - Q1_gyr[i] for i in range(len(Q1_gyr))]
lower_gyr = [Q1_gyr[i] - k * IQR_gyr[i] for i in range(len(IQR_gyr))]
upper_gyr = [Q3_gyr[i] + k * IQR_gyr[i] for i in range(len(IQR_gyr))]

IQR_mag = [Q3_mag[i] - Q1_mag[i] for i in range(len(Q1_mag))]
lower_mag = [Q1_mag[i] - k * IQR_mag[i] for i in range(len(IQR_mag))]
upper_mag = [Q3_mag[i] + k * IQR_mag[i] for i in range(len(IQR_mag))]
```

```
In [15]: print(lower_acc, upper_acc)
print(lower_gyr, upper_gyr)
print(lower_mag, upper_mag)

[np.float64(9.743662725569116), np.float64(9.799157510722972), np.float64(9.798014453460969), np.float64(6.612498726444903)] [np.float64(9.946285885758812), np.float64(10.071221874547229), np.float64(10.026434583152177), np.float64(13.458190350224163)]
[np.float64(-0.533950673384628), np.float64(0.5821659840989554), np.float64(0.30396709524986776), np.float64(-56.13027240818663)] [np.float64(4.613948152664597), np.float64(2.1290811247635157), np.float64(2.8802035543138267), np.float64(206.17040435371354)]
[np.float64(0.6198323224692492), np.float64(0.9761446915510885), np.float64(0.9971648013985037), np.float64(0.7726883251699128)] [np.float64(1.5421322504449284), np.float64(1.4969966364195213), np.float64(1.4734486533840796), np.float64(1.2730382682665713)]
```

Points that were classified as outliers

```
In [16]: outliers_acc = []

for i, values in enumerate(accelerometer_activity_sorted):
    outliers_i = [v for v in values if v < lower_acc[i] or v > upper_acc[i]]
```

```

        outliers_acc.append(outliers_i)

outliers_gyr = []
for i, values in enumerate(gyroscope_activity_sorted):
    outliers_i = [v for v in values if v < lower_gyr[i] or v > upper_gyr[i]]
    outliers_gyr.append(outliers_i)

outliers_mag = []
for i, values in enumerate(magnetometer_activity_sorted):
    outliers_i = [v for v in values if v < lower_mag[i] or v > upper_mag[i]]
    outliers_mag.append(outliers_i)

```

```

In [17]: nr_outliers_acc = [len(outliers) for outliers in outliers_acc]
nr_outliers_gyr = [len(outliers) for outliers in outliers_gyr]
nr_outliers_mag = [len(outliers) for outliers in outliers_mag]

print("Number of outliers of accelerometer (for each activity): ",nr_outliers_acc)
print("Number of outliers of gyroscope (for each activity): ",nr_outliers_gyr)
print("Number of outliers of magnetometer (for each activity): ",nr_outliers_mag)

```

Number of outliers of accelerometer (for each activity): [9047, 532, 3285, 4556]

Number of outliers of gyroscope (for each activity): [15149, 5448, 8188, 2691]

Number of outliers of magnetometer (for each activity): [1, 0, 9, 1541]

Outliers Density

- $\text{density} = (\text{n}^\circ \text{ of points classified as outliers} / \text{n}^\circ \text{ total points}) * 100$

```

In [18]: density_accelerometer = [(nr_outliers_acc[i] / len_accelerometer[i]) * 100 for i in range(len_accelerometer)]
density_gyroscope = [(nr_outliers_gyr[i] / len_gyroscope[i]) * 100 for i in range(len_gyroscope)]
density_magnetometer = [(nr_outliers_mag[i] / len_magnetometer[i]) * 100 for i in range(len_magnetometer)]

print("Outliers density (accelerometer): ",density_accelerometer)
print("Outliers density (gyroscope): ",density_gyroscope)
print("Outliers density (magnetometer): ",density_magnetometer)

```

Outliers density (accelerometer): [5.5802621434078645, 0.6212630792227205, 3.930178024502297, 3.223387245121762]

Outliers density (gyroscope): [9.34402467232074, 6.362107623318386, 9.796133231240429, 1.9038926858258691]

Outliers density (magnetometer): [0.0006168080185042406, 0.0, 0.01076761102603369, 1.0902633329088312]

```

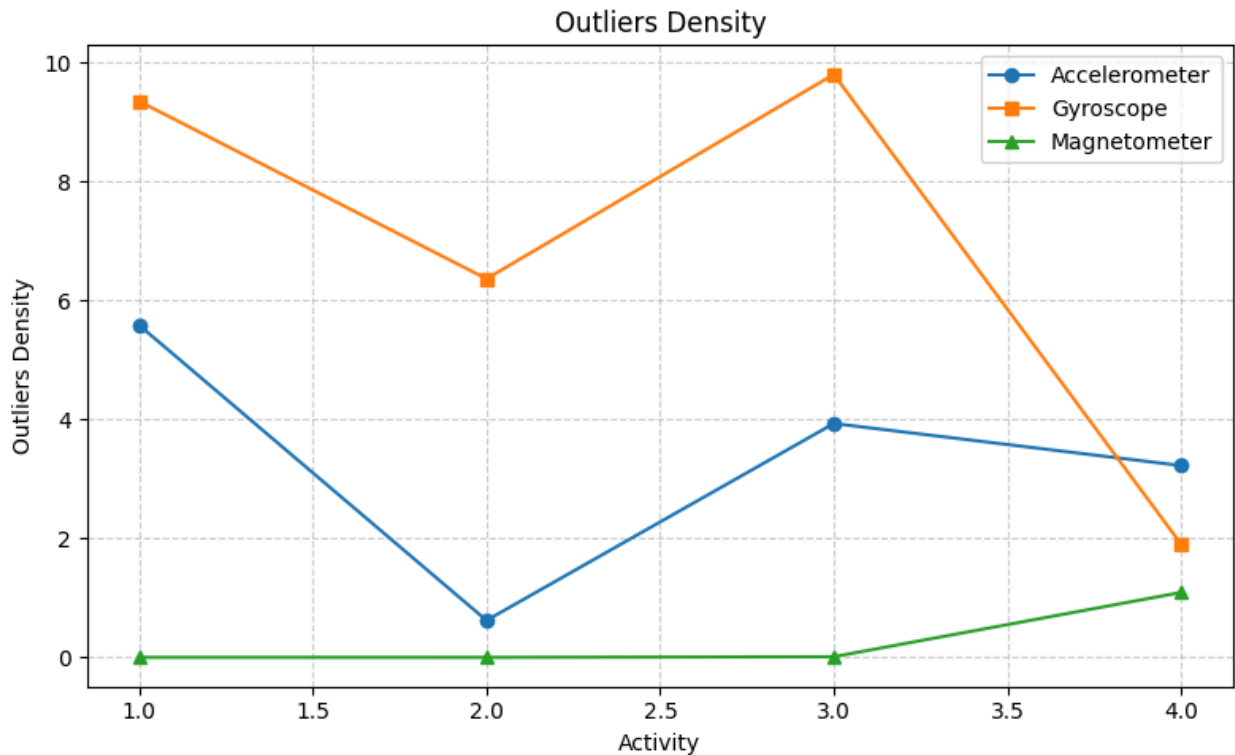
In [19]: x = np.arange(1, 5)

plt.figure(figsize=(8, 5))
plt.plot(x, density_accelerometer, marker='o', label='Accelerometer')
plt.plot(x, density_gyroscope, marker='s', label='Gyroscope')
plt.plot(x, density_magnetometer, marker='^', label='Magnetometer')

```

```
plt.title('Outliers Density')
plt.xlabel('Activity')
plt.ylabel('Outliers Density')
plt.legend()
plt.grid(True, linestyle='--', alpha=0.6)

plt.tight_layout()
plt.show()
```



- Analysing the line plot, i conclude that activities 1 (Stand) and 3 (Sit and Talk) have the higher outliers density, mostly on the gyroscope and accelerometer. This is related to the sudden movement that someone could have while standing or talking with the hands, and possibly their are useful data.

3.3 - Z-Score Implementation

```
In [20]: def zscore_sensor(sensor_data, k=3):

    z_scores_sensor = []
    outliers_sensor = []
    nr_outliers = []

    for values in sensor_data:
        n = len(values)
        if n == 0:
```

```

        z_scores_sensor.append([])
        outliers_sensor.append([])
        nr_outliers.append(0)
        continue

    mean = sum(values) / n
    variance = sum((x - mean) ** 2 for x in values) / n
    std_deviation = variance ** 0.5

    if std_deviation == 0:
        z_scores_sensor.append([0] * n)
        outliers_sensor.append([])
        nr_outliers.append(0)
        continue

    z_scores = [(x - mean) / std_deviation for x in values]
    outliers = [values[i] for i, z in enumerate(z_scores) if abs(z) > k]

    z_scores_sensor.append(z_scores)
    outliers_sensor.append(outliers)
    nr_outliers.append(len(outliers))

return z_scores_sensor, outliers_sensor, nr_outliers

```

- Z-Score (K=3)

```

In [21]: z_acc,out_acc,nr_out_acc = zscore_sensor(accelerometer_activity_sorted, k=3)
z_gyr,out_gyr,nr_out_gyr = zscore_sensor(gyroscope_activity_sorted, k=3)
z_mag,out_mag,nr_out_mag = zscore_sensor(magnetometer_activity_sorted, k=3)

print("Accelerometer outliers:", nr_out_acc)
print("Gyroscope outliers:", nr_out_gyr)
print("Magnetometer outliers:", nr_out_mag)

```

```

Accelerometer outliers: [1401, 236, 523, 1530]
Gyroscope outliers: [1963, 341, 434, 1329]
Magnetometer outliers: [8, 2, 10, 961]

```

- Z-Score (k=1.5)

```

In [22]: z_acc,out_acc,nr_out_acc = zscore_sensor(accelerometer_activity_sorted, k=1.5)
z_gyr,out_gyr,nr_out_gyr = zscore_sensor(gyroscope_activity_sorted, k=1.5)
z_mag,out_mag,nr_out_mag = zscore_sensor(magnetometer_activity_sorted, k=1.5)

print("Accelerometer outliers:", nr_out_acc)
print("Gyroscope outliers:", nr_out_gyr)
print("Magnetometer outliers:", nr_out_mag)

```

```

Accelerometer outliers: [2841, 1480, 1880, 15859]
Gyroscope outliers: [3690, 597, 834, 13129]
Magnetometer outliers: [7706, 8163, 7018, 16489]

```

3.4 - Plot of Outliers using Z-Score (k=3, 3.5 and 4)

```
In [23]: def plot_outliers_by_sensor(sensor_magnitudes, z_scores_list, k_values, activity_names):
    for k in k_values:
        outlier_counts = []
        for i, values_activity in enumerate(sensor_magnitudes):
            z = np.array(z_scores_list[i])
            count = np.sum(np.abs(z) > k)
            outlier_counts.append(count)

        print(f"\n--- {sensor_name}, k = {k} ---")
        for name, count in zip(activity_names, outlier_counts):
            print(f"{name}: {count} outliers")

        plt.figure(figsize=(10, 6))
        plt.title(f"{sensor_name} Magnitude - Outliers per Activity (k={k})")
        plt.xlabel("Activity")
        plt.ylabel("Magnitude")

        for i, values_activity in enumerate(sensor_magnitudes):
            z = np.array(z_scores_list[i])
            vals = np.array(values_activity)
            outlier_mask = np.abs(z) > k

            jitter_amount = 0.2
            x_pos_jittered = i + np.random.uniform(-jitter_amount, jitter_amount)

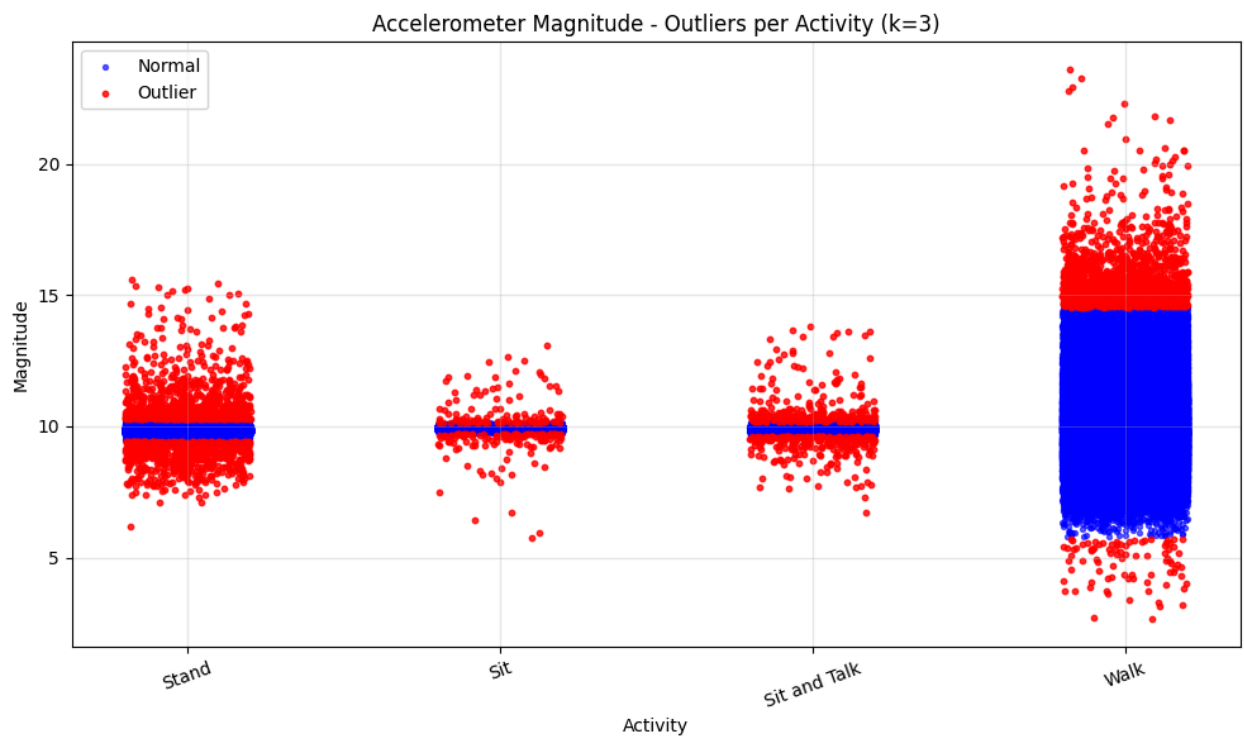
            plt.scatter(x_pos_jittered[~outlier_mask], vals[~outlier_mask],
                        color='blue', s=8, alpha=0.6, label='Normal' if i == 0 else None)
            plt.scatter(x_pos_jittered[outlier_mask], vals[outlier_mask],
                        color='red', s=10, alpha=0.8, label='Outlier' if i == 0 else None)

        plt.xticks(range(len(activity_names)), activity_names, rotation=20)
        plt.legend()
        plt.grid(alpha=0.3)
        plt.tight_layout()
        plt.show()

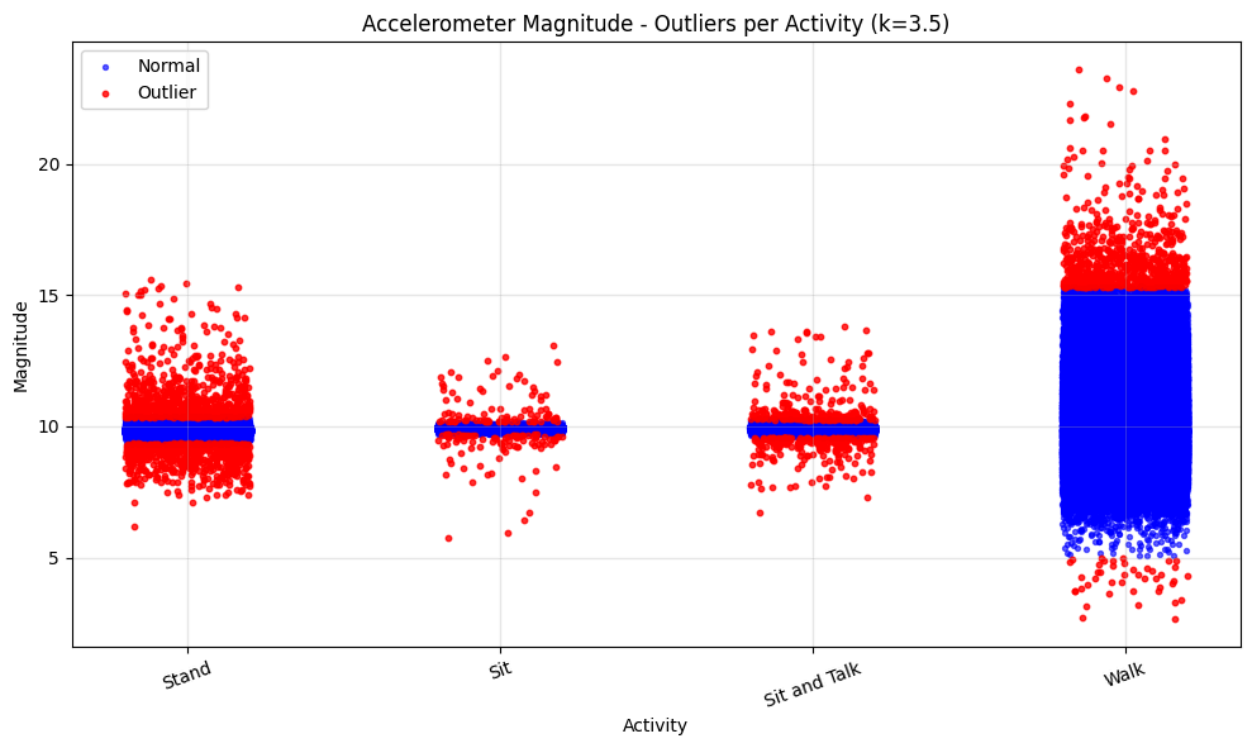
k_values = [3, 3.5, 4]
activity_names = ['Stand', 'Sit', 'Sit and Talk', 'Walk']

plot_outliers_by_sensor(accelerometer_activity_sorted, z_acc, k_values, activity_names)
plot_outliers_by_sensor(gyroscope_activity_sorted, z_gyr, k_values, activity_names)
plot_outliers_by_sensor(magnetometer_activity_sorted, z_mag, k_values, activity_names)
```

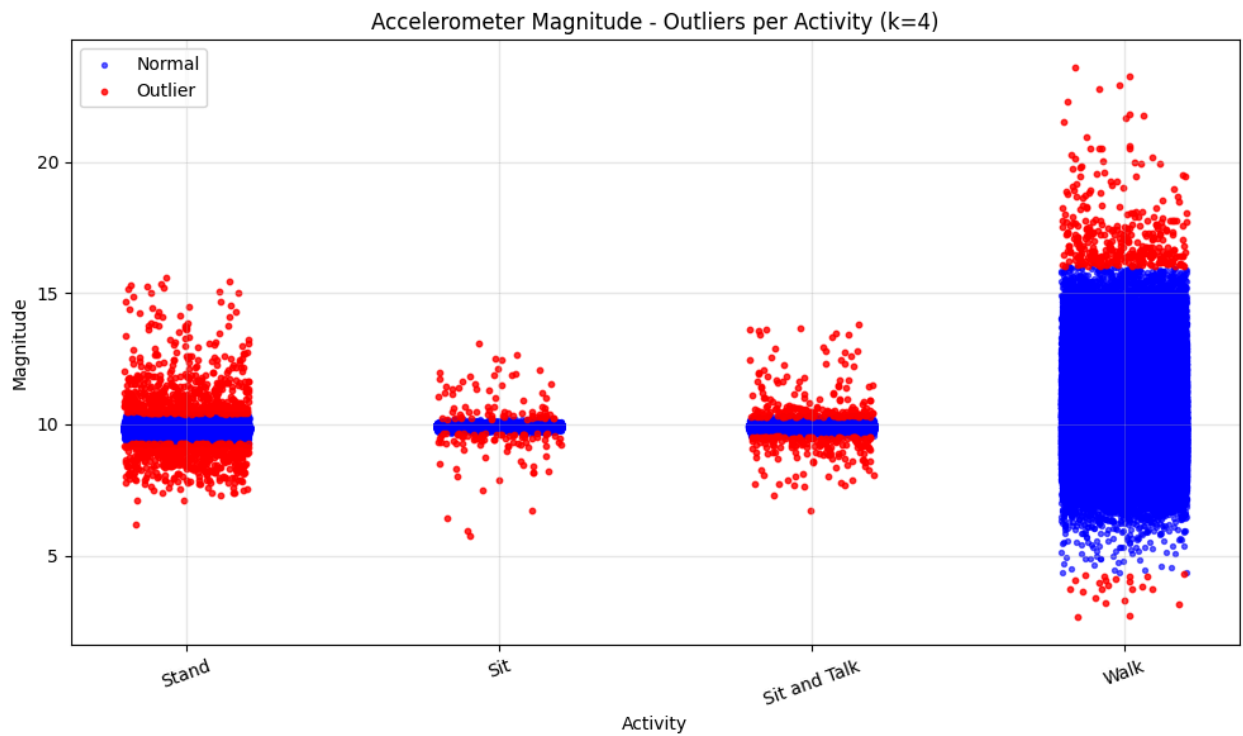
--- Accelerometer, $k = 3$ ---
Stand: 1401 outliers
Sit: 236 outliers
Sit and Talk: 523 outliers
Walk: 1530 outliers



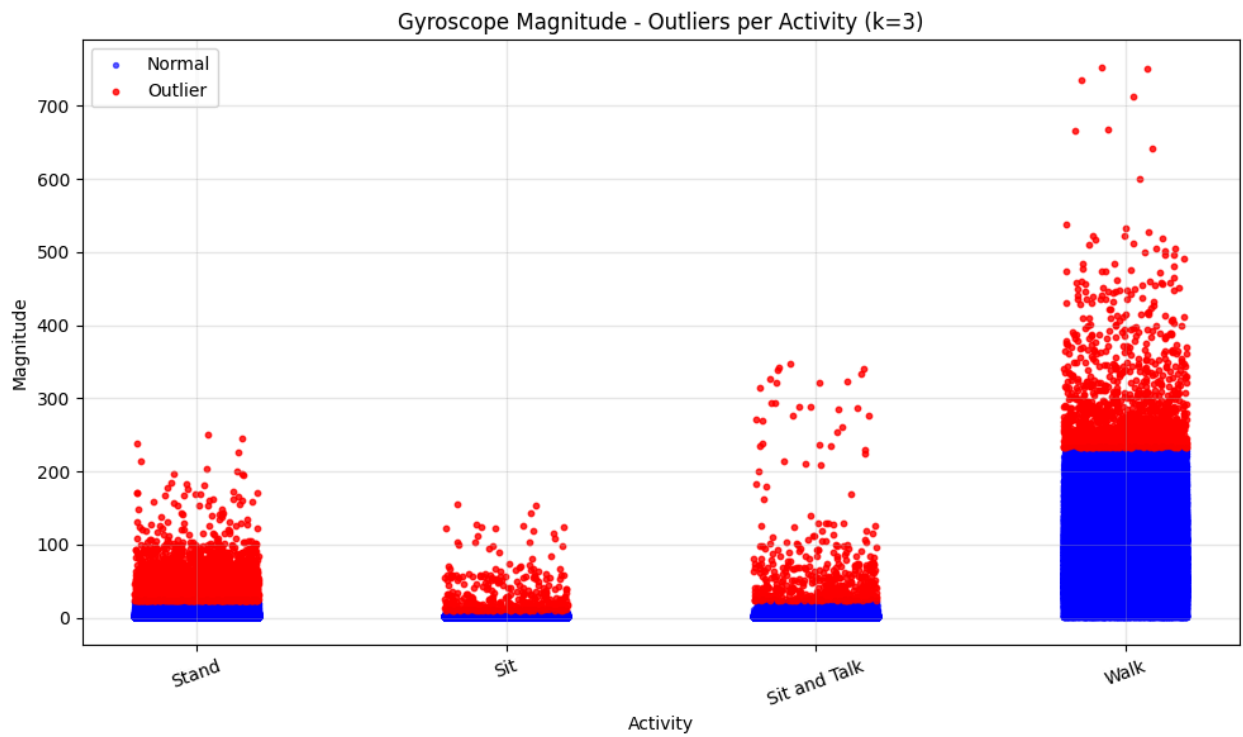
--- Accelerometer, $k = 3.5$ ---
Stand: 1201 outliers
Sit: 182 outliers
Sit and Talk: 438 outliers
Walk: 700 outliers



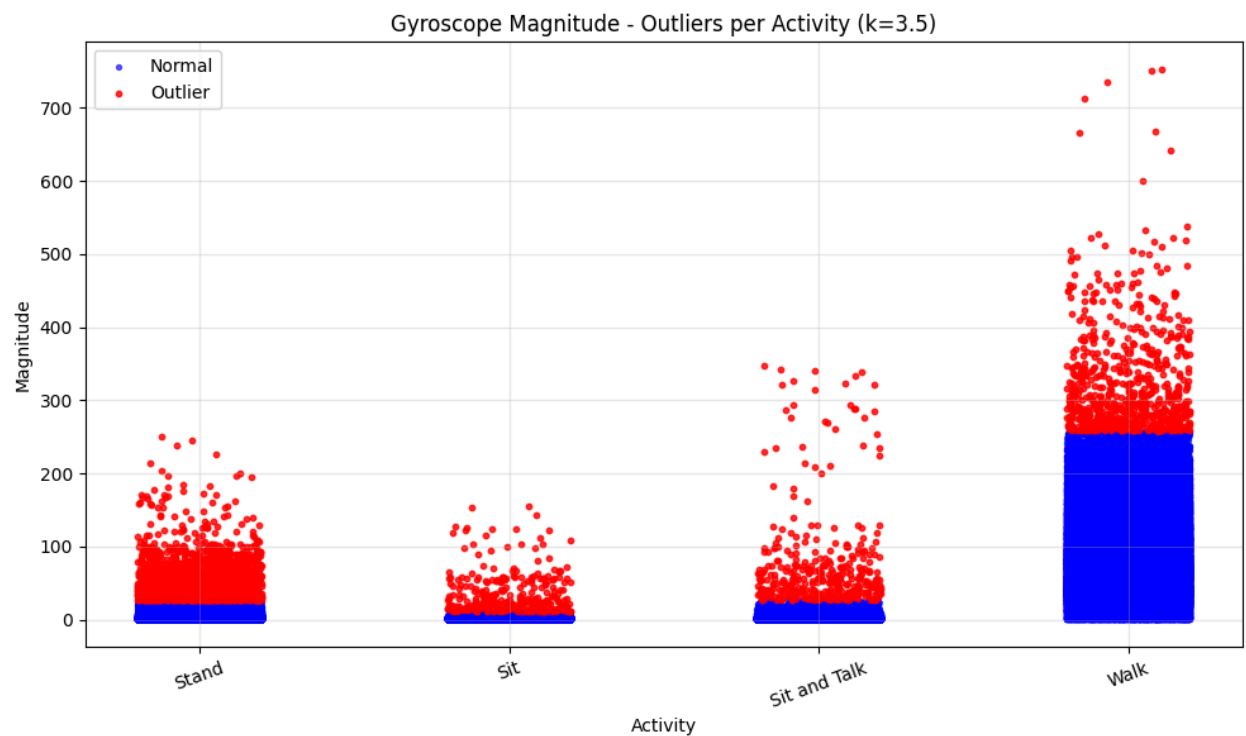
--- Accelerometer, k = 4 ---
Stand: 1047 outliers
Sit: 150 outliers
Sit and Talk: 367 outliers
Walk: 368 outliers



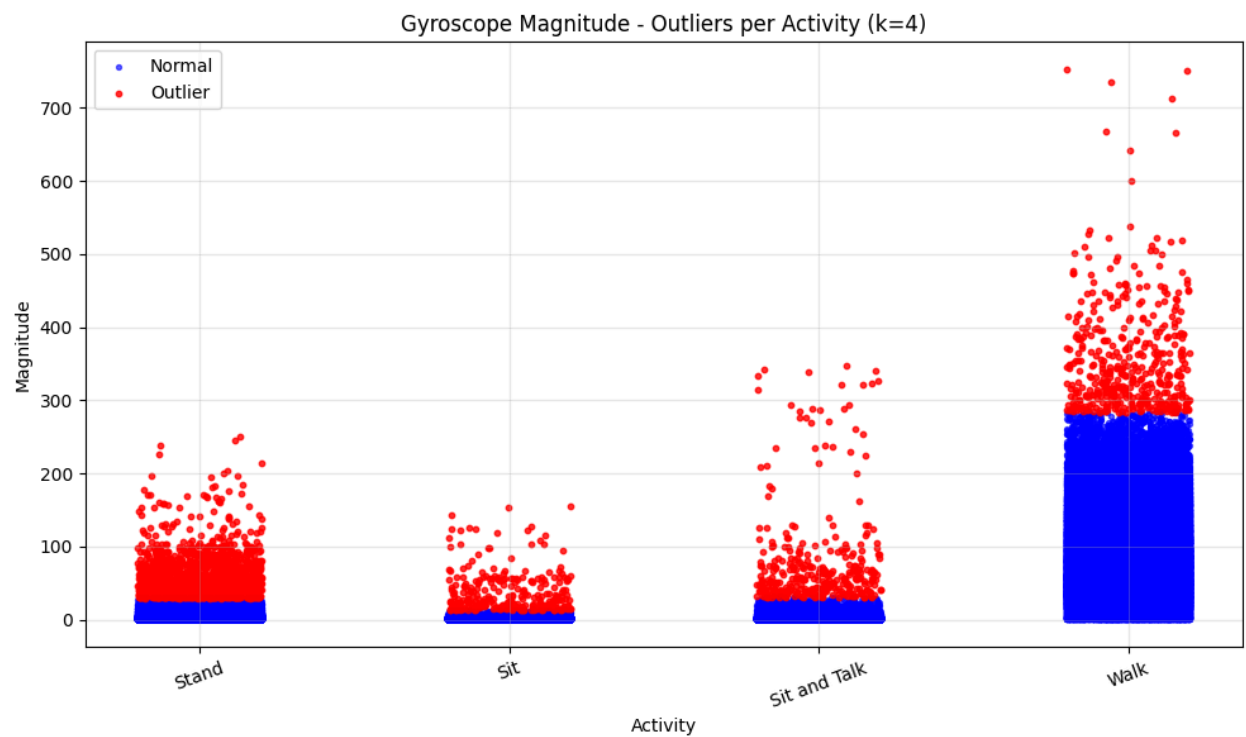
--- Gyroscope, k = 3 ---
Stand: 1963 outliers
Sit: 341 outliers
Sit and Talk: 434 outliers
Walk: 1329 outliers



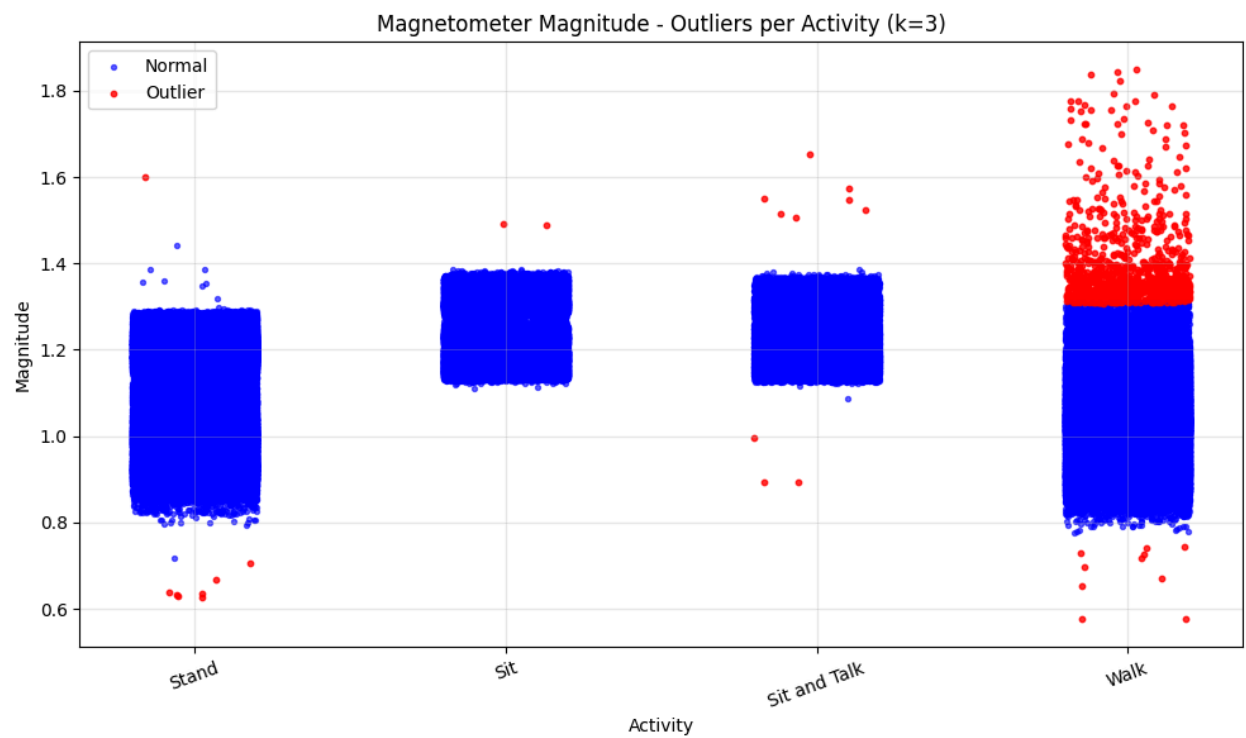
--- Gyroscope, $k = 3.5$ ---
Stand: 1681 outliers
Sit: 304 outliers
Sit and Talk: 370 outliers
Walk: 712 outliers



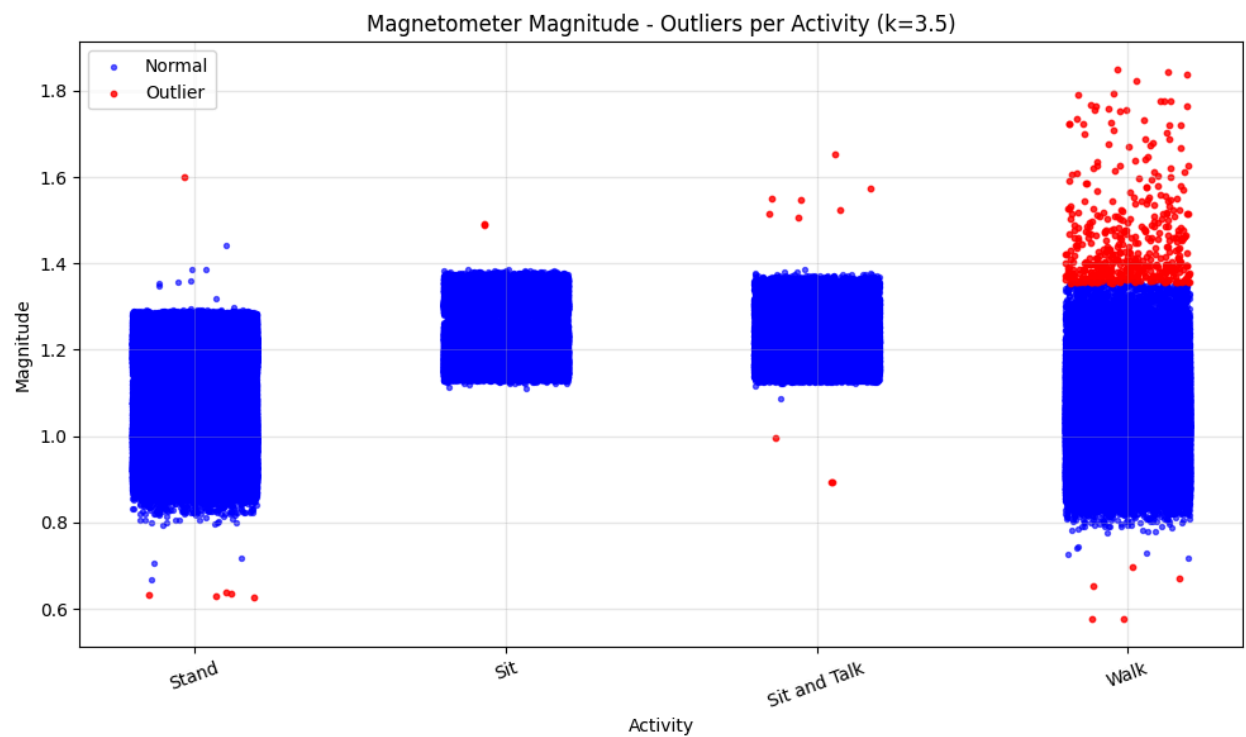
--- Gyroscope, $k = 4$ ---
Stand: 1461 outliers
Sit: 276 outliers
Sit and Talk: 329 outliers
Walk: 437 outliers



--- Magnetometer, $k = 3$ ---
Stand: 8 outliers
Sit: 2 outliers
Sit and Talk: 10 outliers
Walk: 961 outliers



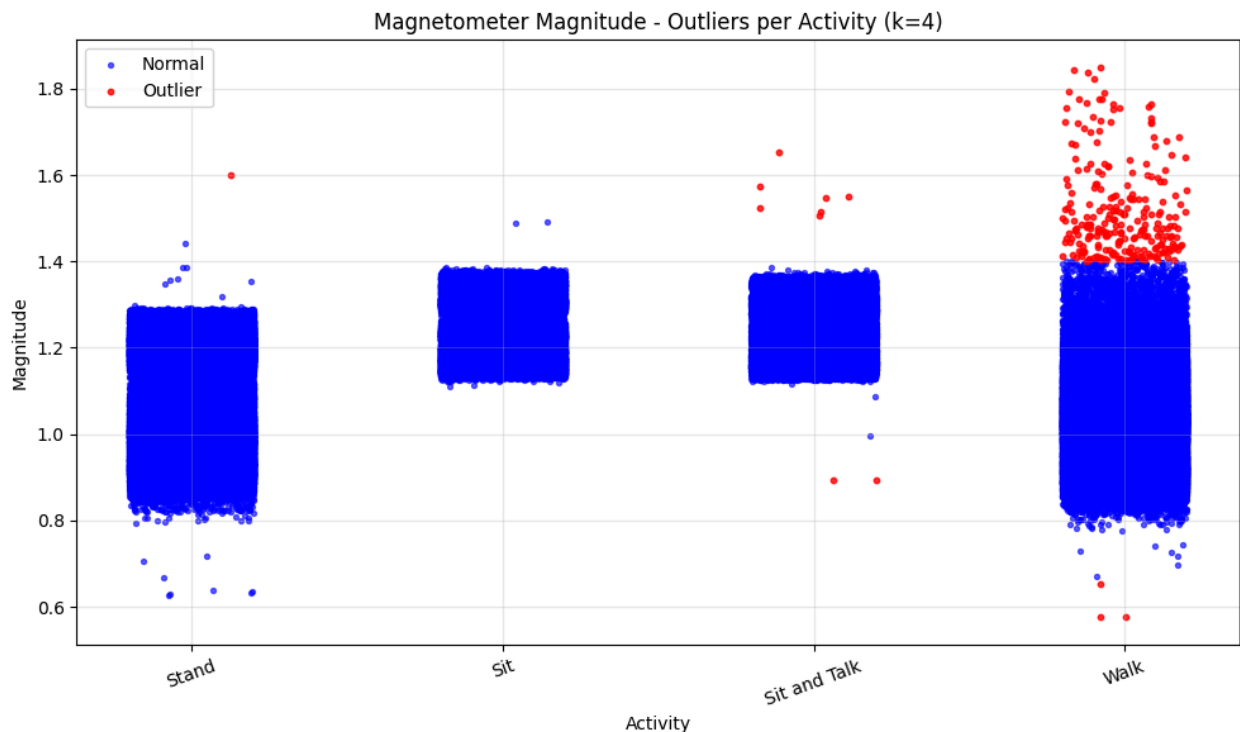
--- Magnetometer, $k = 3.5$ ---
Stand: 6 outliers
Sit: 2 outliers
Sit and Talk: 10 outliers
Walk: 485 outliers



```

--- Magnetometer, k = 4 ---
Stand: 1 outliers
Sit: 0 outliers
Sit and Talk: 9 outliers
Walk: 249 outliers

```



- Plot function (Accelerometer)

```

In [24]: k_values = [3, 3.5, 4]
activity_names = ['Stand', 'Sit', 'Sit and Talk', 'Walk']

for k in k_values:
    outlier_counts = []
    for i, valores in enumerate(accelerometer_activity_sorted):
        z = np.array(z_acc[i])
        count = np.sum(np.abs(z) > k)
        outlier_counts.append(count)

    print(f"\n--- k = {k} ---")
    for name, count in zip(activity_names, outlier_counts):
        print(f"{name}: {count} outliers")

    plt.figure(figsize=(8, 5))
    plt.title(f"Accelerometer - Outliers per Activity (k={k})")
    plt.xlabel("Activity")
    plt.ylabel("Magnitude")

    for i, valores in enumerate(accelerometer_activity_sorted):
        z = np.array(z_acc[i])
        vals = np.array(valores)

```

```

outlier_mask = np.abs(z) > k
x_pos = np.ones_like(vals) * i

plt.scatter(x_pos[~outlier_mask], vals[~outlier_mask],
            color='blue', s=8, alpha=0.6, label='Normal' if i == 0 else
plt.scatter(x_pos[outlier_mask], vals[outlier_mask],
            color='red', s=10, alpha=0.8, label='Outlier' if i == 0 else

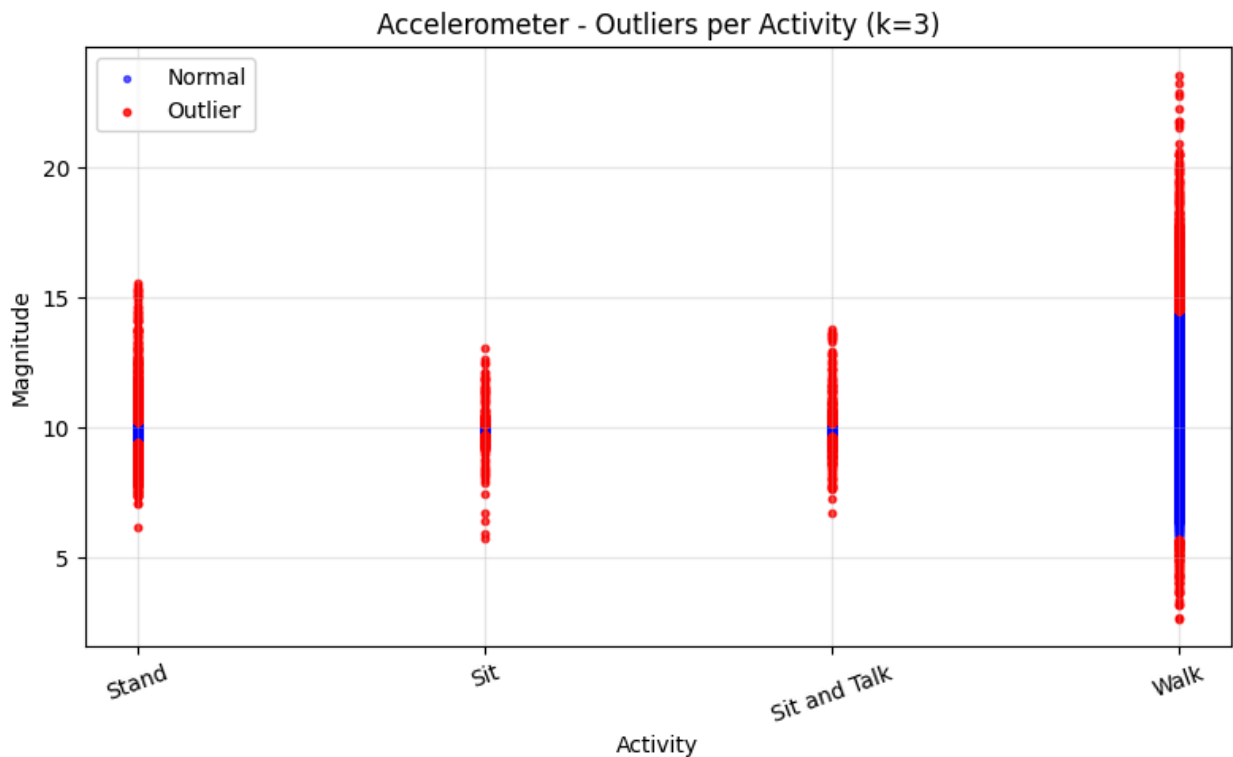
plt.xticks(range(len(accelerometer_activity_sorted)), activity_names, rota
plt.legend()
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()

```

```

--- k = 3 ---
Stand: 1401 outliers
Sit: 236 outliers
Sit and Talk: 523 outliers
Walk: 1530 outliers

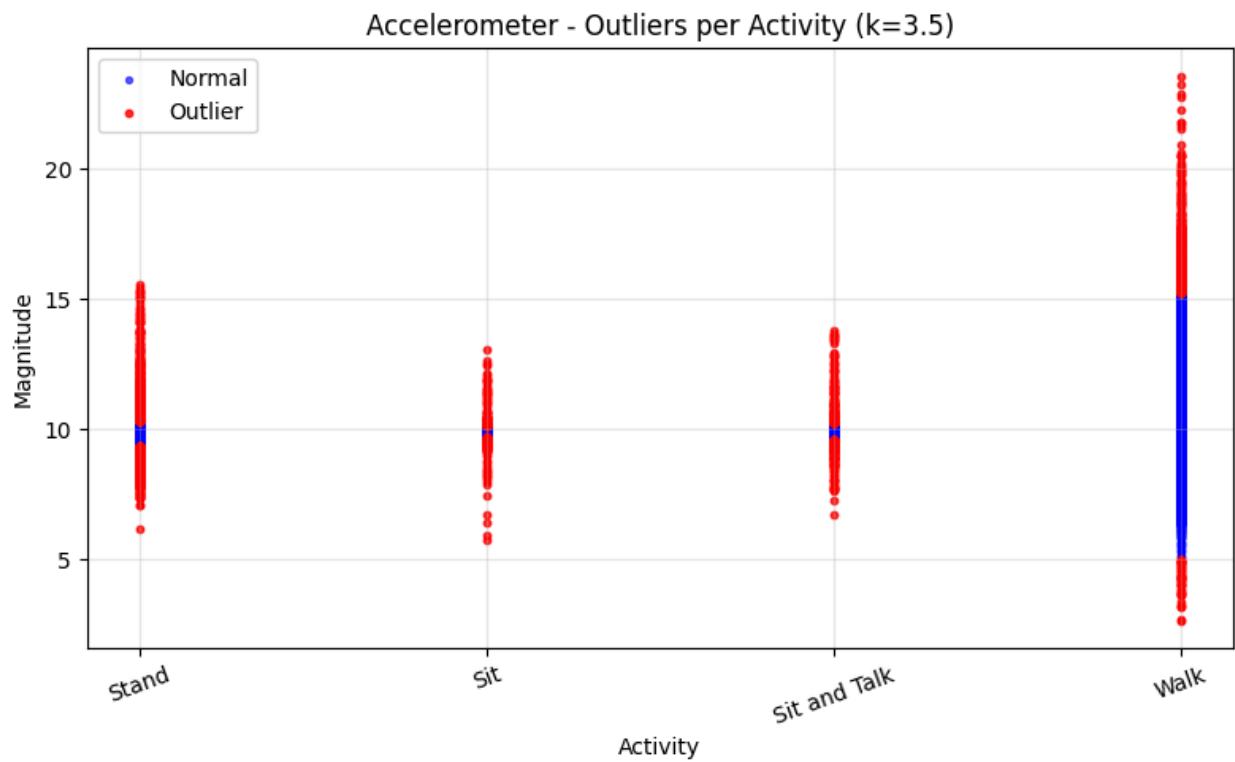
```



```

--- k = 3.5 ---
Stand: 1201 outliers
Sit: 182 outliers
Sit and Talk: 438 outliers
Walk: 700 outliers

```



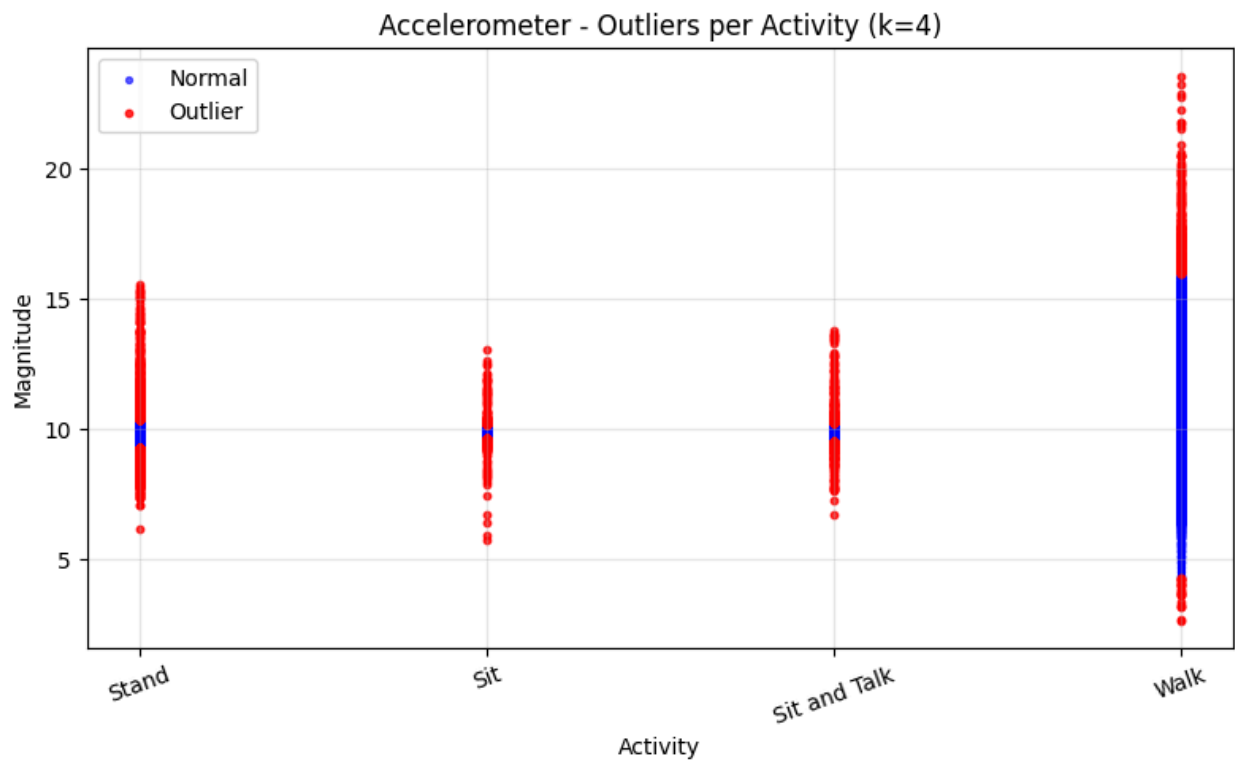
--- k = 4 ---

Stand: 1047 outliers

Sit: 150 outliers

Sit and Talk: 367 outliers

Walk: 368 outliers



- Plot function (Gyroscope)

```

In [25]: k_values = [3, 3.5, 4]
activity_names = ['Stand', 'Sit', 'Sit and Talk', 'Walk']

for k in k_values:
    outlier_counts = []
    for i, valores in enumerate(gyroscope_activity_sorted):
        z = np.array(z_gyr[i])
        count = np.sum(np.abs(z) > k)
        outlier_counts.append(count)

    print(f"\n--- k = {k} ---")
    for name, count in zip(activity_names, outlier_counts):
        print(f"{name}: {count} outliers")

    plt.figure(figsize=(8, 5))
    plt.title(f"Gyroscope - Outliers per Activity (k={k})")
    plt.xlabel("Activity")
    plt.ylabel("Magnitude")

    for i, valores in enumerate(gyroscope_activity_sorted):
        z = np.array(z_gyr[i])
        vals = np.array(valores)
        outlier_mask = np.abs(z) > k
        x_pos = np.ones_like(vals) * i

        plt.scatter(x_pos[~outlier_mask], vals[~outlier_mask],
                    color='blue', s=8, alpha=0.6, label='Normal' if i == 0 else '')
        plt.scatter(x_pos[outlier_mask], vals[outlier_mask],
                    color='red', s=10, alpha=0.8, label='Outlier' if i == 0 else '')

    plt.xticks(range(len(gyroscope_activity_sorted)), activity_names, rotation=45)
    plt.legend()
    plt.grid(alpha=0.3)
    plt.tight_layout()
    plt.show()

```

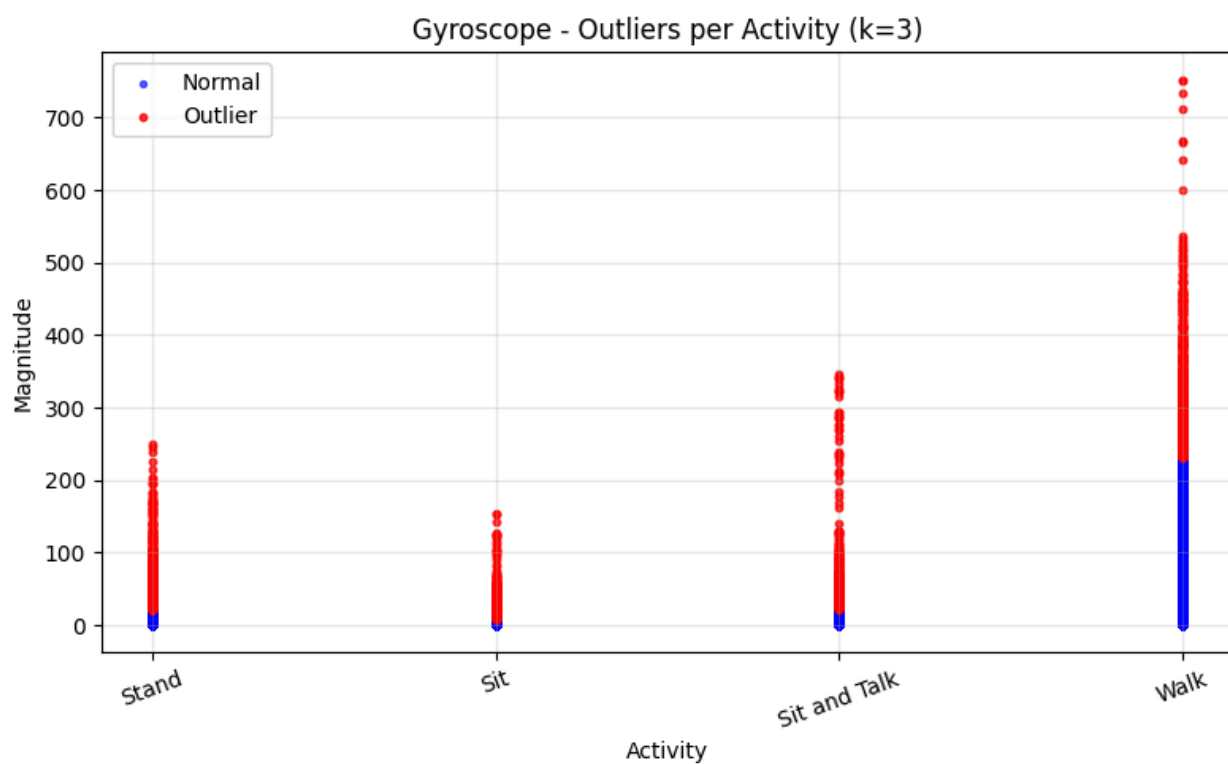
--- k = 3 ---

Stand: 1963 outliers

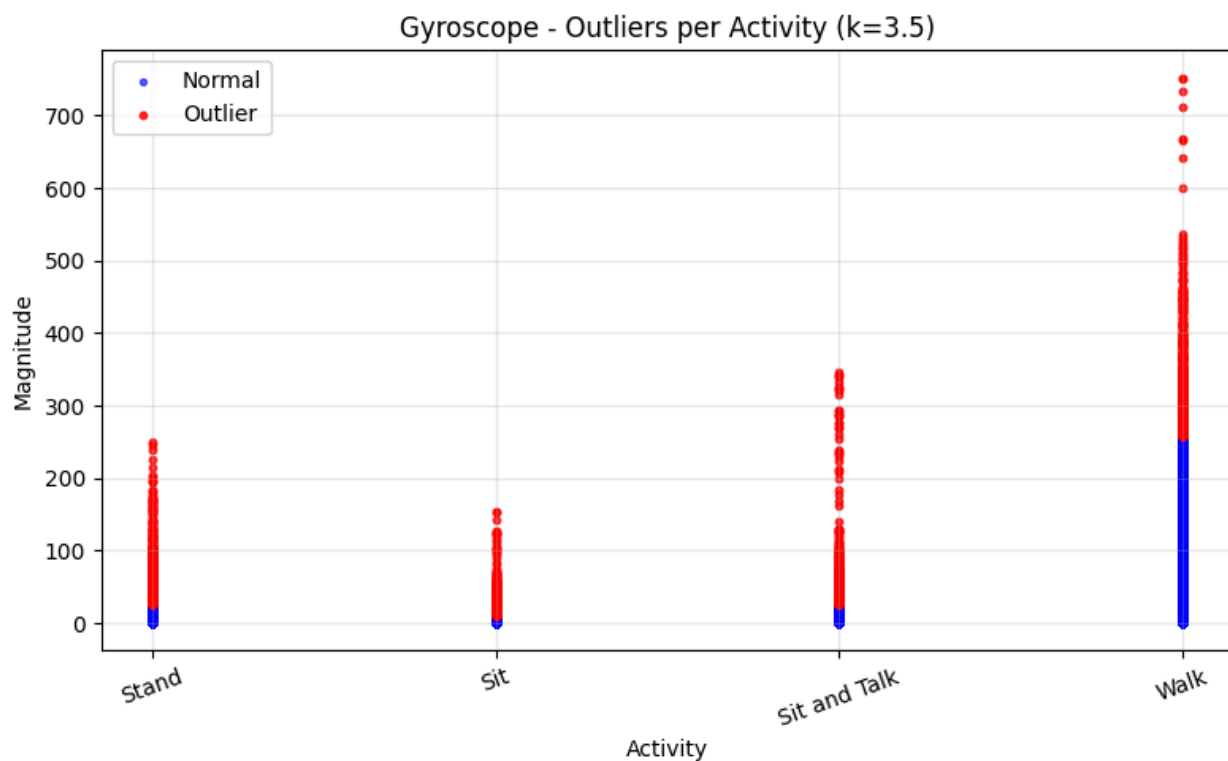
Sit: 341 outliers

Sit and Talk: 434 outliers

Walk: 1329 outliers



--- k = 3.5 ---
 Stand: 1681 outliers
 Sit: 304 outliers
 Sit and Talk: 370 outliers
 Walk: 712 outliers



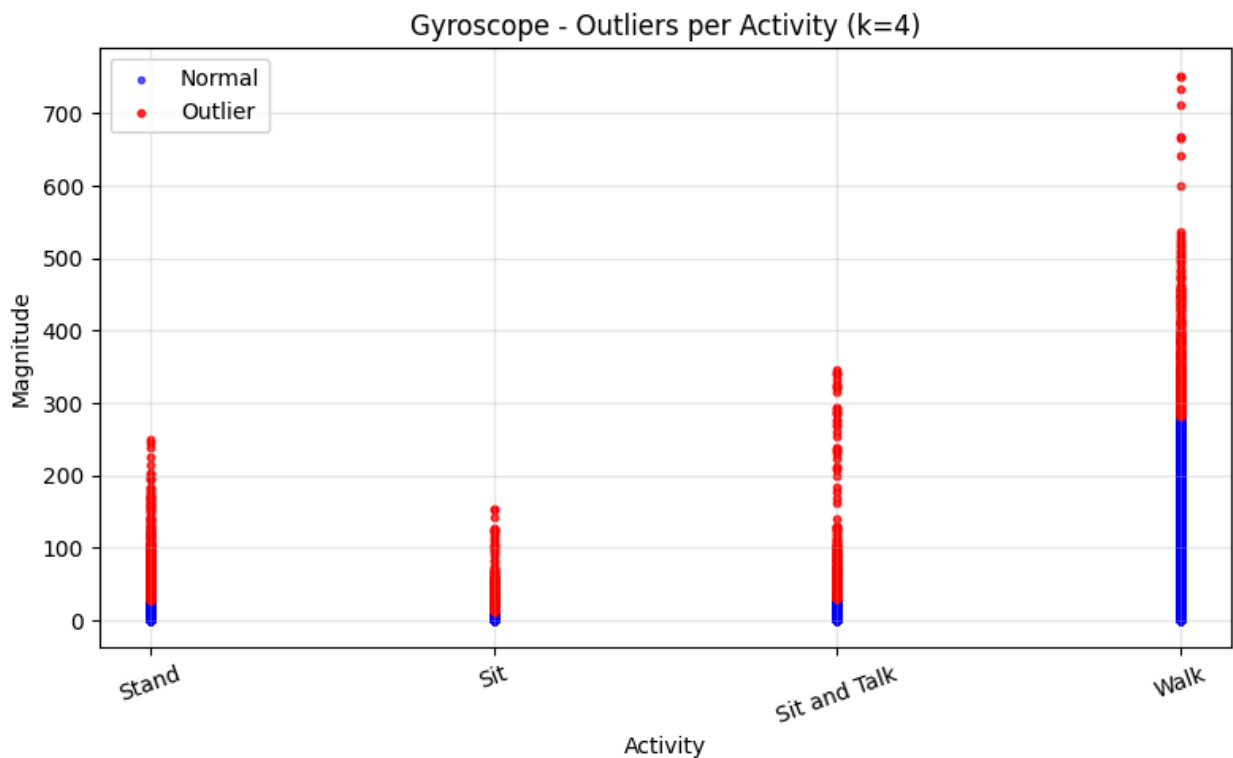
--- k = 4 ---

Stand: 1461 outliers

Sit: 276 outliers

Sit and Talk: 329 outliers

Walk: 437 outliers



- Plot function (Magnetometer)

```
In [26]: k_values = [3, 3.5, 4]
activity_names = ['Stand', 'Sit', 'Sit and Talk', 'Walk']

for k in k_values:
    outlier_counts = []
    for i, valores in enumerate(magnetometer_activity_sorted):
        z = np.array(z_mag[i])
        count = np.sum(np.abs(z) > k)
        outlier_counts.append(count)

    print(f"\n--- k = {k} ---")
    for name, count in zip(activity_names, outlier_counts):
        print(f"{name}: {count} outliers")

    plt.figure(figsize=(8, 5))
    plt.title(f"Magnetometer - Outliers per Activity (k={k})")
    plt.xlabel("Activity")
    plt.ylabel("Magnitude")

    for i, valores in enumerate(magnetometer_activity_sorted):
        z = np.array(z_mag[i])
```

```

vals = np.array(valores)
outlier_mask = np.abs(z) > k
x_pos = np.ones_like(vals) * i

plt.scatter(x_pos[~outlier_mask], vals[~outlier_mask],
            color='blue', s=8, alpha=0.6, label='Normal' if i == 0 else
plt.scatter(x_pos[outlier_mask], vals[outlier_mask],
            color='red', s=10, alpha=0.8, label='Outlier' if i == 0 else

plt.xticks(range(len(magnetometer_activity_sorted)), activity_names, rotat
plt.legend()
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()

```

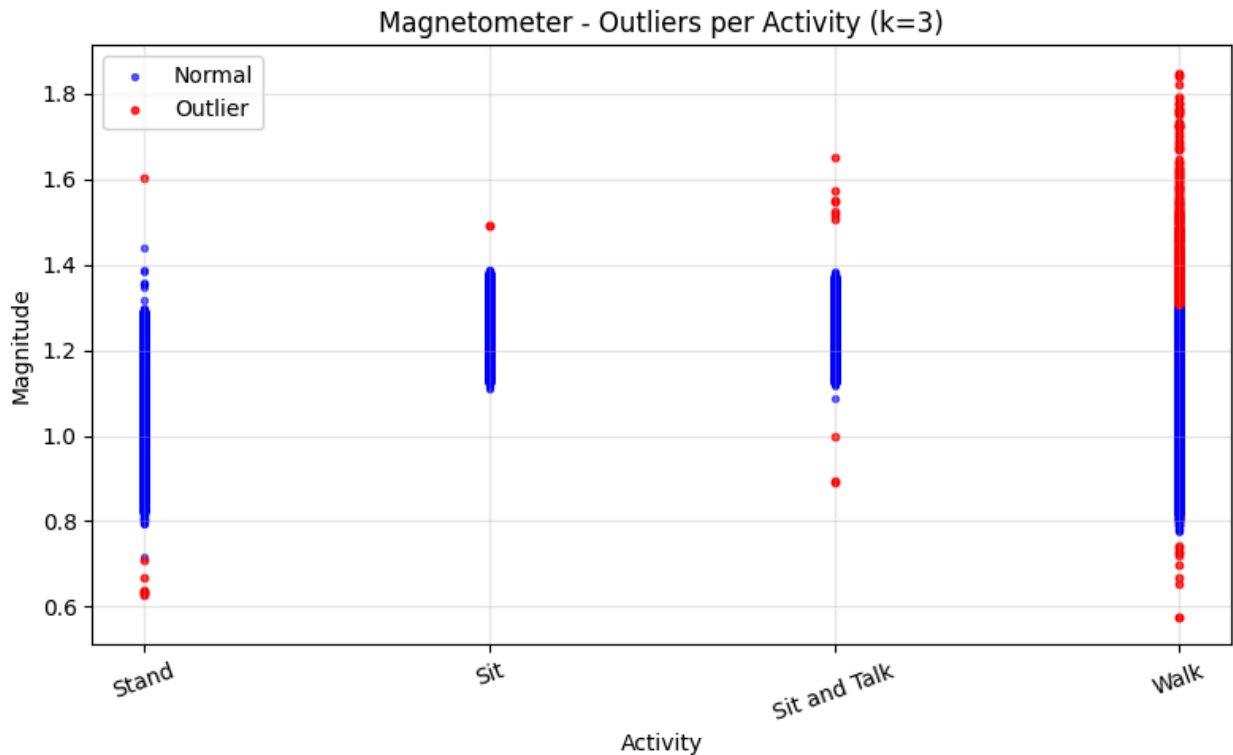
--- k = 3 ---

Stand: 8 outliers

Sit: 2 outliers

Sit and Talk: 10 outliers

Walk: 961 outliers



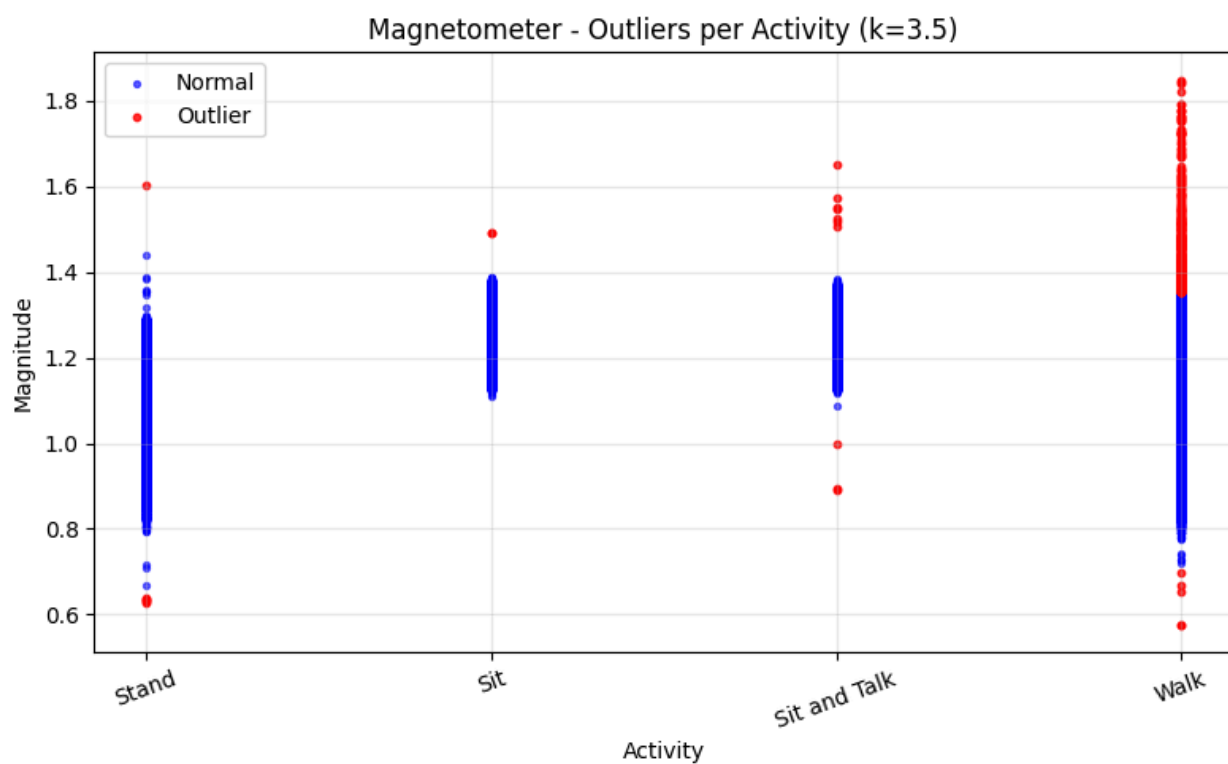
--- k = 3.5 ---

Stand: 6 outliers

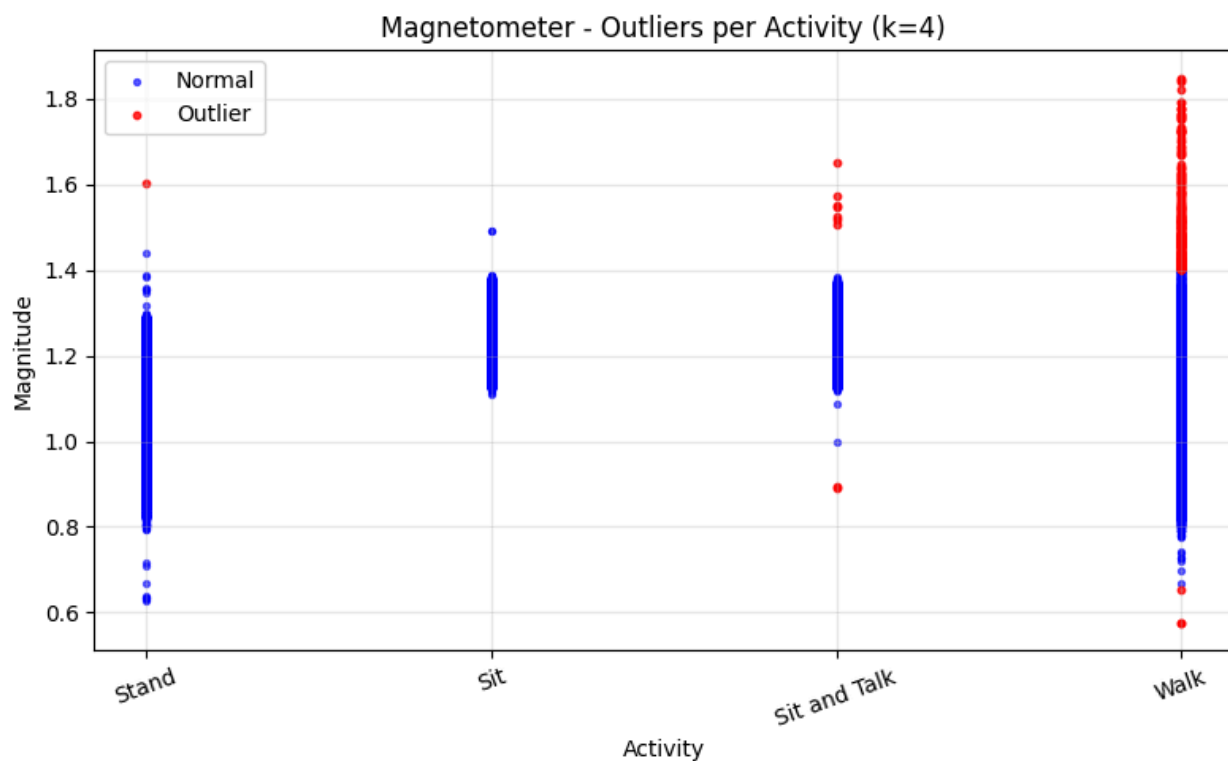
Sit: 2 outliers

Sit and Talk: 10 outliers

Walk: 485 outliers



--- k = 4 ---
 Stand: 1 outliers
 Sit: 0 outliers
 Sit and Talk: 9 outliers
 Walk: 249 outliers



3.5 - Compare the results obtained on 3.1 and 3.4 and the outliers density (IQR vs Z-Score)

```
In [27]: for i, valores in enumerate(accelerometer_activity_sorted):
        out_z = [idx for idx, z in enumerate(z_acc[i]) if abs(z) > 3]
        out_iqr = [idx for idx, v in enumerate(valores)
                    if v < lower_acc[i] or v > upper_acc[i]]

        intersec = len(set(out_z) & set(out_iqr))
        only_z = len(set(out_z) - set(out_iqr))
        only_iqr = len(set(out_iqr) - set(out_z))

        print(f"Activity {i+1}:")
        print(f"  Common outliers: {intersec}")
        print(f"  Only z-score: {only_z}")
        print(f"  Only boxplot: {only_iqr}")
```

```
Activity 1:
  Common outliers: 1401
  Only z-score: 0
  Only boxplot: 7646
```

```
Activity 2:
  Common outliers: 236
  Only z-score: 0
  Only boxplot: 296
```

```
Activity 3:
  Common outliers: 523
  Only z-score: 0
  Only boxplot: 2762
```

```
Activity 4:
  Common outliers: 1530
  Only z-score: 0
  Only boxplot: 3026
```

3.6 - K-Means Implementation

- combined_data with the modules of acc,gyr and mag

```
In [28]: combined_data_module = []

        for person_data in individuals:
            acc_mag = np.sqrt(person_data[:, 1]**2 + person_data[:, 2]**2 + person_data[:, 3]**2)
            gyro_mag = np.sqrt(person_data[:, 4]**2 + person_data[:, 5]**2 + person_data[:, 6]**2)
            mag_mag = np.sqrt(person_data[:, 7]**2 + person_data[:, 8]**2 + person_data[:, 9]**2)
            activity_label = person_data[:, 10]
            combined = np.column_stack((acc_mag, gyro_mag, mag_mag, activity_label))
            combined_data_module.append(combined)

        combined_data_module = np.vstack(combined_data_module)
```

```
In [29]: combined_data_module
```

```
Out[29]: array([[9.91434009, 1.85251737, 1.22633992, 1.      ],
               [9.85318402, 1.79826785, 1.20399498, 1.      ],
               [9.77315129, 1.75491943, 1.22718534, 1.      ],
               ...,
               [9.84891563, 2.60580398, 0.91459902, 1.      ],
               [9.87151674, 3.09543186, 0.92667445, 1.      ],
               [9.90604344, 2.78560635, 0.90053616, 1.      ]],
          shape=(794257, 4))
```

- Each point has the following format ($|a|, |g|, |m|$)

```
In [30]: def k_means(data, k=3, max_iters=100):
        X = np.array(data)
        n_samples, n_features = X.shape

        indices = np.random.choice(n_samples, k, replace=False)
        centroids = X[indices]

        for _ in range(max_iters):
            distances = np.linalg.norm(X[:, np.newaxis] - centroids, axis=2)
            labels = np.argmin(distances, axis=1)
            new_centroids = np.array([
                X[labels == j].mean(axis=0) if np.any(labels == j) else centroids[j]
                for j in range(k)
            ])
            if np.allclose(centroids, new_centroids):
                break
            centroids = new_centroids

        final_distances = np.linalg.norm(X - centroids[labels], axis=1)
        return centroids, labels, final_distances

def detecting_outliers(a_values, g_values, m_values, k=3, threshold=2):
    data = np.column_stack((a_values, g_values, m_values))
    centroids, labels, distances = k_means(data, k=k)

    stds = [np.std(distances[labels == j]) for j in range(k)]
    limits = [threshold * s for s in stds]

    outliers_idx = [i for i, (d, l) in enumerate(zip(distances, labels)) if d > limits[l]]
    return data, centroids, labels, outliers_idx

def count_outliers_per_activity(combined_matrix, activity_labels, activity_names):
    for label, name in zip(activity_labels, activity_names):
        activity_data = combined_matrix[combined_matrix[:, 3] == label]
        a_vals = activity_data[:, 0]
```

```

g_vals = activity_data[:, 1]
m_vals = activity_data[:, 2]

_, _, _, outliers_idx = detecting_outliers(a_vals, g_vals, m_vals, k=k)

print(f"{name:<15} → {len(outliers_idx)} outliers ({len(outliers_idx)/

def plot_kmeans_clusters_3d(data, centroids, labels, activity_name="Activity")
    fig = plt.figure()
    ax = fig.add_subplot(111, projection='3d')

    ax.scatter(data[:,0], data[:,1], data[:,2],
               c=labels, cmap='viridis', s=20, label='Samples')

    ax.scatter(centroids[:,0], centroids[:,1], centroids[:,2],
               c='white', edgecolors='black', s=100, label='Centroids')

    ax.set_xlabel('|a| (Accelerometer)')
    ax.set_ylabel('|g| (Gyroscope)')
    ax.set_zlabel('|m| (Magnetometer)')
    ax.set_title(f'K-Means 3D Clusters - {activity_name}')
    ax.legend()
    plt.show()

activity_names = ['Stand', 'Sit', 'Sit and Talk', 'Walk']
activity_labels = [1, 2, 3, 4]

for label, name in zip(activity_labels, activity_names):
    activity_data = combined_data_module[combined_data_module[:, 3] == label]
    a_vals = activity_data[:, 0]
    g_vals = activity_data[:, 1]
    m_vals = activity_data[:, 2]

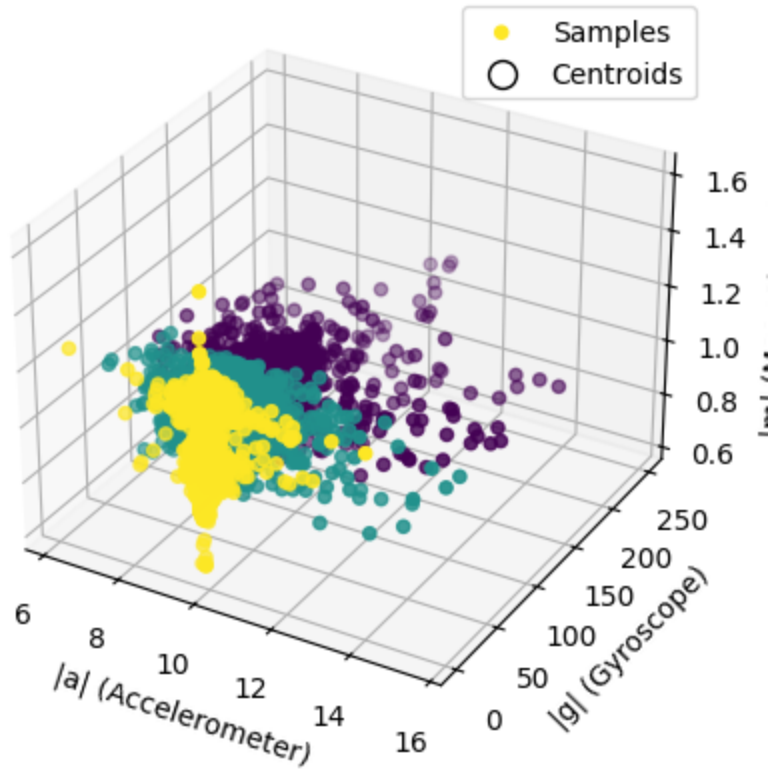
    data, centroids, labels, outliers_idx = detecting_outliers(
        a_vals, g_vals, m_vals, k=3, threshold=2
    )

    plot_kmeans_clusters_3d(data, centroids, labels, activity_name=name)

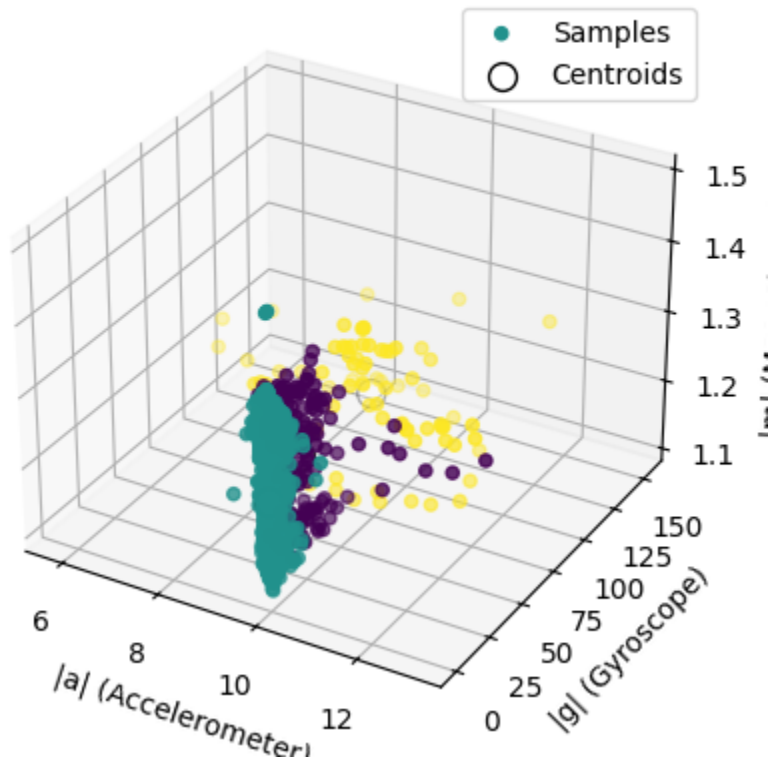
count_outliers_per_activity(combined_data_module, activity_labels, activity_na

```

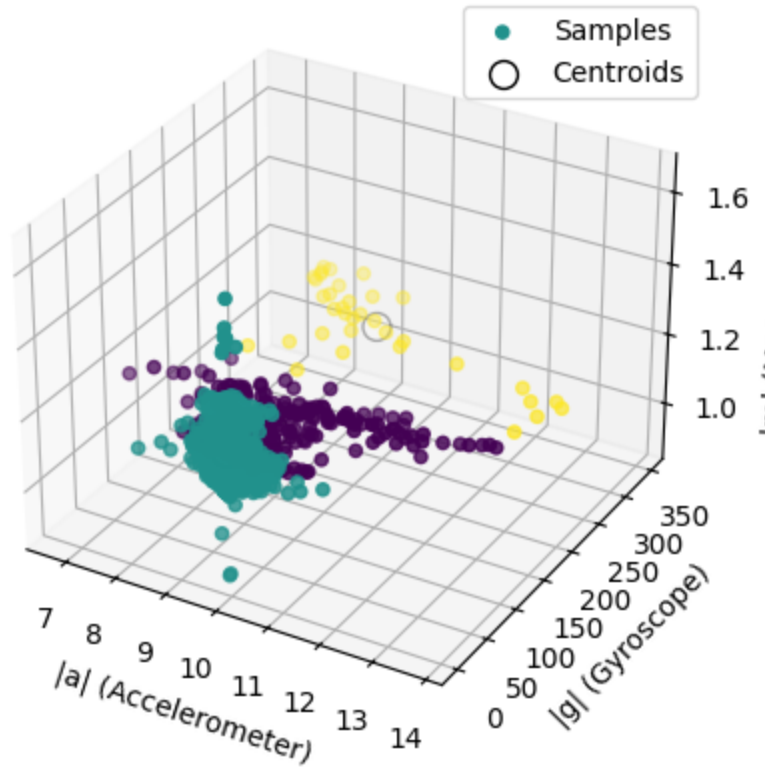
K-Means 3D Clusters - Stand



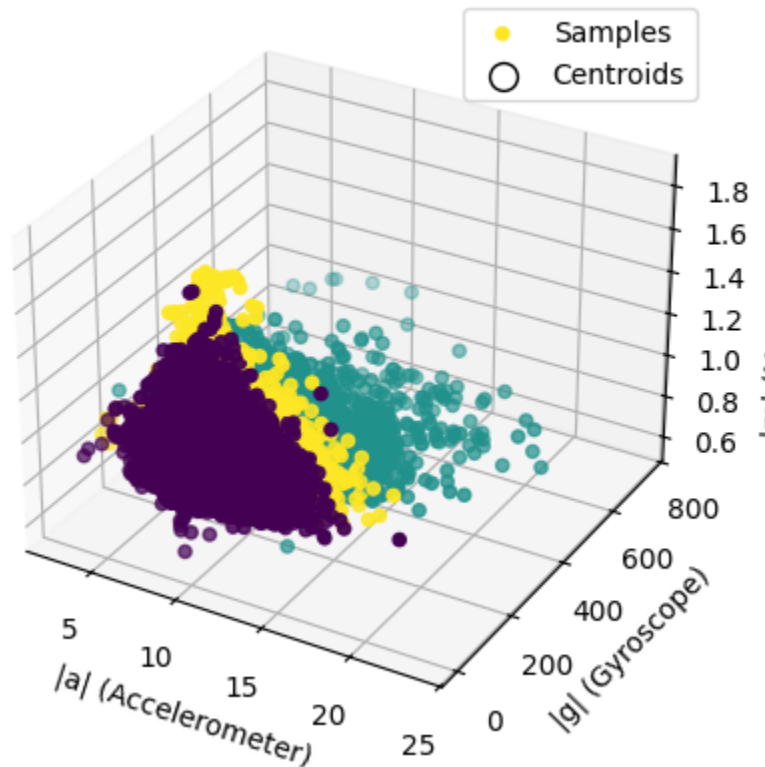
K-Means 3D Clusters - Sit



K-Means 3D Clusters - Sit and Talk



K-Means 3D Clusters - Walk



Stand	→ 10739 outliers (6.62%)
Sit	→ 2711 outliers (3.17%)
Sit and Talk	→ 2653 outliers (3.17%)
Walk	→ 45757 outliers (32.37%)

```
In [31]: print("Data: ",data,"\n")
print("Centroid: ",centroids,"\n")
print("Labels: ",labels,"\n")
# print("Outliers indices: ",outliers_idx,"\n")
```

```
Data: [[ 9.93599505 87.7540551  1.0548599 ]
[ 9.82292664 92.29748452  1.05410797]
[ 9.65611414 97.28865756  1.05350155]
...
[10.21752813 28.20139821  0.98498468]
[10.16815157 28.7876341  0.96593739]
[10.05269614 29.99084284  0.96626527]]
```

```
Centroid: [[ 9.97498239 38.9549167  1.03261483]
[10.81446763 173.0938747  1.01984269]
[10.13363008 96.10744243  1.02748463]]
```

```
Labels: [2 2 2 ... 0 0 0]
```

4 - Feature Engineering

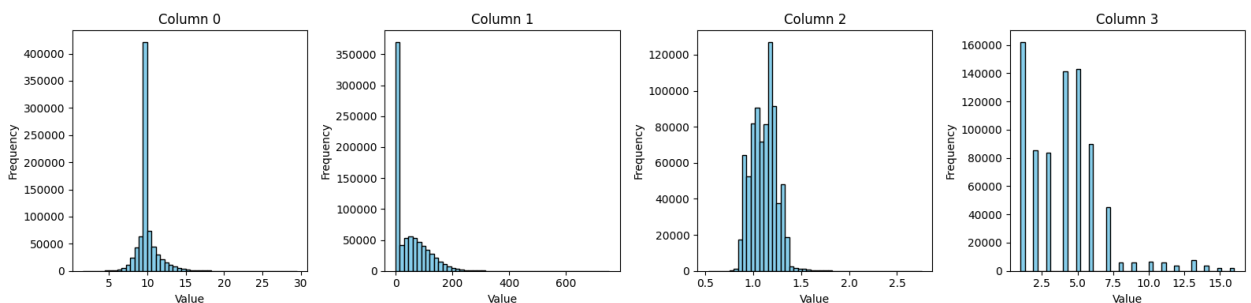
4.1 - Statistical Significance

- Data doesn't follow a normal distribution

```
In [32]: fig, axs = plt.subplots(1, 4, figsize=(16, 4))

for i in range(4):
    axs[i].hist(combined_data_module[:, i], bins=50, color='skyblue', edgecolor='black')
    axs[i].set_title(f'Column {i}')
    axs[i].set_xlabel('Value')
    axs[i].set_ylabel('Frequency')

plt.tight_layout()
plt.show()
```



- Kolmogorov-Smirnov

```
In [33]: activity_names = ['Stand', 'Sit', 'Sit and Talk', 'Walk']
activity_labels = [1, 2, 3, 4]

print("=== Kolmogorov-Smirnov test for normality ===\n")

for label, name in zip(activity_labels, activity_names):
    activity_data = combined_data_module[combined_data_module[:, 3] == label]

    acc = activity_data[:, 0]
    gyr = activity_data[:, 1]
    mag = activity_data[:, 2]

    acc = (acc - np.mean(acc)) / np.std(acc)
    gyr = (gyr - np.mean(gyr)) / np.std(gyr)
    mag = (mag - np.mean(mag)) / np.std(mag)

    ks_acc = kstest(acc, 'norm')
    ks_gyr = kstest(gyr, 'norm')
    ks_mag = kstest(mag, 'norm')

    print(f"Activity: {name}")
    print(f" Accelerometer: statistic={ks_acc.statistic:.4f}, p-value={ks_acc.pvalue:.4f}")
    print(f" Gyroscope:      statistic={ks_gyr.statistic:.4f}, p-value={ks_gyr.pvalue:.4f}")
    print(f" Magnetometer:  statistic={ks_mag.statistic:.4f}, p-value={ks_mag.pvalue:.4f}")
```

=== Kolmogorov-Smirnov test for normality ===

Activity: Stand

Accelerometer: statistic=0.2560, p-value=0.0000e+00
Gyroscope: statistic=0.3428, p-value=0.0000e+00
Magnetometer: statistic=0.1684, p-value=0.0000e+00

Activity: Sit

Accelerometer: statistic=0.1082, p-value=0.0000e+00
Gyroscope: statistic=0.3599, p-value=0.0000e+00
Magnetometer: statistic=0.1216, p-value=0.0000e+00

Activity: Sit and Talk

Accelerometer: statistic=0.1590, p-value=0.0000e+00
Gyroscope: statistic=0.3991, p-value=0.0000e+00
Magnetometer: statistic=0.1006, p-value=0.0000e+00

Activity: Walk

Accelerometer: statistic=0.0734, p-value=0.0000e+00
Gyroscope: statistic=0.0684, p-value=0.0000e+00
Magnetometer: statistic=0.0409, p-value=1.3775e-205

- Kruskal-Wallis H-test

```
In [34]: activity_labels = [1, 2, 3, 4]
activity_names = ['Stand', 'Sit', 'Sit and Talk', 'Walk']
```

```

print("=== Kruskal-Wallis Test for Mean Differences Across Activities ===\n")

for i, sensor_name in enumerate(['|a| (Accelerometer)', '|g| (Gyroscope)', '|m| (Magnetometer)']):
    groups = [combined_data_module[combined_data_module[:,3] == label][:, i] for label in range(1, 5)]

    stat, p = kruskal(*groups)
    print(f"{sensor_name}: statistic={stat:.4f}, p-value={p:.4e}")

    if p < 0.05:
        print("Difference is statistical significant.")
    else:
        print("No significant difference.\n")
    print()

```

=== Kruskal-Wallis Test for Mean Differences Across Activities ===

|a| (Accelerometer): statistic=77827.9787, p-value=0.0000e+00
Difference is statistical significant.

|g| (Gyroscope): statistic=307611.2942, p-value=0.0000e+00
Difference is statistical significant.

|m| (Magnetometer): statistic=210212.5372, p-value=0.0000e+00
Difference is statistical significant.

- Since the K-S test indicated that our data is not normally distributed, I chose the non-parametric Kruskal-Wallis test to compare the mean (median) values across activities, as it does not assume normality.

4.2 - Extract Temporal and Spectral Features

- combined_data with (x,y,z)

```

In [35]: combined_data = []
activities_to_keep = [1, 2, 3, 4]

for i, person_data in enumerate(individuals):

    mask = np.isin(person_data[:, 11], activities_to_keep)
    filtered_person_data = person_data[mask]

    if filtered_person_data.shape[0] > 0:
        ax, ay, az = filtered_person_data[:, 1], filtered_person_data[:, 2], filtered_person_data[:, 3]
        gx, gy, gz = filtered_person_data[:, 4], filtered_person_data[:, 5], filtered_person_data[:, 6]
        mx, my, mz = filtered_person_data[:, 7], filtered_person_data[:, 8], filtered_person_data[:, 9]
        activity_label = filtered_person_data[:, 11]

        participant_column = np.full(filtered_person_data.shape[0], i)

```

```

        combined = np.column_stack((participant_column, ax, ay, az, gx, gy, gz))
        combined_data.append(combined)

    if combined_data:
        combined_data = np.vstack(combined_data)
        print("combined_data shape:", combined_data.shape)
        print("Example:", combined_data[0])
    else:
        print("No data found for activities 1-4.")

```

combined_data shape: (472683, 11)

Example: [0. -1.865 9.389 2.5812 -1.1418 -1.1856 0.84998 -0.344

76

0.59839 1.0134 1.]

- 139 features per window (using x,y,z)

```

In [36]: def zero_cross_rate(x):
        return np.sum(np.diff(np.sign(x)) != 0) / len(x)

    def mean_cross_rate(x):
        mean_val = np.mean(x)
        return np.sum(np.diff(np.sign(x - mean_val)) != 0) / len(x)

    def spectral_entropy(x, fs):
        X = np.abs(rfft(x))
        psd = X**2
        psd /= np.sum(psd)
        return -np.sum(psd * np.log2(psd + 1e-12))

    def dominant_frequency(x, fs):
        X = np.abs(rfft(x))
        freqs = rfftfreq(len(x), 1/fs)
        return freqs[np.argmax(X[1:])] # not show the first element (DC)

    def energy(x):
        X = np.abs(rfft(x))
        return np.sum(X**2) / len(x)

    def extract_statistical_features(signal, fs):
        x = signal
        feats = [
            np.mean(x),
            np.median(x),
            np.std(x),
            np.var(x),
            np.sqrt(np.mean(x**2)),
            np.mean(np.diff(x)),
            stats.skew(x),
            stats.kurtosis(x),
            np.percentile(x, 75) - np.percentile(x, 25),
            zero_cross_rate(x),

```

```

        mean_cross_rate(x),
        spectral_entropy(x, fs),
        dominant_frequency(x, fs),
        energy(x)
    ]
    return np.array(feats)

```

```

def extract_physical_features(acc, gyro, fs):

```

```

    MI = np.linalg.norm(acc, axis=1)
    AI = np.mean(MI)
    VI = np.var(MI)
    SMA = np.mean(np.sum(np.abs(acc), axis=1))

    cov = np.cov(acc.T)
    eigvals = np.sort(np.linalg.eigvals(cov))[:, -1][:2]

    gravity = acc[:, 0]
    heading = np.linalg.norm(acc[:, 1:3], axis=1)
    CAGH = np.corrcoef(gravity, heading)[0, 1]

    dt = 1/fs
    vel_y = np.cumsum(acc[:, 1]) * dt
    vel_z = np.cumsum(acc[:, 2]) * dt
    AVH = np.mean(np.sqrt(vel_y**2 + vel_z**2))
    vel_x = np.cumsum(acc[:, 0]) * dt
    AVG = np.mean(vel_x)

    rot_angle = np.cumsum(gyro[:, 0]) * dt
    ARATG = np.mean(rot_angle)

    DF_acc = dominant_frequency(MI, fs)
    ENERGY_acc = energy(MI)
    AAE = np.mean([energy(acc[:, i]) for i in range(3)])
    ARE = np.mean([energy(gyro[:, i]) for i in range(3)])

    return np.array([AI, VI, SMA, *eigvals, CAGH, AVH, AVG, ARATG,
                    DF_acc, ENERGY_acc, AAE, ARE])

```

```

def extract_features_from_window_9d(window, fs):

```

```

    features_all = []

    for i in range(9):
        signal = window[:, i]
        feats = extract_statistical_features(signal, fs)
        features_all.extend(feats)

    acc = window[:, 0:3]
    gyro = window[:, 3:6]
    phys = extract_physical_features(acc, gyro, fs)

```

```

features_all.extend(phys)

return np.array(features_all)

def segment_data_combined(combined_matrix, fs=50, win_time=5, overlap=0.5):

    participants = combined_matrix[:, 0] # Column 0: participant ID
    data = combined_matrix[:, 1:10]      # Columns 1-9: sensor features
    labels = combined_matrix[:, 10]      # Column 10: activity label

    win_len = int(win_time * fs)
    step = int(win_len * (1 - overlap))
    segments = []

    for start in range(0, len(data) - win_len + 1, step):
        end = start + win_len
        seg_labels = labels[start:end]
        seg_participants = participants[start:end]

        if (np.all(seg_labels == seg_labels[0]) and
            np.all(seg_participants == seg_participants[0])):
            segments.append((data[start:end, :], seg_labels[0], seg_participants[0]))

    return segments

# each row will have 250 samples
fs = 50
segments = segment_data_combined(combined_data, fs=fs, win_time=5, overlap=0.5)

print(f"Number of valid windows: {len(segments)}")

feature_vectors = []
labels = []
participants = []

for seg_data, seg_label, seg_participant in segments:
    feat = extract_features_from_window_9d(seg_data, fs)
    feature_vectors.append(feat)
    labels.append(seg_label)
    participants.append(seg_participant)

X = np.vstack(feature_vectors)
y = np.array(labels)
participants = np.array(participants)

print("Final Shape of X:", X.shape)
print("Shape of y:", y.shape)
print("Shape of participants:", participants.shape)

```

Number of valid windows: 3512
Final Shape of X: (3512, 139)
Shape of y: (3512,)
Shape of participants: (3512,)

```
In [37]: activity_label = 4
         activity_name = 'Walk'
         fs = 50
         duration_s = 100
         window_s = 5

         mask = combined_data[:,10] == activity_label
         data = combined_data[mask]

         n_samples = int(duration_s * fs)
         n_samples = min(n_samples, len(data))

         ax, ay, az = data[:n_samples,1], data[:n_samples,2], data[:n_samples,3]
         gx, gy, gz = data[:n_samples,4], data[:n_samples,5], data[:n_samples,6]
         mx, my, mz = data[:n_samples,7], data[:n_samples,8], data[:n_samples,9]

         time = np.arange(n_samples) / fs

         fig, axs = plt.subplots(3, 1, figsize=(12, 8), sharex=True)

         # Acelerómetro
         axs[0].plot(time, ax, label='ax', alpha=0.8)
         axs[0].plot(time, ay, label='ay', alpha=0.8)
         axs[0].plot(time, az, label='az', alpha=0.8)
         axs[0].set_ylabel("Accelerometer")
         axs[0].legend(loc='upper right')
         axs[0].grid(True)

         # Giroscópio
         axs[1].plot(time, gx, label='gx', alpha=0.8)
         axs[1].plot(time, gy, label='gy', alpha=0.8)
         axs[1].plot(time, gz, label='gz', alpha=0.8)
         axs[1].set_ylabel("Gyroscope")
         axs[1].legend(loc='upper right')
         axs[1].grid(True)

         # Magnetómetro
         axs[2].plot(time, mx, label='mx', alpha=0.8)
         axs[2].plot(time, my, label='my', alpha=0.8)
         axs[2].plot(time, mz, label='mz', alpha=0.8)
         axs[2].set_ylabel("Magnetometer")
         axs[2].set_xlabel("Tempo (s)")
         axs[2].legend(loc='upper right')
         axs[2].grid(True)

         # Linhas verticais a cada janela
         for t in np.arange(0, duration_s + window_s, window_s):
             for ax in axs:
```

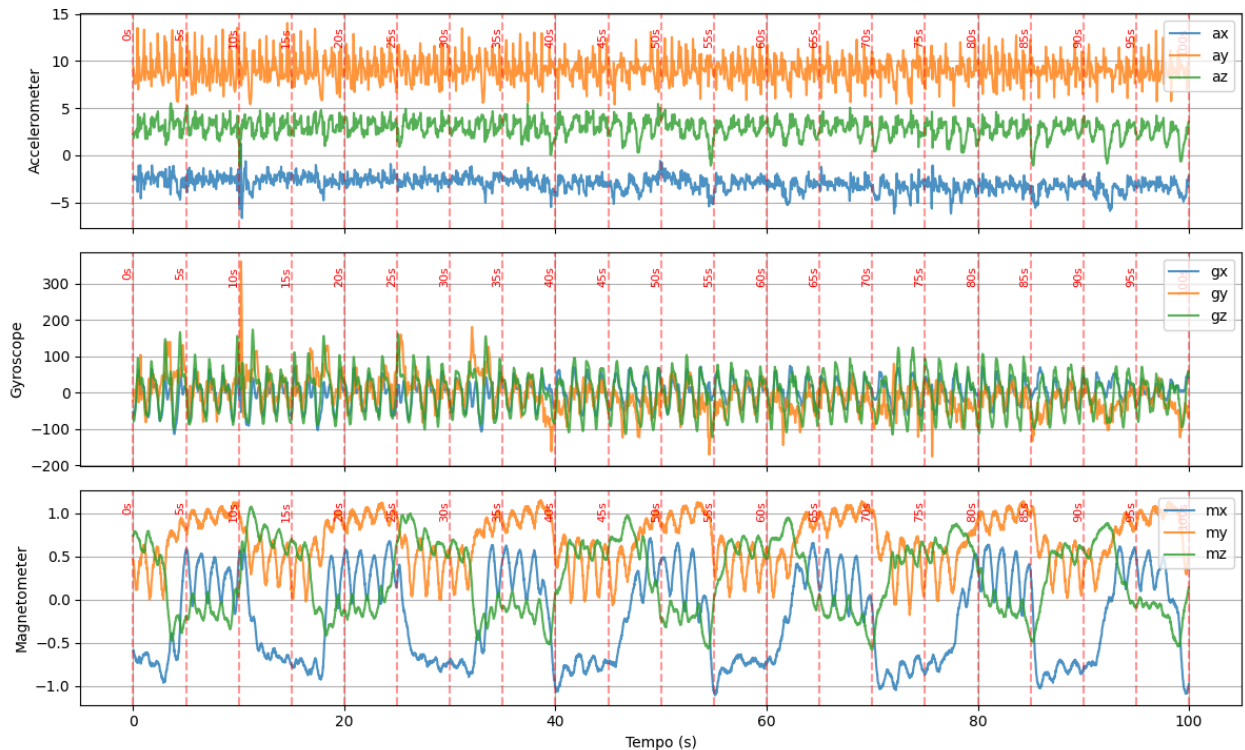
```

ax.axvline(x=t, color='red', linestyle='--', alpha=0.4)
ax.text(t, ax.get_ylim()[1]*0.9, f"{int(t)}s", color='red',
        rotation=90, va='top', ha='right', fontsize=8)

fig.suptitle(f"Signal of Activity - {activity_name} (First {duration_s}s)")
plt.tight_layout(rect=[0, 0, 1, 0.96])
plt.show()

```

Signal of Activity - Walk (First 100s)



4.3 - PCA

In [38]: `X_zscore = zscore(X,axis=0)`

```
X_zscore = np.nan_to_num(X_zscore)
```

```

print("Shape de X_zscore:", X_zscore.shape)
print("Mean (≈0):", np.mean(X_zscore, axis=0)[:5])
print("Standard Deviation (≈1):", np.std(X_zscore, axis=0)[:5])

```

Shape de X_zscore: (3512, 139)

Mean (≈0): [-1.84109877e-16 -6.25473319e-15 -6.57693338e-16 4.26718266e-16
7.79392686e-16]

Standard Deviation (≈1): [1. 1. 1. 1. 1.]

In [39]: `def pca(data,n_components):`
`mean = np.mean(data,axis=0)`
`centered_data = data-mean`

```
covariance_matrix = np.cov(centered_data,rowvar=False)

eigenvalues,eigenvectors = np.linalg.eigh(covariance_matrix)

sorted_indices = np.argsort(eigenvalues)[::-1]
eigenvalues = eigenvalues[sorted_indices]
eigenvectors = eigenvectors[:,sorted_indices]

principal_components = eigenvectors[:, :n_components]

transformed_data = np.dot(centered_data,principal_components)

return transformed_data,principal_components,eigenvalues
```

```
In [40]: n_components = 3

X_pca,principal_components, eigenvalues = pca(X_zscore,n_components)

print("Original Shape of X:", X.shape)
print("Shape after PCA of X:", X_pca.shape)
print("\nEigenvalues: ",eigenvalues)
```

Original Shape of X: (3512, 139)
Shape after PCA of X: (3512, 3)

```
Eigenvalues: [ 4.94585636e+01  1.33689746e+01  7.08615907e+00  5.34709271e+00
 3.72064974e+00  3.55137481e+00  3.21378249e+00  2.77182676e+00
 2.26654879e+00  2.07606174e+00  2.03946701e+00  1.83272450e+00
 1.76538659e+00  1.65513272e+00  1.55025367e+00  1.50942771e+00
 1.36026114e+00  1.29313636e+00  1.24129870e+00  1.21286864e+00
 1.21010876e+00  1.18888293e+00  1.10102523e+00  1.08522055e+00
 1.06426203e+00  1.03955487e+00  1.02889082e+00  9.46883771e-01
 9.24020494e-01  9.07581214e-01  8.94221288e-01  8.49293498e-01
 8.20518325e-01  7.87924862e-01  7.20052702e-01  6.91078881e-01
 6.67231458e-01  6.47720758e-01  6.37434633e-01  6.21899270e-01
 5.98244235e-01  5.95721925e-01  5.69316614e-01  5.44548680e-01
 5.19726275e-01  4.76217381e-01  4.61597276e-01  4.35649335e-01
 4.26329676e-01  4.13965662e-01  3.95732182e-01  3.94304464e-01
 3.79406479e-01  3.60246931e-01  3.39280535e-01  3.36463170e-01
 3.27191272e-01  3.17722393e-01  3.12690737e-01  2.91303974e-01
 2.78655979e-01  2.63402235e-01  2.54047521e-01  2.47450049e-01
 2.32631914e-01  2.09126194e-01  1.75227855e-01  1.67186295e-01
 1.61712624e-01  1.45925506e-01  1.42453564e-01  1.31907155e-01
 1.28643741e-01  1.22938344e-01  1.15833396e-01  1.13091247e-01
 1.04198554e-01  9.97353893e-02  9.50846236e-02  8.92136684e-02
 8.27883754e-02  7.90899380e-02  6.99414715e-02  6.66610084e-02
 6.23715382e-02  5.75404289e-02  4.96051012e-02  4.75525508e-02
 4.43927719e-02  4.41342639e-02  4.31352704e-02  3.89023370e-02
 3.66146834e-02  3.30352557e-02  3.10418701e-02  2.84131558e-02
 2.70155855e-02  2.61507696e-02  2.45800873e-02  2.28542475e-02
 2.16307488e-02  2.05035497e-02  1.82923718e-02  1.72059262e-02
 1.54818941e-02  1.27618144e-02  1.19586489e-02  9.35627897e-03
 8.37586346e-03  7.02364365e-03  6.32402647e-03  5.77779512e-03
 5.24639431e-03  4.49198985e-03  4.30728421e-03  4.16250393e-03
 3.84452008e-03  3.28841183e-03  3.02660446e-03  2.75808384e-03
 2.24572778e-03  1.77834774e-03  1.40190262e-03  9.84041351e-04
 5.69250416e-04  4.95634053e-04  4.23380886e-04  3.22838213e-04
 2.71990516e-04  2.70738495e-04  9.95186300e-05  7.26967816e-05
 4.95814015e-05  2.64394979e-05  9.78149154e-06  6.16438765e-06
 4.25844027e-10  4.55223435e-16 -2.71510610e-15]
```

4.4 - Importance of each principal component

- Graphical visualization of the principal components

```
In [41]: eigenvalues = np.array(eigenvalues)

sorted_indices = np.argsort(eigenvalues)[::-1]
eigenvalues_sorted = eigenvalues[sorted_indices]

explained_variance_ratio = eigenvalues_sorted / np.sum(eigenvalues_sorted) * 1

top_n = 10
```

```

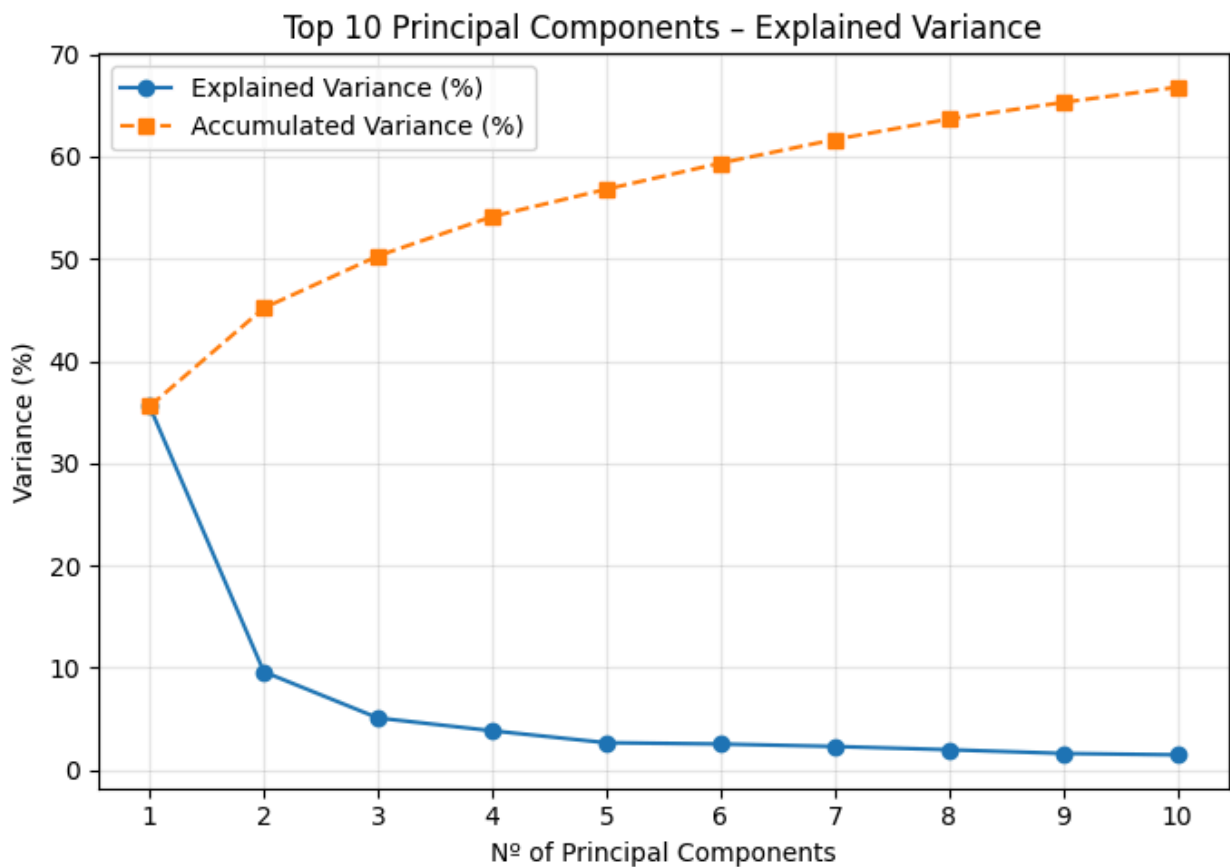
top_eigenvalues = eigenvalues_sorted[:top_n]
top_variance = explained_variance_ratio[:top_n]

cumulative_top = np.cumsum(top_variance)

plt.figure(figsize=(7, 5))
plt.plot(range(1, top_n + 1), top_variance, marker='o', label='Explained Variance (%)')
plt.plot(range(1, top_n + 1), cumulative_top, marker='s', linestyle='--', label='Accumulated Variance (%)')

plt.title("Top 10 Principal Components – Explained Variance")
plt.xlabel("Nº of Principal Components")
plt.ylabel("Variance (%)")
plt.xticks(range(1, top_n + 1))
plt.grid(True, alpha=0.3)
plt.legend()
plt.tight_layout()
plt.show()

```



```

In [42]: print("=== Top 5 Principal Components ===")
for i, (eig, var) in enumerate(zip(top_eigenvalues, top_variance), start=1):
    print(f"PC{i}: eigenvalue = {eig:.4f} | explained variance = {var:.2f}%")

print(f"\nAccumulated Variance (Top 5): {np.sum(top_variance[:5]):.2f}%")

```

```

=== Top 5 Principal Components ===
PC1: eigenvalue = 49.4586 | explained variance = 35.57%
PC2: eigenvalue = 13.3690 | explained variance = 9.62%
PC3: eigenvalue = 7.0862 | explained variance = 5.10%
PC4: eigenvalue = 5.3471 | explained variance = 3.85%
PC5: eigenvalue = 3.7206 | explained variance = 2.68%
PC6: eigenvalue = 3.5514 | explained variance = 2.55%
PC7: eigenvalue = 3.2138 | explained variance = 2.31%
PC8: eigenvalue = 2.7718 | explained variance = 1.99%
PC9: eigenvalue = 2.2665 | explained variance = 1.63%
PC10: eigenvalue = 2.0761 | explained variance = 1.49%

```

Accumulated Variance (Top 5): 66.79%

- Features that are needed to explain 75% of the data

```

In [43]: cumulative_variance = np.cumsum(explained_variance_ratio)

dimensions_needed = np.argmax(cumulative_variance >= 75) + 1

print(f"\n--- Dimensions for 75% Variance ---")
print(f"To explain 75% of the variance, you would need {dimensions_needed} dim")
print(f"The first {dimensions_needed} components explain {cumulative_variance[dimensions_needed - 1]:.2%} of the variance.")

--- Dimensions for 75% Variance ---
To explain 75% of the variance, you would need 17 dimensions.
The first 17 components explain 75.21% of the variance.

```

```

In [44]: # Statistical Features
axes = ['ax', 'ay', 'az', 'gx', 'gy', 'gz', 'mx', 'my', 'mz']
stat_feats = ['mean', 'median', 'std', 'var', 'rms', 'mean_diff',
              'skew', 'kurtosis', 'iqr', 'zcr', 'mcr',
              'spec_entropy', 'dom_freq', 'energy']

feature_names = [f"{axis}_{feat}" for axis in axes for feat in stat_feats]

# Physical Features
phys_feats = ['AI', 'VI', 'SMA', 'eig1', 'eig2', 'CAGH',
              'AVH', 'AVG', 'ARATG', 'DF_acc', 'ENERGY_acc', 'AAE', 'ARE']
feature_names.extend(phys_feats)

print(f"Total de features: {len(feature_names)}")
print("Example:", feature_names[:10])

```

Total de features: 139

Example: ['ax_mean', 'ax_median', 'ax_std', 'ax_var', 'ax_rms', 'ax_mean_diff', 'ax_skew', 'ax_kurtosis', 'ax_iqr', 'ax_zcr']

```

In [45]: for i in range(principal_components.shape[1]):
          print(f"\n=== Principal Component {i+1} ===")

          loadings = principal_components[:, i]

```

```

sorted_idx = np.argsort(np.abs(loadings))[::-1][:10]

for idx in sorted_idx:
    print(f"{feature_names[idx]:<25} loading = {loadings[idx]:.4f}")

=== Principal Component 1 ===
gy_rms          loading = 0.1325
gy_std          loading = 0.1315
gz_rms          loading = 0.1309
gz_std          loading = 0.1308
az_std          loading = 0.1302
gy_iqr          loading = 0.1291
ax_std          loading = 0.1288
my_mcr          loading = -0.1288
az_iqr          loading = 0.1281
mz_mcr          loading = -0.1279

=== Principal Component 2 ===
az_rms          loading = -0.1833
az_median       loading = -0.1754
az_mean         loading = -0.1752
my_mean         loading = 0.1748
ay_median       loading = 0.1740
ay_mean         loading = 0.1721
my_median       loading = 0.1711
ay_energy       loading = 0.1692
az_energy       loading = -0.1684
ay_rms          loading = 0.1653

=== Principal Component 3 ===
mx_rms          loading = -0.2833
mx_energy       loading = -0.2645
mx_mean         loading = 0.2534
mx_median       loading = 0.2475
ax_rms          loading = -0.2420
AVG             loading = 0.2377
ax_mean         loading = 0.2373
ax_median       loading = 0.2356
AVH             loading = 0.2331
ax_energy       loading = -0.2294

```

4.4.1 - Instant of my choice

- Perform manually the center of the data and the transformation that happened on the PCA
- They match so the mathematical steps are correct.

In [46]: `instant_index = 0`

```

single_instant_scaled = X_zscore[instant_index]

data_mean = np.mean(X_zscore, axis=0)

single_instant_centered = single_instant_scaled - data_mean

compressed_features_manual = np.dot(single_instant_centered, principal_componen

print(f"--- Example for Instant (row {instant_index}) ---")
print(f"\nOriginal Scaled Features (first 5 of 139):")
print(single_instant_scaled[:5])

print(f"\n--- After Compression ---")
print(f"New 'compressed' features (manual calc): {compressed_features_manual}")

print(f"Value from your pre-computed X_pca array: {X_pca[instant_index]}")
print(f"\nSuccess! The results match.")

```

--- Example for Instant (row 0) ---

Original Scaled Features (first 5 of 139):

```
[ 0.67100438  0.66569702 -0.53777858 -0.42832668 -0.75486569]
```

--- After Compression ---

New 'compressed' features (manual calc): [-1.13208541 4.06711953 0.02501034]

Value from your pre-computed X_pca array: [-1.13208541 4.06711953 0.02501034]

Success! The results match.

4.4.2

Advantages:

- The PCA combats the high-dimensional space of 139 features that I had, and reducing it to only 17 features that explain 75% of the feature set helps the future implementation of KNN, that on a high-dimensional space is too slow and perform poorly. So the KNN, that has to calculate all the distances, will be much faster which results on a speeding training.
- With 139 features the possibility of them being highly correlated is likely, instead of that, with principal components that are mathematically uncorrelated this problem will be solved.
- The principal components with very small eigenvalues, often represent noise in the data, will be throw away benefiting the model.

- Helps visualize the data.

Limitations:

- As in PCA i selected three principal components, that are a complex combination of tyhe 139 features, they become mathematical abstract meaning they cannot be interpretable as with the 139 features.
- I lost information because i only keep the 75% of the variance, so i lose 25% of the information, that can be noise or can actually be useful information.
- Assumes linearity and if in the case of the data being with a non-linear structure, like a spiral, the PCA will not be able to "unroll" it properly.
- PCA is sensitive to scaling, but i did the zscore before apply it. If i didn't the zscore any feature with a very large range would dominated the covariance matrix and the PCA would have just discovered taht feature.

4.5 - Fisher Score and ReliefF

- Fisher Score

FS = Variance between classes/Variance on classes

```
In [47]: def fisher_score(X, y):

    n_samples, n_features = X.shape
    scores = []

    classes = np.unique(y)
    K = len(classes)

    # for each feature
    for j in range(n_features):
        feature_col = X[:, j]
        overall_mean = np.mean(feature_col) # calculate the mean of the featur

    # for each activity/class (1,2,3,4)
    ss_between = 0
    for k in classes:
        X_k = feature_col[y == k]
        n_k = len(X_k)
        mean_k = np.mean(X_k) # mean of the activity/class
        ss_between += n_k * ((mean_k - overall_mean) ** 2) # the formula
```

```

# Degrees of freedom between groups
df_between = K - 1
var_between = ss_between / df_between # transform the square sum bet

# same applies to within each activity/class
ss_within = 0
for k in classes:
    X_k = feature_col[y == k]
    mean_k = np.mean(X_k)
    ss_within += np.sum((X_k - mean_k) ** 2)

df_within = n_samples - K
var_within = ss_within / df_within

if var_within == 0:
    score = 0
else:
    score = var_between / var_within

scores.append(score)

return np.array(scores)

print("Fisher Scores:")
fs_scores = fisher_score(X_zscore, y)

df_fisher = pd.DataFrame({
    'Feature': feature_names,
    'FisherScore': fs_scores
})

df_fisher = df_fisher.sort_values(by='FisherScore', ascending=False)

print("--- Top 10 Features ---")
print(df_fisher.head(10))

```

Fisher Scores:

--- Top 10 Features ---

	Feature	FisherScore
27	ay_energy	15986.656598
99	my_median	9293.650131
98	my_mean	9236.043228
108	my_mcr	8705.305870
28	az_mean	8205.529415
29	az_median	8048.864381
18	ay_rms	8004.529307
32	az_rms	7990.670431
14	ay_mean	7308.911512
60	gy_rms	7110.290498

- ReliefF

```

In [48]: def reliefF(X, y, m_iterations, k=1):
    n_samples, n_features = X.shape
    weights = np.zeros(n_features) # reliefF initializes the scores list with
    classes = np.unique(y)

    # max and min values of each feature (array (139,))
    max_f = np.max(X, axis=0)
    min_f = np.min(X, axis=0)

    feature_range = (max_f - min_f) + 1e-10

    # proportion of each class/activity on the data
    class_priors = {}
    for c in classes:
        class_priors[c] = np.sum(y == c) / n_samples

    # distances for all the points to a given sample
    def find_all_distances(sample):
        return np.sum((X - sample)**2, axis=1)

    # calculate the difference/distance for a unic feature between two samples
    def diff(A, idx_1, idx_2):
        val = np.abs(X[idx_1, A] - X[idx_2, A]) / feature_range[A] # normaliz
        return val

    # for all the samples
    for i in range(m_iterations):
        if i % 100 == 0:
            print(f"ReliefF Iteration: {i}/{m_iterations}")

        idx_R = random.randint(0, n_samples - 1)
        sample_R = X[idx_R]
        label_R = y[idx_R]

        distances = find_all_distances(sample_R)
        distances[idx_R] = np.inf

        hit_mask = (y == label_R) # to identify the samples that are from th
        distances_hit = np.where(hit_mask, distances, np.inf) # if the sample
        idx_H = np.argmin(distances_hit) # if the sample is not from the sa

        miss_indices = {} # save the index of the nearest samples that are n
        for c in classes:
            if c == label_R: # only for the "enemy" class
                continue

            miss_mask = (y == c)
            distances_miss = np.where(miss_mask, distances, np.inf) # only "enem
            miss_indices[c] = np.argmin(distances_miss) # find the lower dista

    # for each feature
    for A in range(n_features):
        term_hit = diff(A, idx_R, idx_H) # calculate the distance for featu

```

```

        term_miss = 0
        for c in classes:
            if c == label_R:
                continue
            term_miss += class_priors[c] * diff(A,idx_R,miss_indices[c])

        weights[A] += -term_hit + term_miss

    return weights / m_iterations

n_iter = X_zscore.shape[0]

print(f"Calculating ReliefF Scores ({n_iter} ")
relieff_scores = reliefF(X_zscore, y, m_iterations=n_iter)

df_relieff = pd.DataFrame({
    'Feature': feature_names,
    'ReliefF_Score': relieff_scores
})

df_relieff = df_relieff.sort_values(by='ReliefF_Score', ascending=False)

```

Calculating ReliefF Scores (3512

ReliefF Iteration: 0/3512

ReliefF Iteration: 100/3512

ReliefF Iteration: 200/3512

ReliefF Iteration: 300/3512

ReliefF Iteration: 400/3512

ReliefF Iteration: 500/3512

ReliefF Iteration: 600/3512

ReliefF Iteration: 700/3512

ReliefF Iteration: 800/3512

ReliefF Iteration: 900/3512

ReliefF Iteration: 1000/3512

ReliefF Iteration: 1100/3512

ReliefF Iteration: 1200/3512

ReliefF Iteration: 1300/3512

ReliefF Iteration: 1400/3512

ReliefF Iteration: 1500/3512

ReliefF Iteration: 1600/3512

ReliefF Iteration: 1700/3512

ReliefF Iteration: 1800/3512

ReliefF Iteration: 1900/3512

ReliefF Iteration: 2000/3512

ReliefF Iteration: 2100/3512

ReliefF Iteration: 2200/3512

ReliefF Iteration: 2300/3512

ReliefF Iteration: 2400/3512

ReliefF Iteration: 2500/3512

ReliefF Iteration: 2600/3512

ReliefF Iteration: 2700/3512

ReliefF Iteration: 2800/3512

ReliefF Iteration: 2900/3512

ReliefF Iteration: 3000/3512

ReliefF Iteration: 3100/3512

ReliefF Iteration: 3200/3512

ReliefF Iteration: 3300/3512

ReliefF Iteration: 3400/3512

ReliefF Iteration: 3500/3512

```
In [49]: print("\n--- Top 10 Features ---")
         print(df_relieff.head(10))
```

--- Top 10 Features ---

	Feature	ReliefF_Score
27	ay_energy	0.273503
29	az_median	0.273266
32	az_rms	0.272075
28	az_mean	0.269567
41	az_energy	0.263936
98	my_mean	0.240208
99	my_median	0.239863
18	ay_rms	0.239217
125	mz_energy	0.213680
14	ay_mean	0.192950

4.6 - Compare the top 10 results for Fisher Score and Relief

4.6.1 - Instant of my choice

```
In [50]: top_10_names = df_relieff['Feature'].values[:10]

all_feature_names_list = list(feature_names)

top_10_indices = [all_feature_names_list.index(name) for name in top_10_names]

X_selected_features = X_zscore[:, top_10_indices]

instant_index = 0
original_instant = X_zscore[instant_index]
selected_instant = X_selected_features[instant_index]

print(f"--- How to Obtain the Selected Features ---")
print(f"New selected data shape: {X_selected_features.shape}")

print(f"\n--- Example for Instant (row {instant_index}) ---")

print("\nOriginal Scaled Instant (first 5 of 139 features):")
print(original_instant[:5])

print(f"\n--- After Feature Selection (The 10 Selected Features) ---")
print("These are the 10 feature values for this instant:")
print(selected_instant)

print("\nNames and values of the selected features:")

for name, value in zip(top_10_names, selected_instant):
    print(f"{name:<25} value = {value:.4f}")
```

--- How to Obtain the Selected Features ---

New selected data shape: (3512, 10)

--- Example for Instant (row 0) ---

Original Scaled Instant (first 5 of 139 features):

```
[ 0.67100438  0.66569702 -0.53777858 -0.42832668 -0.75486569]
```

--- After Feature Selection (The 10 Selected Features) ---

These are the 10 feature values for this instant:

```
[ 0.69677846 -0.58139158 -0.59164572 -0.55467462 -0.63866801  0.55016809  
 0.53690796  0.68104958  1.00131456  0.69768753]
```

Names and values of the selected features:

ay_energy	value = 0.6968
az_median	value = -0.5814
az_rms	value = -0.5916
az_mean	value = -0.5547
az_energy	value = -0.6387
my_mean	value = 0.5502
my_median	value = 0.5369
ay_rms	value = 0.6810
mz_energy	value = 1.0013
ay_mean	value = 0.6977

- The Fisher Score, a univariate method, prioritizes features that offer high individual class separation (large ratio of between-class to within-class variance), thus highly ranking ay_energy (1st) and the my_ features (2nd, 3rd, 4th).
- In contrast, the ReliefF Score, a multivariate technique, assesses feature relevance based on distinguishing nearest neighbors, making it sensitive to feature interactions and local feature relevance. ReliefF also ranked ay_energy first, but significantly favored az_median (2nd) and az_rms (3rd) over the my_ features, which it placed lower.
- The consensus on ay_energy confirms its robust importance, while the discrepancy highlights that the my_ features are strong discriminators in isolation (Fisher), and the az_ features are more crucial for local decision boundaries and cooperative classification (ReliefF).

5 - Data Augmentation

5.1 - Analyze the balance of the dataset

- The dataset is imbalanced

```
In [51]: X_zscore_participants = np.column_stack((participants,X_zscore))
```

```
In [52]: classes, counts = np.unique(y, return_counts=True)
for label, count in zip(classes, counts):
    print(f"Label {label}: {count} samples")
```

```
Label 1.0: 1144 samples
Label 2.0: 656 samples
Label 3.0: 638 samples
Label 4.0: 1074 samples
```

- the dataset is not balanced, so the model won't perform as well if they were balanced. This happens because the model will learn to generalize better for the majority class instead of learning to generalize to all classes.

5.2 - SMOTE Implementation

- each minority class (walk,sit,sit and walk) will have the same number of samples of the majority class (stand), so every class will have 1172 samples.

- **Formula: $x = x_i + \text{rand}(0-1)(x^i - x_i)$**

```
In [53]: def smote(X,y,target_class,n_new,k=5,random_state=None):
    if random_state:
        np.random.seed(random_state)

    X_min = X[y == target_class]    # to map the samples that are on label=target_class
    nn = NearestNeighbors(n_neighbors=k)
    nn.fit(X_min)

    X_synthetic = []
    for _ in range(n_new):
        idx = np.random.randint(0,len(X_min))    # choose a random sample
        x_i = X_min[idx]    # x_i is the value of the random selected sample or
        neighbors = nn.kneighbors([x_i],return_distance=False)[0]
        neighbor_idx = np.random.choice(neighbors[1:])    # choose randomly a xi
```

```

        x_neighbor = X_min[neighbor_idx]    # assign the value to the xi neighbor

        diff = x_neighbor-x_i
        gap = np.random.rand() # random number between 0-1
        synthetic_sample = x_i+gap*diff # formula
        X_synthetic.append(synthetic_sample)

X_synthetic = np.array(X_synthetic)
y_synthetic = np.full(len(X_synthetic), target_class)

return X_synthetic,y_synthetic

```

```

In [54]: max_count = counts.max()

X_synthetic_all = []
y_synthetic_all = []

unique_original_ids = np.unique(X_zscore_participants[:, 0]).astype(int)
num_original_ids = unique_original_ids.size

for cls, count in zip(classes, counts):
    if count < max_count:
        n_new = max_count - count
        print(f"Class {cls}: Generating {n_new} new samples.")

        X_synthetic_features, y_synthetic_cls = smote(
            X_zscore_participants[:, 1:],
            y,
            target_class=cls,
            n_new=n_new,
            k=5,
            random_state=42
        )

        if num_original_ids > 0 and n_new > 0:
            repeating_ids = np.tile(unique_original_ids, int(np.ceil(n_new / num_original_ids)))
            synthetic_participant_ids = repeating_ids[:n_new].reshape(-1, 1)
        else:
            synthetic_participant_ids = np.zeros((n_new, 1), dtype=int)

        X_synthetic_cls_with_id = np.hstack((synthetic_participant_ids, X_synthetic_features))

        X_synthetic_all.append(X_synthetic_cls_with_id)
        y_synthetic_all.append(y_synthetic_cls)

if X_synthetic_all:
    X_synthetic_all = np.vstack(X_synthetic_all)
    y_synthetic_all = np.hstack(y_synthetic_all)
else:
    num_cols_for_synthetic = X_zscore_participants.shape[1] if X_zscore_participants.shape[1] > 0 else 0
    X_synthetic_all = np.array([]).reshape(0, num_cols_for_synthetic)
    y_synthetic_all = np.array([])

```

```
y_col = y.reshape(-1, 1)
combines_data_features = np.hstack((X_zscore_participants, y_col))

print("Shape final (original data):", combines_data_features.shape)
print(combines_data_features[:5])

y_syn = y_synthetic_all.reshape(-1, 1)
combines_data_features_synthetic = np.hstack((X_synthetic_all, y_syn))

print("Shape final (synthetic data):", combines_data_features_synthetic.shape)
print(combines_data_features_synthetic[:5])

merged_full_data = np.vstack((combines_data_features, combines_data_features_s

print("\nShape of merged_full_data (original + synthetic with cycling IDs):",
print("First 5 rows of merged_full_data (original):\n", merged_full_data[:5])
print("Last 5 rows of merged_full_data (synthetic with cycling IDs):\n", merge
```

Class 2.0: Generating 488 new samples.

Class 3.0: Generating 506 new samples.

Class 4.0: Generating 70 new samples.

Shape final (original data): (3512, 141)

```
[[ 0.00000000e+00  6.71004380e-01  6.65697019e-01 -5.37778581e-01
-4.28326684e-01 -7.54865691e-01  2.02034806e-01 -3.19978904e-01
-1.84473052e-01 -5.30547118e-01 -3.43669644e-01 -7.26423695e-01
-4.30124356e-01 -5.30567384e-01 -7.07269212e-01  6.97687529e-01
 7.28512905e-01 -5.60651075e-01 -2.75516290e-01  6.81049583e-01
 2.36962501e-02 -1.43306864e+00 -2.65941026e-01 -5.08910035e-01
-6.40926623e-02 -6.33530677e-01 -4.14973460e-01 -7.89238006e-01
 6.96778460e-01 -5.54674622e-01 -5.81391582e-01 -3.98382647e-01
-4.67781210e-01 -5.91645716e-01 -2.14002211e-01  1.15801566e+00
-5.09796537e-01 -2.95016077e-01 -3.39982130e-01 -1.03363335e+00
-4.96115865e-01 -5.76301591e-01 -6.38668014e-01  1.08054005e-01
 3.37008405e-02 -4.91666782e-01 -3.83524977e-01 -5.23168179e-01
 1.09508004e-02 -2.12706536e+00  1.08563617e+00 -5.15041597e-01
 1.23327143e+00 -4.22105694e-01  9.59174101e-01 -5.82691223e-01
-4.00536500e-01 -1.15881283e-01 -9.24868404e-02 -5.99145233e-01
-5.05502825e-01 -6.06498195e-01  2.29918733e-02 -3.53089523e-01
-4.05421166e-01 -5.75652293e-01 -3.32502929e-01 -3.88308398e-01
-2.13796226e-02 -4.20337509e-01 -5.36293196e-01 -1.45418003e-02
-1.20981740e-02 -6.06283066e-01 -5.16823300e-01 -6.11668079e-01
-1.45341244e-02 -4.35634922e+00  2.68638404e+00 -5.90690330e-01
 2.13831831e-01  2.83974419e-02  1.42889938e+00 -1.29451400e-01
-5.21222803e-01  7.38649633e-02  6.64937376e-02 -5.47237125e-01
-4.84555945e-01 -8.03606477e-01  1.02832281e-01  6.33122798e-01
-1.28936486e-01 -5.03297090e-01 -1.96125955e-01 -1.62010708e-01
-5.46800056e-01 -8.98546778e-01 -7.73821305e-01  5.50168093e-01
 5.36907964e-01 -4.90924565e-01 -3.86232730e-01  3.71640443e-02
-3.11321153e-02 -2.42507429e-02 -1.08878635e-01 -5.01531419e-01
-1.80193824e-01 -1.82361109e-01 -3.20935806e-01 -1.40666137e+00
-1.63032554e-01  9.62840308e-01  9.44474798e-01 -5.99400929e-01
-4.80865547e-01  1.01261722e+00  2.36540824e-02  3.88840158e-01
-8.89363197e-02 -5.45640451e-01 -2.17190372e-01  2.79950670e-01
-5.62984872e-01 -8.28709321e-01  1.00131456e+00 -6.82610470e-01
-5.52740554e-01 -4.66221476e-01 -2.83615065e-01 -5.14352627e-01
-6.30200258e-01  5.25095728e-01  6.68842819e-01  5.26647501e-02
-7.09707646e-01 -6.74897088e-01 -6.33991637e-01 -5.51311067e-01
 1.00000000e+00]
[ 0.00000000e+00  6.88744125e-01  6.67421810e-01 -3.46532924e-01
-3.95214358e-01 -7.69852484e-01  5.05211027e-01  1.10636703e+00
 7.51589840e-01 -5.07930421e-01 -3.43669644e-01 -1.41512461e-01
-3.63173273e-01  2.20693614e-01 -7.14899537e-01  6.98718040e-01
 7.28067775e-01 -5.65322841e-01 -2.75679542e-01  6.82071586e-01
 1.04732342e-01 -9.23054761e-01 -3.02119877e-01 -5.08590165e-01
-6.40926623e-02  5.12086474e-02 -4.15163164e-01 -7.89238006e-01
 6.98409805e-01 -5.47087958e-01 -5.54201683e-01 -4.21129951e-01
-4.72476537e-01 -5.84145235e-01 -3.63473734e-01  3.90791272e-01
-5.15507979e-01 -3.38061501e-01 -3.39982130e-01 -1.10077139e+00
-4.99387052e-01 -5.76301591e-01 -6.34994020e-01  1.54021818e-01
 3.65784020e-02 -5.12666030e-01 -3.84907119e-01 -5.47073682e-01
-1.27789584e-02 -4.11231320e-01 -3.26643036e-01 -5.00667916e-01
 2.48196486e+00 -6.22186026e-01  1.02214351e+00 -7.35340154e-01
```

-4.02534529e-01 -1.29438853e-01 -1.07708934e-01 -4.79345795e-01
-4.94740853e-01 -4.98651315e-01 -3.46813357e-02 -2.68442279e+00
2.32156080e+00 -5.33162489e-01 -5.53266367e-01 -6.52223846e-01
9.65274169e-01 -3.12599563e-01 -5.28193918e-01 4.28026467e-02
7.26135894e-03 -6.08233181e-01 -5.16877819e-01 -6.08842940e-01
-1.61638763e-02 1.74143106e-01 6.63294997e-01 -5.85171987e-01
9.83050697e-01 3.59854732e-01 9.45597254e-01 -2.74200291e-03
-5.20988300e-01 1.15113083e-01 1.14271719e-01 -5.37130433e-01
-4.83706600e-01 -8.73061906e-01 1.44149004e-01 1.27094020e-01
-1.17590611e-01 -4.88574732e-01 -1.96125955e-01 -8.14325533e-01
-5.38457603e-01 -7.98066097e-01 -8.17157980e-01 5.49559198e-01
5.36907964e-01 -5.09024786e-01 -3.89297620e-01 3.54707087e-02
-2.20187203e-01 4.34791326e-02 -9.82286317e-02 -4.97045172e-01
-1.80193824e-01 2.98098167e-01 -3.22157775e-01 -1.40666137e+00
-1.64816290e-01 9.71897546e-01 9.54566708e-01 -6.02501167e-01
-4.81020315e-01 1.02427796e+00 -4.17737580e-02 4.24654481e-01
-8.50236152e-02 -5.46115728e-01 -2.17190372e-01 5.68171204e-01
-5.63073307e-01 -8.28709321e-01 1.01818177e+00 -6.59482151e-01
-5.51765959e-01 -4.71463253e-01 -2.83344483e-01 -4.90977865e-01
-1.03737768e-01 5.31455784e-01 6.73381451e-01 2.20291786e-01
-4.19444079e-01 -6.55075241e-01 -6.14876899e-01 -5.47145277e-01
1.00000000e+00]
[0.00000000e+00 7.39554373e-01 7.38650264e-01 -3.37032412e-01
-3.93030544e-01 -8.22469621e-01 8.48862132e-02 -3.24128384e-01
5.53082923e-01 -3.72786391e-01 -3.43669644e-01 -8.36094551e-01
-3.50468610e-01 -5.30567384e-01 -7.38092345e-01 7.00618745e-01
7.29307780e-01 -5.78494648e-01 -2.76094148e-01 6.83951459e-01
3.05002238e-02 -1.29330144e+00 -1.29722314e-02 -5.38092058e-01
-6.40926623e-02 4.13717702e-01 -4.16140157e-01 1.40985835e+00
7.01416177e-01 -5.31531206e-01 -5.31787445e-01 -5.22229529e-01
-4.89210874e-01 -5.69080990e-01 2.98106286e-01 -8.92700112e-01
1.99468752e-01 -5.85890619e-01 -3.39982130e-01 5.10541599e-01
-5.18547382e-01 -5.12403527e-01 -6.27441981e-01 2.28039760e-02
-5.20708918e-02 -5.43255549e-01 -3.86477878e-01 -5.52001929e-01
-1.09611026e-02 -9.86577027e-01 4.04745302e-01 -5.56712277e-01
6.08924712e-01 4.58247768e-01 3.48971187e-01 -5.82691223e-01
-4.01653469e-01 -1.03357555e-01 -7.05500422e-02 -4.83081291e-01
-4.95228561e-01 -5.03443407e-01 2.53611365e-02 -3.02847720e+00
2.73307487e+00 -5.65368107e-01 -3.32502929e-01 -4.32294306e-01
1.09594933e+00 -3.12599563e-01 -5.28856683e-01 1.44808029e-02
-9.27057875e-03 -6.08191174e-01 -5.16876674e-01 -6.11227544e-01
3.00033500e-02 4.55769124e-01 6.96377065e-01 -5.87979455e-01
2.74126525e+00 1.75711793e-01 1.32458439e+00 1.23967395e-01
-5.21142582e-01 1.76377562e-01 1.85965029e-01 -5.24916248e-01
-4.82587739e-01 -9.76191965e-01 4.77660808e-02 -1.75220043e-01
-1.24953821e-01 -4.59202901e-01 -1.96125955e-01 -6.27949869e-01
-5.29095432e-01 -8.98546778e-01 -8.77735280e-01 5.52254731e-01
5.40503243e-01 -5.55140099e-01 -3.96388140e-01 4.12408693e-02
-1.30738715e-03 -1.02846576e-01 -9.10409770e-02 -5.14726263e-01
-1.80193824e-01 8.03844774e-01 -3.26565239e-01 7.12479740e-01
-1.57886161e-01 9.87467988e-01 9.69368177e-01 -6.03450998e-01
-4.81066457e-01 1.04433950e+00 9.61552029e-02 -3.57159043e-02
-1.33899589e-01 -5.46115728e-01 -2.17190372e-01 5.68171204e-01
-5.63072178e-01 -8.28709321e-01 1.04734274e+00 -6.51906601e-01

-5.52097168e-01	-5.00705900e-01	-2.83041635e-01	-5.08138160e-01
8.24612870e-02	5.50149291e-01	7.21030746e-01	-1.77674318e-02
1.19616832e-01	-6.48678938e-01	-6.06514683e-01	-5.47420741e-01
1.00000000e+00]			
[0.00000000e+00	7.60981432e-01	7.51208894e-01	-5.96856036e-01
-4.34376768e-01	-8.49552393e-01	-8.21590030e-02	2.71124834e-01
-2.45963606e-01	-5.53302851e-01	-3.43669644e-01	1.14386203e-01
-4.38441466e-01	-5.30567384e-01	-7.48493847e-01	7.05503560e-01
7.29530345e-01	-5.81967506e-01	-2.76192230e-01	6.88826685e-01
1.37911269e-01	1.47307334e+00	2.30914418e-01	-5.49115284e-01
-6.40926623e-02	5.34554053e-01	-4.16459451e-01	-7.89238006e-01
7.09186297e-01	-5.42672189e-01	-5.31194845e-01	-4.05222382e-01
-4.69228939e-01	-5.79498694e-01	-6.37892291e-01	-3.45788869e+00
6.82861355e-01	-5.75601337e-01	-3.39982130e-01	-8.99357265e-01
-4.94400596e-01	-5.76301591e-01	-6.32773242e-01	2.39490393e-01
-7.92297893e-03	-5.22642017e-01	-3.85477057e-01	-5.63947729e-01
-1.79269616e-02	1.92051455e+00	2.42469963e-02	-5.54157201e-01
2.96751354e-01	-9.02298491e-01	7.48387974e-01	-7.35340154e-01
-4.03958828e-01	-5.46978316e-02	-5.37188342e-02	-5.98502386e-01
-5.05471955e-01	-6.07063783e-01	1.16572501e-01	1.05149012e+00
1.90328804e-02	-5.94281312e-01	-1.11739490e-01	-2.12364766e-01
1.90439233e-01	-9.71236719e-02	-5.36361489e-01	-1.54941672e-02
-2.18652886e-02	-6.29050176e-01	-5.17286714e-01	-6.30328541e-01
1.28693407e-02	-4.25700768e-01	-1.28153278e-01	-5.94383863e-01
2.52148843e+00	9.49112138e-01	-2.74993525e-02	-7.62998388e-01
-5.21678603e-01	2.13540241e-01	2.09854020e-01	-5.97895789e-01
-4.87769838e-01	-1.04245383e+00	-4.40412710e-02	-1.91215548e-01
-7.60153069e-02	-5.41870395e-01	-1.96125955e-01	8.16461530e-01
-5.59448616e-01	1.11106685e+00	-9.13274788e-01	5.66847558e-01
5.51271194e-01	-5.38336690e-01	-3.93923953e-01	7.58937730e-02
1.57873429e-01	4.54434126e-01	-7.51488800e-02	-5.32407354e-01
-1.80193824e-01	6.52120792e-01	-3.25481259e-01	-1.40666137e+00
-1.18175601e-01	9.95535677e-01	9.84393910e-01	-5.72399652e-01
-4.79248337e-01	1.05486086e+00	-1.73042047e-01	-1.21230517e+00
1.72069099e-01	-5.45640451e-01	-2.17190372e-01	8.04133775e-02
-5.62221665e-01	-8.28709321e-01	1.06261885e+00	-6.55060993e-01
-5.53548196e-01	-5.38599704e-01	-2.85498505e-01	-5.14995190e-01
-9.13339690e-01	5.69902910e-01	7.68723338e-01	9.14637554e-02
1.19616832e-01	-6.51583125e-01	-6.08068231e-01	-5.52072090e-01
1.00000000e+00]			
[0.00000000e+00	7.26448039e-01	7.22318654e-01	-4.41795080e-01
-4.14291675e-01	-8.11211726e-01	-1.68393384e-01	6.69170011e-01
8.85444895e-01	-5.61691421e-01	-3.43669644e-01	5.16512677e-01
-3.95683250e-01	-5.30567384e-01	-7.32737464e-01	7.12303151e-01
7.35539599e-01	-5.15399898e-01	-2.73496008e-01	6.95741012e-01
-1.26170927e-02	1.07838788e+00	1.41198313e+00	-4.99289318e-01
-6.40926623e-02	-7.14088245e-01	-4.10206510e-01	-7.89238006e-01
7.20126038e-01	-5.91121552e-01	-5.80223811e-01	-2.41147080e-01
-4.25982423e-01	-6.26673871e-01	-1.78203433e-01	-4.73230705e-01
-5.44403930e-01	-4.78806178e-02	-3.39982130e-01	-1.20147845e+00
-4.39797912e-01	-5.76301591e-01	-6.55654210e-01	1.13571272e-01
-3.88929415e-02	-4.68760544e-01	-3.81735249e-01	-5.02829900e-01
5.29150084e-02	4.94762465e-02	-3.05963293e-01	-4.47562171e-01
1.70153147e+00	-8.22266358e-01	1.41579823e+00	-5.82691223e-01

```
-3.98933797e-01 -4.94084208e-02 -4.21923287e-02 -5.47979479e-01
-5.02138698e-01 -5.61492155e-01 -6.16808211e-02 7.76780536e-02
-6.72372412e-02 -5.62226908e-01 -3.76655616e-01 -5.64252030e-01
5.71598367e-01 -4.20337509e-01 -5.33860882e-01 -8.74818744e-02
-2.96216689e-02 -5.79632419e-01 -5.15799862e-01 -5.88838212e-01
-5.96636935e-02 -3.09403993e+00 2.46140688e+00 -5.88109417e-01
7.63273878e-01 1.02054618e-01 2.03499885e+00 -7.62998388e-01
-5.20429612e-01 1.54317759e-01 1.56755668e-01 -5.26589143e-01
-4.82746959e-01 -9.38928179e-01 -1.63370341e-01 4.35703444e-02
-1.47789116e-01 -4.59202901e-01 -1.96125955e-01 -9.54107282e-01
-5.31704697e-01 -8.98546778e-01 -8.56405714e-01 5.87875022e-01
5.69229704e-01 -5.11827942e-01 -3.89758065e-01 1.25868131e-01
2.17572428e-01 5.89980252e-01 -1.60977809e-01 -5.14726263e-01
-1.80193824e-01 -4.35234413e-01 -3.25380951e-01 -1.40666137e+00
-5.99829341e-02 9.66304144e-01 9.46941709e-01 -5.24617423e-01
-4.75202642e-01 1.01747496e+00 -6.15199981e-02 3.81445712e-01
-2.00246509e-01 -4.40128962e-01 -2.17190372e-01 -6.29052552e-01
-5.60681196e-01 -8.28709321e-01 1.00804525e+00 -6.68634292e-01
-5.47983311e-01 -5.74807139e-01 -2.75240323e-01 -4.96019756e-01
7.88391197e-02 5.56086914e-01 7.39856682e-01 1.64479603e-01
-4.60910303e-01 -6.62289510e-01 -6.25503410e-01 -5.49433323e-01
1.00000000e+00]]
```

Shape final (synthetic data): (1064, 141)

```
[[ 0.00000000e+00 -2.16711439e+00 -2.17225929e+00 -6.58391308e-01
-4.38581342e-01 2.21277977e+00 -7.11185205e-03 3.55296176e-01
-1.22511165e-01 -6.39367557e-01 -3.43669644e-01 9.36656497e-01
-4.56044670e-01 -4.59143926e-01 2.73686236e+00 -7.89331850e-01
-7.80266652e-01 -6.16188660e-01 -2.76907292e-01 -8.04479275e-01
1.68430219e-02 -7.72677741e-02 -2.49535526e-01 -5.61495393e-01
-6.40926623e-02 -1.92449223e-01 -4.16339864e-01 -7.89238006e-01
-1.06818842e+00 3.74460563e-01 3.70787787e-01 -6.63600829e-01
-5.01293714e-01 3.53997219e-01 6.03980803e-02 -5.94086733e-01
6.33897610e-01 -6.62207972e-01 -3.39982130e-01 1.21762508e+00
-5.39372203e-01 3.34930468e-01 5.44705627e-02 7.05574184e-02
-3.67316775e-02 -5.82781782e-01 -3.87730198e-01 -5.84917931e-01
-3.21678065e-02 -3.95351782e-01 -8.27724288e-02 -5.66486226e-01
-3.42981042e-01 -2.65985867e-01 -6.47046102e-01 -4.45089108e-01
-4.03839331e-01 -7.72808157e-02 -6.85261628e-02 -6.40864961e-01
-5.06882970e-01 -6.46801595e-01 1.08196624e-02 2.71940759e-01
8.08045810e-02 -6.19964236e-01 7.16281098e-01 6.12528107e-01
8.13468457e-01 -2.75429988e-01 -5.37508189e-01 5.82912449e-03
-1.20724770e-02 -6.35511876e-01 -5.17349085e-01 -6.31148350e-01
-3.32235350e-03 1.42653795e+00 4.41595587e-01 -6.01363709e-01
-1.50105531e+00 1.35656786e+00 -1.19034316e+00 8.59243938e-01
-5.21646386e-01 -1.17517304e+00 -1.10549868e+00 -5.99299495e-01
-4.87833597e-01 1.31580663e+00 -2.75539323e-02 -1.18010498e-01
-8.95285593e-02 -5.33031722e-01 -1.96125955e-01 9.01611915e-01
-5.73844048e-01 1.21154753e+00 1.52053289e+00 -8.03733287e-01
-8.03869883e-01 -5.74621197e-01 -3.99073559e-01 -1.85674335e+00
1.65870796e-01 6.47240131e-02 -1.75284164e-01 -5.31753786e-01
-1.80193824e-01 6.39296498e-01 -1.49337727e-01 7.65458267e-01
-1.51981785e+00 1.08980764e+00 1.07013078e+00 -6.09912451e-01
-4.81364342e-01 1.17619933e+00 3.69164825e-02 -4.84177812e-01
1.09955176e-01 -5.56119969e-01 -2.17190372e-01 8.44055262e-01
```

-5.63303659e-01	1.21923398e+00	1.24379453e+00	-6.33662553e-01
-5.54539325e-01	1.68904372e+00	-2.92981185e-01	-5.20193581e-01
1.20917850e+00	-2.78392073e+00	-2.16651596e+00	6.79481518e-02
1.61083056e-01	-6.33521362e-01	-5.85133122e-01	-5.52663805e-01
2.00000000e+00]			
[1.00000000e+00	-2.22505849e+00	-2.22810741e+00	-6.67511540e-01
-4.39021619e-01	2.27338196e+00	2.02149012e-02	1.40127500e-01
-1.09532756e-01	-6.38672855e-01	-3.43669644e-01	1.08724250e+00
-4.56133870e-01	-5.00416161e-01	2.85110313e+00	-7.90472899e-01
-7.83648918e-01	-6.12027700e-01	-2.76844014e-01	-8.05613313e-01
-3.39447365e-02	2.09566418e-02	-1.66704532e-01	-5.65501294e-01
-6.40926623e-02	1.88775511e-02	-4.15680140e-01	-7.57716982e-01
-1.06908841e+00	3.21213953e-01	3.18103373e-01	-6.66051596e-01
-5.01374291e-01	2.99710845e-01	2.36962351e-02	-5.32862195e-01
2.74930219e-01	-6.48810760e-01	-3.39982130e-01	9.27067655e-01
-5.39394168e-01	-1.21127791e-02	2.33374388e-03	4.63667395e-02
-5.10661121e-02	-5.88056997e-01	-3.87831021e-01	-5.81490410e-01
3.46483032e-02	-3.44117327e-01	-2.00499771e-01	-5.72332970e-01
-1.07723021e+00	1.78135303e-01	-9.02477232e-01	-7.35340154e-01
-4.03446165e-01	-7.34173388e-02	-6.19518757e-02	-6.46304403e-01
-5.06976095e-01	-6.51423765e-01	-2.60228560e-02	3.62967461e-01
-1.24515081e-01	-6.23770060e-01	5.77575522e-01	8.96257140e-01
1.00711977e+00	-4.41596034e-01	-5.37569419e-01	-5.99650860e-03
-1.57806778e-02	-6.35172432e-01	-5.17346418e-01	-6.32622217e-01
-1.09348518e-02	7.85354109e-01	2.70529193e-01	-6.00080553e-01
-1.17061471e+00	8.67693996e-01	-1.04360704e+00	-7.12144647e-01
-5.21697352e-01	-1.24236014e+00	-1.16745986e+00	-6.00866363e-01
-4.87904116e-01	1.42990809e+00	-2.29531982e-02	-2.06313212e-01
-7.84499867e-02	-5.40008284e-01	-1.96125955e-01	9.51958926e-01
-5.74051249e-01	1.18147073e+00	1.69938951e+00	-8.05138447e-01
-8.06402869e-01	-5.67428652e-01	-3.98062167e-01	-1.85312888e+00
-9.89676695e-02	-1.76298318e-01	-7.96375360e-02	-5.24468653e-01
-1.80193824e-01	6.01037281e-01	-1.46208761e-01	7.86720750e-01
-1.51882435e+00	1.05200431e+00	1.03354335e+00	-6.12438631e-01
-4.81472863e-01	1.12747666e+00	-1.16474096e-02	-1.00965168e-01
-6.10650505e-02	-5.56381073e-01	-2.17190372e-01	7.41163010e-01
-5.63344092e-01	1.23928093e+00	1.17061624e+00	-6.55845294e-01
-5.54607027e-01	1.65573490e+00	-2.93062848e-01	-5.20082536e-01
1.20487562e+00	-2.91913301e+00	-2.22301332e+00	3.70176547e-02
5.17636954e-01	-6.52407845e-01	-6.05484144e-01	-5.52652598e-01
2.00000000e+00]			
[2.00000000e+00	-1.49849658e+00	-1.49929128e+00	-6.45535269e-01
-4.37879795e-01	1.51338524e+00	-8.96282066e-02	-2.77758015e-01
-2.12085045e-01	-6.18600833e-01	-3.43669644e-01	-3.30298772e-01
-4.55723048e-01	-5.30567384e-01	1.55136461e+00	-1.45425599e+00
-1.45398205e+00	-6.15388153e-01	-2.76892485e-01	-1.46869092e+00
-3.22332793e-02	-5.35446388e-01	-2.19280757e-01	-5.55536174e-01
-6.40926623e-02	-1.75647643e-01	-4.09436546e-01	-7.89238006e-01
-1.47082195e+00	1.22997909e+00	1.22817277e+00	-6.58584682e-01
-5.01086671e-01	1.22619659e+00	-6.87460007e-02	-9.24412189e-02
-1.83064847e-01	-6.34491095e-01	-3.39982130e-01	-8.97789780e-03
-5.39677749e-01	-5.76301591e-01	1.11585416e+00	5.92261367e-03
-8.27951041e-02	-5.91706371e-01	-3.87888623e-01	-5.72872131e-01
-8.07234796e-04	-1.61585669e+00	7.60130181e-01	-5.78471574e-01

-1.21541810e+00	4.25918733e-01	-1.08061509e+00	-6.06503620e-01
-4.02591003e-01	-7.15714336e-02	-6.25020815e-02	-6.48022915e-01
-5.07000311e-01	-6.52607812e-01	1.99617402e-03	1.57401918e+00
5.28564771e-01	-6.31830093e-01	7.60366334e-01	7.07294287e-01
3.55528981e-01	-4.37144038e-01	-5.37579920e-01	-1.38803171e-02
-2.33160692e-02	-6.37712263e-01	-5.17363595e-01	-6.34806331e-01
-1.55586215e-03	1.38324462e+00	2.96614026e-01	-6.02492339e-01
-1.47581476e+00	1.20927049e+00	-1.34933162e+00	-7.62998388e-01
-5.21747254e-01	-5.63886058e-01	-5.26881858e-01	-6.02567200e-01
-4.87977719e-01	2.77536995e-01	-2.46710462e-02	-1.56034158e-01
-8.49369990e-02	-5.46212218e-01	-1.96125955e-01	8.43392686e-01
-5.72309272e-01	1.06082651e+00	1.62100796e-01	-1.43231894e+00
-1.43090730e+00	-5.82125396e-01	-4.00050953e-01	-3.76042667e-01
-8.90227749e-02	-4.02402137e-01	1.10208774e-02	-5.35151785e-01
-1.80193824e-01	8.33989766e-01	-3.23879243e-01	6.59501212e-01
-5.94943716e-01	7.71926675e-01	7.60161136e-01	-6.06643717e-01
-4.81214431e-01	7.66573342e-01	-1.42897353e-02	-2.17628432e-02
-9.53602190e-02	-5.49129819e-01	-2.17190372e-01	5.77127118e-01
-5.62995413e-01	-8.28709321e-01	6.61571740e-01	-3.80921055e-01
-5.54738273e-01	1.08476309e+00	-2.92668494e-01	-5.20350968e-01
1.94563702e+00	-1.43582809e+00	-1.49062079e+00	1.10988551e-02
3.36071657e-01	-4.17947728e-01	-3.52832558e-01	-5.52538740e-01
2.00000000e+00]			
[3.00000000e+00	-7.60318933e-01	-7.67428315e-01	-6.90272206e-01
-4.39918306e-01	7.41135407e-01	-4.91485451e-02	2.24445975e-01
-1.94186538e-01	-6.81741751e-01	-3.43669644e-01	1.65208980e+00
-4.56014122e-01	-4.55441284e-01	5.08906900e-01	-7.28673939e-01
-7.21032235e-01	-6.26037276e-01	-2.77029321e-01	-7.43895985e-01
-7.95975988e-03	2.24941371e-01	-1.99461223e-01	-5.66134893e-01
-6.40926623e-02	6.21094610e-01	-4.17613201e-01	-7.10698850e-01
-1.01957238e+00	1.16940003e+00	1.16902955e+00	-6.89765460e-01
-5.02082465e-01	1.16442048e+00	-6.56499887e-03	1.10269091e-01
-9.51222398e-02	-6.80374516e-01	-3.39982130e-01	2.07480856e+00
-5.39911649e-01	4.46537589e-01	1.02721532e+00	5.85337742e-02
-4.66240898e-02	-6.01675944e-01	-3.88019237e-01	-5.88661837e-01
3.17924684e-03	4.29589254e-01	-3.52593046e-01	-5.82143922e-01
-1.26411544e+00	1.30364584e+00	-1.24363716e+00	-5.82691223e-01
-4.03876483e-01	-7.36769295e-02	-6.02004583e-02	-6.55026512e-01
-5.07077819e-01	-6.59125657e-01	-1.16144483e-03	-1.35409169e-01
-2.86435959e-01	-6.34252771e-01	4.59128675e-01	9.48604295e-01
2.55064454e-01	-4.20337509e-01	-5.37645161e-01	-1.12882738e-02
-1.91478448e-02	-6.40544214e-01	-5.17376854e-01	-6.35342749e-01
1.61647115e-03	-6.67880258e-02	-2.41539496e-01	-6.03949585e-01
-1.51497134e+00	1.95826964e+00	-1.70669316e+00	-5.26536333e-01
-5.21749589e-01	1.26015080e-01	1.24912460e-01	-6.04914762e-01
-4.88077499e-01	-8.94063719e-01	4.89554487e-02	-3.81911866e-01
-2.84024969e-02	-5.46374613e-01	-1.96125955e-01	7.10974358e-01
-5.65115869e-01	1.21827089e+00	-8.29061172e-01	-1.00582654e+00
-1.00754175e+00	-5.93495084e-01	-4.01483679e-01	-1.38229760e+00
6.62778396e-02	-6.10584727e-02	-9.60136393e-02	-5.33869937e-01
-1.80193824e-01	6.94328363e-01	-2.82644215e-01	8.25526586e-01
-1.33018908e+00	1.45237361e+00	1.42822623e+00	-6.21571259e-01
-4.81831021e-01	1.64339685e+00	-7.89742532e-03	-3.67845096e-01
6.61460525e-02	-5.65628640e-01	-2.17190372e-01	8.62325705e-01

```

-5.63677042e-01  1.27586830e+00  2.00670631e+00 -2.47277547e-01
-5.54842776e-01  1.60302738e+00 -2.93361618e-01 -5.20749252e-01
6.63303757e-01 -4.24198508e-01 -7.57813700e-01  4.32084448e-02
1.61083056e-01 -3.03579561e-01 -2.29220938e-01 -5.52782878e-01
2.00000000e+00]
[ 4.00000000e+00 -2.17158739e+00 -2.18067434e+00 -6.33499161e-01
-4.37137793e-01  2.21749624e+00 -1.43450585e-01  7.10890110e-01
-1.94498164e-01 -6.09556976e-01 -3.43669644e-01 -5.95662961e-03
-4.55831693e-01 -4.24177921e-01  2.74597003e+00 -7.79321558e-01
-7.73331672e-01 -5.99011670e-01 -2.76605203e-01 -7.94451978e-01
-1.57105072e-02  4.21425068e-01 -3.71443748e-01 -5.34207959e-01
-6.40926623e-02 -9.13730361e-01 -4.13694773e-01 -7.89238006e-01
-1.06027230e+00  3.59062673e-01  3.54790964e-01 -6.44558956e-01
-5.00425555e-01  3.38319628e-01 -1.53701892e-01  1.25101927e+00
4.80442173e-01 -6.45827294e-01 -3.39982130e-01  2.69928605e-01
-5.38948622e-01 -4.85812657e-01  3.95929673e-02  5.52159128e-02
-5.09910719e-02 -5.79844003e-01 -3.87666944e-01 -5.80067173e-01
9.53975795e-04  4.17233121e-01 -3.52308509e-01 -5.61127701e-01
-1.97685931e-01 -1.10428621e-01 -6.72140651e-01 -7.35340154e-01
-4.03476586e-01 -6.62276959e-02 -5.46928119e-02 -6.36434668e-01
-5.06797074e-01 -6.41775841e-01  5.10405139e-03 -1.02612648e-01
-3.24827906e-01 -6.13307977e-01  3.36079536e-02 -5.28126228e-02
-7.17024526e-02 -5.28075454e-01 -5.37408597e-01 -1.82701981e-02
-1.82061308e-02 -6.32480771e-01 -5.17323711e-01 -6.32810623e-01
-1.38594786e-02 -7.32891794e-01 -1.79236918e-01 -5.96430851e-01
4.92988268e-01 -2.96883338e-01 -6.41091846e-01 -7.62998388e-01
-5.21725243e-01 -1.18810573e+00 -1.12229248e+00 -5.94322506e-01
-4.87597420e-01  1.33782534e+00  7.23001634e-03  2.91957719e-01
-1.24222046e-01 -5.36120913e-01 -1.96125955e-01  4.60206138e-01
-5.73627448e-01  5.95553141e-01  1.55596748e+00 -7.96353548e-01
-7.97282220e-01 -5.70108126e-01 -3.98467462e-01 -1.87381067e+00
3.88400550e-02 -1.74026341e-02 -1.14056211e-01 -5.24301711e-01
-1.80193824e-01  8.77665426e-01 -1.33343889e-01  8.02970910e-01
-1.52474770e+00  1.08354771e+00  1.06325859e+00 -6.04160470e-01
-4.81100147e-01  1.16815001e+00 -1.89612675e-02 -4.33639475e-02
-2.16173569e-02 -5.49168148e-01 -2.17190372e-01  6.78199752e-01
-5.63177778e-01 -7.57973101e-01  1.23187758e+00 -6.41140374e-01
-5.54628614e-01  1.68626773e+00 -2.92316492e-01 -5.19604123e-01
2.01037675e+00 -2.79294538e+00 -2.16946915e+00  4.50633029e-02
-1.10121096e-01 -6.39898142e-01 -5.92343312e-01 -5.52580570e-01
2.00000000e+00]]

```

Shape of merged_full_data (original + synthetic with cycling IDs): (4576, 141)

First 5 rows of merged_full_data (original):

```

[[ 0.00000000e+00  6.71004380e-01  6.65697019e-01 -5.37778581e-01
-4.28326684e-01 -7.54865691e-01  2.02034806e-01 -3.19978904e-01
-1.84473052e-01 -5.30547118e-01 -3.43669644e-01 -7.26423695e-01
-4.30124356e-01 -5.30567384e-01 -7.07269212e-01  6.97687529e-01
7.28512905e-01 -5.60651075e-01 -2.75516290e-01  6.81049583e-01
2.36962501e-02 -1.43306864e+00 -2.65941026e-01 -5.08910035e-01
-6.40926623e-02 -6.33530677e-01 -4.14973460e-01 -7.89238006e-01
6.96778460e-01 -5.54674622e-01 -5.81391582e-01 -3.98382647e-01
-4.67781210e-01 -5.91645716e-01 -2.14002211e-01  1.15801566e+00
-5.09796537e-01 -2.95016077e-01 -3.39982130e-01 -1.03363335e+00

```

-4.96115865e-01	-5.76301591e-01	-6.38668014e-01	1.08054005e-01
3.37008405e-02	-4.91666782e-01	-3.83524977e-01	-5.23168179e-01
1.09508004e-02	-2.12706536e+00	1.08563617e+00	-5.15041597e-01
1.23327143e+00	-4.22105694e-01	9.59174101e-01	-5.82691223e-01
-4.00536500e-01	-1.15881283e-01	-9.24868404e-02	-5.99145233e-01
-5.05502825e-01	-6.06498195e-01	2.29918733e-02	-3.53089523e-01
-4.05421166e-01	-5.75652293e-01	-3.32502929e-01	-3.88308398e-01
-2.13796226e-02	-4.20337509e-01	-5.36293196e-01	-1.45418003e-02
-1.20981740e-02	-6.06283066e-01	-5.16823300e-01	-6.11668079e-01
-1.45341244e-02	-4.35634922e+00	2.68638404e+00	-5.90690330e-01
2.13831831e-01	2.83974419e-02	1.42889938e+00	-1.29451400e-01
-5.21222803e-01	7.38649633e-02	6.64937376e-02	-5.47237125e-01
-4.84555945e-01	-8.03606477e-01	1.02832281e-01	6.33122798e-01
-1.28936486e-01	-5.03297090e-01	-1.96125955e-01	-1.62010708e-01
-5.46800056e-01	-8.98546778e-01	-7.73821305e-01	5.50168093e-01
5.36907964e-01	-4.90924565e-01	-3.86232730e-01	3.71640443e-02
-3.11321153e-02	-2.42507429e-02	-1.08878635e-01	-5.01531419e-01
-1.80193824e-01	-1.82361109e-01	-3.20935806e-01	-1.40666137e+00
-1.63032554e-01	9.62840308e-01	9.44474798e-01	-5.99400929e-01
-4.80865547e-01	1.01261722e+00	2.36540824e-02	3.88840158e-01
-8.89363197e-02	-5.45640451e-01	-2.17190372e-01	2.79950670e-01
-5.62984872e-01	-8.28709321e-01	1.00131456e+00	-6.82610470e-01
-5.52740554e-01	-4.66221476e-01	-2.83615065e-01	-5.14352627e-01
-6.30200258e-01	5.25095728e-01	6.68842819e-01	5.26647501e-02
-7.09707646e-01	-6.74897088e-01	-6.33991637e-01	-5.51311067e-01
1.00000000e+00]			
[0.00000000e+00	6.88744125e-01	6.67421810e-01	-3.46532924e-01
-3.95214358e-01	-7.69852484e-01	5.05211027e-01	1.10636703e+00
7.51589840e-01	-5.07930421e-01	-3.43669644e-01	-1.41512461e-01
-3.63173273e-01	2.20693614e-01	-7.14899537e-01	6.98718040e-01
7.28067775e-01	-5.65322841e-01	-2.75679542e-01	6.82071586e-01
1.04732342e-01	-9.23054761e-01	-3.02119877e-01	-5.08590165e-01
-6.40926623e-02	5.12086474e-02	-4.15163164e-01	-7.89238006e-01
6.98409805e-01	-5.47087958e-01	-5.54201683e-01	-4.21129951e-01
-4.72476537e-01	-5.84145235e-01	-3.63473734e-01	3.90791272e-01
-5.15507979e-01	-3.38061501e-01	-3.39982130e-01	-1.10077139e+00
-4.99387052e-01	-5.76301591e-01	-6.34994020e-01	1.54021818e-01
3.65784020e-02	-5.12666030e-01	-3.84907119e-01	-5.47073682e-01
-1.27789584e-02	-4.11231320e-01	-3.26643036e-01	-5.00667916e-01
2.48196486e+00	-6.22186026e-01	1.02214351e+00	-7.35340154e-01
-4.02534529e-01	-1.29438853e-01	-1.07708934e-01	-4.79345795e-01
-4.94740853e-01	-4.98651315e-01	-3.46813357e-02	-2.68442279e+00
2.32156080e+00	-5.33162489e-01	-5.53266367e-01	-6.52223846e-01
9.65274169e-01	-3.12599563e-01	-5.28193918e-01	4.28026467e-02
7.26135894e-03	-6.08233181e-01	-5.16877819e-01	-6.08842940e-01
-1.61638763e-02	1.74143106e-01	6.63294997e-01	-5.85171987e-01
9.83050697e-01	3.59854732e-01	9.45597254e-01	-2.74200291e-03
-5.20988300e-01	1.15113083e-01	1.14271719e-01	-5.37130433e-01
-4.83706600e-01	-8.73061906e-01	1.44149004e-01	1.27094020e-01
-1.17590611e-01	-4.88574732e-01	-1.96125955e-01	-8.14325533e-01
-5.38457603e-01	-7.98066097e-01	-8.17157980e-01	5.49559198e-01
5.36907964e-01	-5.09024786e-01	-3.89297620e-01	3.54707087e-02
-2.20187203e-01	4.34791326e-02	-9.82286317e-02	-4.97045172e-01
-1.80193824e-01	2.98098167e-01	-3.22157775e-01	-1.40666137e+00

-1.64816290e-01	9.71897546e-01	9.54566708e-01	-6.02501167e-01
-4.81020315e-01	1.02427796e+00	-4.17737580e-02	4.24654481e-01
-8.50236152e-02	-5.46115728e-01	-2.17190372e-01	5.68171204e-01
-5.63073307e-01	-8.28709321e-01	1.01818177e+00	-6.59482151e-01
-5.51765959e-01	-4.71463253e-01	-2.83344483e-01	-4.90977865e-01
-1.03737768e-01	5.31455784e-01	6.73381451e-01	2.20291786e-01
-4.19444079e-01	-6.55075241e-01	-6.14876899e-01	-5.47145277e-01
1.00000000e+00]			
[0.00000000e+00	7.39554373e-01	7.38650264e-01	-3.37032412e-01
-3.93030544e-01	-8.22469621e-01	8.48862132e-02	-3.24128384e-01
5.53082923e-01	-3.72786391e-01	-3.43669644e-01	-8.36094551e-01
-3.50468610e-01	-5.30567384e-01	-7.38092345e-01	7.00618745e-01
7.29307780e-01	-5.78494648e-01	-2.76094148e-01	6.83951459e-01
3.05002238e-02	-1.29330144e+00	-1.29722314e-02	-5.38092058e-01
-6.40926623e-02	4.13717702e-01	-4.16140157e-01	1.40985835e+00
7.01416177e-01	-5.31531206e-01	-5.31787445e-01	-5.22229529e-01
-4.89210874e-01	-5.69080990e-01	2.98106286e-01	-8.92700112e-01
1.99468752e-01	-5.85890619e-01	-3.39982130e-01	5.10541599e-01
-5.18547382e-01	-5.12403527e-01	-6.27441981e-01	2.28039760e-02
-5.20708918e-02	-5.43255549e-01	-3.86477878e-01	-5.52001929e-01
-1.09611026e-02	-9.86577027e-01	4.04745302e-01	-5.56712277e-01
6.08924712e-01	4.58247768e-01	3.48971187e-01	-5.82691223e-01
-4.01653469e-01	-1.03357555e-01	-7.05500422e-02	-4.83081291e-01
-4.95228561e-01	-5.03443407e-01	2.53611365e-02	-3.02847720e+00
2.73307487e+00	-5.65368107e-01	-3.32502929e-01	-4.32294306e-01
1.09594933e+00	-3.12599563e-01	-5.28856683e-01	1.44808029e-02
-9.27057875e-03	-6.08191174e-01	-5.16876674e-01	-6.11227544e-01
3.00033500e-02	4.55769124e-01	6.96377065e-01	-5.87979455e-01
2.74126525e+00	1.75711793e-01	1.32458439e+00	1.23967395e-01
-5.21142582e-01	1.76377562e-01	1.85965029e-01	-5.24916248e-01
-4.82587739e-01	-9.76191965e-01	4.77660808e-02	-1.75220043e-01
-1.24953821e-01	-4.59202901e-01	-1.96125955e-01	-6.27949869e-01
-5.29095432e-01	-8.98546778e-01	-8.77735280e-01	5.52254731e-01
5.40503243e-01	-5.55140099e-01	-3.96388140e-01	4.12408693e-02
-1.30738715e-03	-1.02846576e-01	-9.10409770e-02	-5.14726263e-01
-1.80193824e-01	8.03844774e-01	-3.26565239e-01	7.12479740e-01
-1.57886161e-01	9.87467988e-01	9.69368177e-01	-6.03450998e-01
-4.81066457e-01	1.04433950e+00	9.61552029e-02	-3.57159043e-02
-1.33899589e-01	-5.46115728e-01	-2.17190372e-01	5.68171204e-01
-5.63072178e-01	-8.28709321e-01	1.04734274e+00	-6.51906601e-01
-5.52097168e-01	-5.00705900e-01	-2.83041635e-01	-5.08138160e-01
8.24612870e-02	5.50149291e-01	7.21030746e-01	-1.77674318e-02
1.19616832e-01	-6.48678938e-01	-6.06514683e-01	-5.47420741e-01
1.00000000e+00]			
[0.00000000e+00	7.60981432e-01	7.51208894e-01	-5.96856036e-01
-4.34376768e-01	-8.49552393e-01	-8.21590030e-02	2.71124834e-01
-2.45963606e-01	-5.53302851e-01	-3.43669644e-01	1.14386203e-01
-4.38441466e-01	-5.30567384e-01	-7.48493847e-01	7.05503560e-01
7.29530345e-01	-5.81967506e-01	-2.76192230e-01	6.88826685e-01
1.37911269e-01	1.47307334e+00	2.30914418e-01	-5.49115284e-01
-6.40926623e-02	5.34554053e-01	-4.16459451e-01	-7.89238006e-01
7.09186297e-01	-5.42672189e-01	-5.31194845e-01	-4.05222382e-01
-4.69228939e-01	-5.79498694e-01	-6.37892291e-01	-3.45788869e+00
6.82861355e-01	-5.75601337e-01	-3.39982130e-01	-8.99357265e-01

-4.94400596e-01	-5.76301591e-01	-6.32773242e-01	2.39490393e-01
-7.92297893e-03	-5.22642017e-01	-3.85477057e-01	-5.63947729e-01
-1.79269616e-02	1.92051455e+00	2.42469963e-02	-5.54157201e-01
2.96751354e-01	-9.02298491e-01	7.48387974e-01	-7.35340154e-01
-4.03958828e-01	-5.46978316e-02	-5.37188342e-02	-5.98502386e-01
-5.05471955e-01	-6.07063783e-01	1.16572501e-01	1.05149012e+00
1.90328804e-02	-5.94281312e-01	-1.11739490e-01	-2.12364766e-01
1.90439233e-01	-9.71236719e-02	-5.36361489e-01	-1.54941672e-02
-2.18652886e-02	-6.29050176e-01	-5.17286714e-01	-6.30328541e-01
1.28693407e-02	-4.25700768e-01	-1.28153278e-01	-5.94383863e-01
2.52148843e+00	9.49112138e-01	-2.74993525e-02	-7.62998388e-01
-5.21678603e-01	2.13540241e-01	2.09854020e-01	-5.97895789e-01
-4.87769838e-01	-1.04245383e+00	-4.40412710e-02	-1.91215548e-01
-7.60153069e-02	-5.41870395e-01	-1.96125955e-01	8.16461530e-01
-5.59448616e-01	1.11106685e+00	-9.13274788e-01	5.66847558e-01
5.51271194e-01	-5.38336690e-01	-3.93923953e-01	7.58937730e-02
1.57873429e-01	4.54434126e-01	-7.51488800e-02	-5.32407354e-01
-1.80193824e-01	6.52120792e-01	-3.25481259e-01	-1.40666137e+00
-1.18175601e-01	9.95535677e-01	9.84393910e-01	-5.72399652e-01
-4.79248337e-01	1.05486086e+00	-1.73042047e-01	-1.21230517e+00
1.72069099e-01	-5.45640451e-01	-2.17190372e-01	8.04133775e-02
-5.62221665e-01	-8.28709321e-01	1.06261885e+00	-6.55060993e-01
-5.53548196e-01	-5.38599704e-01	-2.85498505e-01	-5.14995190e-01
-9.13339690e-01	5.69902910e-01	7.68723338e-01	9.14637554e-02
1.19616832e-01	-6.51583125e-01	-6.08068231e-01	-5.52072090e-01
1.00000000e+00]			
[0.00000000e+00	7.26448039e-01	7.22318654e-01	-4.41795080e-01
-4.14291675e-01	-8.11211726e-01	-1.68393384e-01	6.69170011e-01
8.85444895e-01	-5.61691421e-01	-3.43669644e-01	5.16512677e-01
-3.95683250e-01	-5.30567384e-01	-7.32737464e-01	7.12303151e-01
7.35539599e-01	-5.15399898e-01	-2.73496008e-01	6.95741012e-01
-1.26170927e-02	1.07838788e+00	1.41198313e+00	-4.99289318e-01
-6.40926623e-02	-7.14088245e-01	-4.10206510e-01	-7.89238006e-01
7.20126038e-01	-5.91121552e-01	-5.80223811e-01	-2.41147080e-01
-4.25982423e-01	-6.26673871e-01	-1.78203433e-01	-4.73230705e-01
-5.44403930e-01	-4.78806178e-02	-3.39982130e-01	-1.20147845e+00
-4.39797912e-01	-5.76301591e-01	-6.55654210e-01	1.13571272e-01
-3.88929415e-02	-4.68760544e-01	-3.81735249e-01	-5.02829900e-01
5.29150084e-02	4.94762465e-02	-3.05963293e-01	-4.47562171e-01
1.70153147e+00	-8.22266358e-01	1.41579823e+00	-5.82691223e-01
-3.98933797e-01	-4.94084208e-02	-4.21923287e-02	-5.47979479e-01
-5.02138698e-01	-5.61492155e-01	-6.16808211e-02	7.76780536e-02
-6.72372412e-02	-5.62226908e-01	-3.76655616e-01	-5.64252030e-01
5.71598367e-01	-4.20337509e-01	-5.33860882e-01	-8.74818744e-02
-2.96216689e-02	-5.79632419e-01	-5.15799862e-01	-5.88838212e-01
-5.96636935e-02	-3.09403993e+00	2.46140688e+00	-5.88109417e-01
7.63273878e-01	1.02054618e-01	2.03499885e+00	-7.62998388e-01
-5.20429612e-01	1.54317759e-01	1.56755668e-01	-5.26589143e-01
-4.82746959e-01	-9.38928179e-01	-1.63370341e-01	4.35703444e-02
-1.47789116e-01	-4.59202901e-01	-1.96125955e-01	-9.54107282e-01
-5.31704697e-01	-8.98546778e-01	-8.56405714e-01	5.87875022e-01
5.69229704e-01	-5.11827942e-01	-3.89758065e-01	1.25868131e-01
2.17572428e-01	5.89980252e-01	-1.60977809e-01	-5.14726263e-01
-1.80193824e-01	-4.35234413e-01	-3.25380951e-01	-1.40666137e+00

```
-5.99829341e-02  9.66304144e-01  9.46941709e-01 -5.24617423e-01
-4.75202642e-01  1.01747496e+00 -6.15199981e-02  3.81445712e-01
-2.00246509e-01 -4.40128962e-01 -2.17190372e-01 -6.29052552e-01
-5.60681196e-01 -8.28709321e-01  1.00804525e+00 -6.68634292e-01
-5.47983311e-01 -5.74807139e-01 -2.75240323e-01 -4.96019756e-01
 7.88391197e-02  5.56086914e-01  7.39856682e-01  1.64479603e-01
-4.60910303e-01 -6.62289510e-01 -6.25503410e-01 -5.49433323e-01
 1.00000000e+00]]
```

Last 5 rows of merged_full_data (synthetic with cycling IDs):

```
[[ 5.00000000e+00  7.16814572e-01  7.40192792e-01  1.24752653e+00
 6.85000356e-01 -6.86657035e-01  2.25889821e-01  1.71740284e+00
 6.18504315e-01  9.83444811e-01  5.11729215e-01 -8.12520023e-01
 9.13708824e-01 -2.54286317e-01 -7.01594271e-01  7.98335035e-01
 7.26222607e-01  1.66898422e+00  7.70528774e-01  8.27197391e-01
 1.18005655e-01  1.23713710e+00 -1.44308541e-02  1.41134448e+00
-6.40926623e-02 -9.87002929e-01  9.38606560e-01 -3.18003073e-01
 8.97311413e-01 -8.72749658e-01 -8.92136949e-01  9.49139937e-01
 4.21442338e-01 -8.52805563e-01 -6.90634826e-01  1.62716611e+00
 3.47078133e-01  9.44033606e-01 -7.76354786e-02 -7.78099981e-01
 1.32525297e+00 -4.27902449e-01 -7.55556337e-01 -1.30477743e+00
-1.28977681e+00  4.51817648e-01 -6.56687455e-02  4.82400647e-01
 1.43565468e-01 -4.10815159e-03 -5.40428384e-01  4.54370005e-01
 9.76528933e-01 -7.15120765e-01  1.31085147e+00 -2.77393360e-01
-4.19171519e-03  1.05165815e+00  1.13372089e+00  1.26825441e+00
 8.13577257e-01  1.17892346e+00 -1.01204879e+00 -3.81044077e-01
-6.73943775e-02  9.79719522e-01 -6.85724430e-01 -6.67844505e-01
 1.98040550e-02 -2.04861618e-01  6.54863968e-01 -4.31659630e-01
-5.22230226e-01  1.35831498e+00  9.49825782e-01  1.34242631e+00
 1.57745525e+00  3.95463805e-02 -2.75808897e-01  9.96392760e-01
 5.08065120e-01 -7.20049164e-01  5.73277237e-01 -3.94551072e-02
 9.14672392e-01  2.02043846e+00  2.13402421e+00  9.00519567e-01
 3.77700419e-01 -1.49310157e-01 -1.74422006e+00 -5.24326822e-01
-7.77930178e-02  5.66825530e-01  2.01736775e-02 -1.28026469e+00
 9.02507666e-01 -8.49948949e-01 -4.15516438e-01  1.09491374e+00
 1.12166895e+00  7.10164900e-01  1.92918499e-01  1.35542823e+00
-1.15406193e+00 -1.89765701e-01 -8.75409527e-02  6.78996449e-01
-1.80193824e-01 -1.38232606e+00 -2.00839290e-01 -1.35541488e+00
 1.70145603e+00 -6.76866583e-01 -8.44534351e-01  1.59899501e+00
 1.27248903e+00 -6.93527535e-01  2.17456157e+00  7.05897918e-01
-2.08276477e-01  1.76426058e+00  9.76292899e-02 -1.29272720e+00
 1.08151537e+00 -8.28709321e-01 -8.04935928e-01  3.06359063e-01
 1.84767070e+00 -8.74774067e-01  6.91161861e-01  6.47678920e-01
-2.53992862e-01  6.78230523e-01  6.85800046e-01 -8.16279704e-01
-7.92640094e-01  5.37224707e-01  4.94284956e-01  6.97620013e-01
 4.00000000e+00]
[ 6.00000000e+00  3.82770010e-01  3.62888896e-01  1.27475028e+00
 7.27676801e-01 -3.63388906e-01  8.77293573e-01 -4.37072466e-02
-3.26934856e-01  1.97365720e+00 -3.43669644e-01 -1.16422295e+00
 2.28673379e-01 -3.05189084e-01 -5.18998277e-01  8.92518894e-01
 8.79820505e-01  1.12041658e+00  3.34592544e-01  9.01567030e-01
-9.89329829e-02  1.41102395e-01 -3.77896251e-01  1.30872552e+00
-6.40926623e-02 -7.39585667e-01  3.91951005e-01 -2.39463917e-01
 1.03857457e+00 -1.05829188e+00 -1.06316501e+00  8.93341843e-01
 3.59790414e-01 -1.02119436e+00 -4.69000181e-01 -1.87993403e-01
```

-2.61391579e-01	7.79540589e-01	2.91457934e+00	-6.84812201e-01
2.11738212e+00	-3.53431089e-01	-8.05380258e-01	-2.11099596e+00
-1.70800895e+00	8.61380301e-01	2.27732072e-01	9.37376088e-01
-3.45190778e-02	-2.73497548e-01	-4.99640388e-01	9.02249178e-01
1.15711565e+00	-7.83218125e-01	9.58151804e-01	2.58723892e-01
3.89356885e-01	2.28560515e+00	3.00559233e+00	2.60888776e+00
3.26096122e+00	2.53709619e+00	9.07189783e-01	-2.56293150e-01
-4.12642290e-01	2.80235303e+00	-1.17247188e+00	-1.09101908e+00
-1.59225369e+00	-2.04861618e-01	3.16226209e+00	8.95515330e-01
2.40253846e+00	2.21541484e+00	2.47271434e+00	2.19861015e+00
3.00639669e-01	-3.45612994e-01	-4.65444762e-01	2.76249285e+00
-7.18890762e-01	-1.13125770e+00	-6.16590417e-01	-3.82870196e-01
2.41364486e+00	-2.22082262e-01	-3.35079482e-01	1.25578593e+00
8.04363751e-01	1.80425026e-01	9.55609912e-02	8.61833517e-01
-1.20929705e-01	8.18840393e-01	2.24368236e-01	-1.37401599e+00
7.06381481e-01	-8.98546778e-01	-1.66456304e-01	6.68297322e-01
7.17583507e-01	1.24256344e+00	6.43677686e-01	3.92580904e-01
8.99587843e-01	-3.25066874e-01	-1.16665813e-01	7.96582077e-01
-1.80193824e-01	-1.40849083e+00	2.91189831e-02	-1.24772579e+00
2.22712289e-01	-9.59659214e-01	-9.99834411e-01	1.69225354e+00
1.40109537e+00	-9.64857999e-01	-2.26913457e+00	7.57106555e-01
-1.81284041e-01	1.72692357e+00	5.33284478e-01	-1.38286010e+00
2.02732832e+00	-8.28709321e-01	-1.00460781e+00	2.28630883e+00
8.38974057e-01	-6.21699357e-01	2.97990155e-01	6.98411141e-01
-9.05209230e-01	9.70681055e-01	3.47084439e-01	-1.86311533e+00
-7.51173870e-01	2.12542499e+00	2.20627402e+00	2.68493616e+00
4.00000000e+00]			
[7.00000000e+00	2.85597766e-01	3.06926810e-01	1.32477254e+00
7.73740209e-01	-2.65376311e-01	-1.26342786e+00	-6.54509301e-01
-5.98792334e-02	9.06886828e-01	-3.43669644e-01	-1.00119341e+00
2.23261717e-01	-1.54936885e-01	-4.54656739e-01	7.29801565e-01
7.31807771e-01	5.83735181e-01	2.15328602e-02	7.26440053e-01
9.78363558e-01	-1.29176113e-01	-2.80696365e-01	5.52415916e-01
-6.40926623e-02	-1.17144688e+00	3.33490217e-02	-3.96542228e-01
7.58851220e-01	-8.57655983e-01	-8.54730424e-01	3.67307727e-01
-1.09005320e-01	-8.74022575e-01	2.20539944e-01	4.11545229e-01
3.74405711e-02	2.73324769e-01	-5.12803885e-02	-7.96485618e-01
3.38150983e-01	-4.48505463e-01	-7.56743564e-01	-2.39191012e+00
-2.12562937e+00	2.82930659e-01	-1.57823366e-01	4.80681692e-01
8.24006583e-02	-3.38319985e-01	-3.67673325e-01	2.70815404e-01
8.55649546e-01	-9.17794564e-01	5.64183033e-01	-4.64520182e-01
7.92793635e-02	2.02060283e+00	1.98210074e+00	1.30796770e+00
8.71439247e-01	1.41747921e+00	3.57222712e-01	-1.04918491e-01
-3.15603772e-01	1.29881580e+00	-9.50640557e-01	-1.01688222e+00
-1.38107442e+00	-3.12599563e-01	1.19911957e+00	4.71579174e-01
7.58922908e-01	1.11819197e+00	6.24815769e-01	1.10410359e+00
5.92722366e-01	-2.60366201e-01	-4.48444230e-01	1.48176420e+00
-4.45498626e-01	-1.06339241e+00	1.48206521e-01	-5.09579593e-01
5.99256347e-01	8.82490156e-01	1.31396363e+00	2.49951924e+00
2.90836243e+00	5.51883493e-02	-3.14388078e+00	6.53734119e-02
-1.95901898e-01	3.10662570e+00	-8.97155807e-02	-1.46664036e+00
1.55041357e+00	-8.98546778e-01	-6.58471284e-01	8.86199811e-01
8.58891782e-01	1.62797815e+00	1.07572391e+00	9.22271381e-01
-1.82760498e+00	3.62371877e-01	-1.88212588e-01	1.58729406e+00

-1.80193824e-01	-1.40757596e+00	1.63997769e-02	-1.40666137e+00
9.51564272e-01	-2.86904943e-01	9.27918357e-02	2.26448567e+00
2.41561614e+00	-2.13896170e-01	2.11927806e+00	-3.00966991e-01
-2.42957178e-01	3.00928728e+00	3.03156586e-01	-1.40360130e+00
7.26700554e-01	-8.28709321e-01	-5.54672611e-01	5.12897964e-03
8.23390220e-02	-5.03169650e-01	3.85880384e-02	4.29617896e-01
1.21911602e-01	4.46843246e-01	3.32097049e-01	-2.02099389e+00
-8.34106318e-01	9.80046103e-03	-4.19750255e-02	8.88759484e-01
4.00000000e+00]			
[8.00000000e+00	-4.95141724e-01	-4.71225224e-01	5.22012841e-01
1.72460375e-02	4.84968064e-01	5.81570507e-01	-7.96896019e-01
-1.22339152e-01	6.02693331e-01	-3.43669644e-01	-9.17141162e-01
-3.11115417e-01	-1.54936885e-01	2.14646458e-01	5.88554361e-01
5.84909695e-01	3.98888393e-01	-6.21169804e-02	5.82084475e-01
-8.17880789e-01	9.00895946e-01	-6.99238531e-03	2.95579216e-01
-6.40926623e-02	-1.16540622e+00	-3.87278706e-02	7.46927048e-02
5.34367285e-01	-9.54199450e-01	-9.48314543e-01	7.86650851e-01
2.51548488e-01	-9.39235118e-01	-8.79484513e-01	2.42597941e-01
-3.25678206e-01	8.35750623e-01	7.97138884e-01	-1.11452502e+00
1.14188539e+00	-2.56811270e-01	-7.81740825e-01	3.75839613e+00
3.60769143e+00	5.41553950e-01	-2.91926670e-04	7.99031575e-01
-4.51310131e-01	7.47106609e-02	-9.98141851e-02	5.57921811e-01
4.84813319e-01	-8.54570686e-01	6.99585563e-01	2.79045028e-02
3.80010762e-01	-2.43097342e+00	-2.69023959e+00	5.88846132e-01
5.24586806e-02	9.90198565e-01	-9.54659725e-02	3.52970521e-01
-3.15500716e-01	8.06808143e-01	-1.07458969e+00	-1.04809702e+00
-2.18462798e+00	-3.12599563e-01	7.83589325e-01	-1.70119094e+00
-1.11099280e+00	5.66353924e-01	2.06160398e-02	5.75910857e-01
-1.16775157e+00	5.43700350e-01	-1.59270800e-01	2.72465611e-01
4.34942683e-01	-6.85987743e-01	6.89939187e-01	-1.29451400e-01
4.44921881e-02	1.62054901e-01	-3.41039163e-01	1.92811269e+00
1.79179469e+00	6.40189413e-02	-1.70124460e+00	8.02867405e-01
-1.57119976e-01	1.70288496e+00	-8.97155807e-02	-1.44811592e+00
1.48826886e+00	-8.98546778e-01	-4.49829144e-01	8.75513313e-01
8.77529452e-01	2.22884537e+00	1.88022176e+00	9.47778600e-01
-2.56629704e+00	7.64924929e-01	-2.10405125e-01	2.21526954e+00
-1.80193824e-01	-1.80075025e+00	1.46076262e-01	-1.40666137e+00
9.54931105e-01	-1.53466923e+00	-1.59226943e+00	1.82411074e+00
1.59425067e+00	-1.04945073e+00	-1.29451387e-01	5.26415055e-01
-2.23423979e-01	2.16548056e+00	1.35583881e-01	-1.40503091e+00
1.54133739e+00	-8.28709321e-01	-1.06142408e+00	-1.22902068e-01
8.08659970e-03	-1.71932625e-02	-5.72217210e-02	1.78685458e-01
-1.52051963e+00	-1.47708092e-01	-5.35932352e-01	3.18110260e+00
-6.84223837e-01	-1.11411405e-01	-1.20644300e-01	5.03879178e-01
4.00000000e+00]			
[9.00000000e+00	7.48867883e-01	7.28476828e-01	1.07525571e+00
5.27605277e-01	-7.35317273e-01	-7.48349634e-01	4.37574883e-01
-2.25533313e-01	1.25700161e+00	5.11729215e-01	-7.98577590e-01
6.95326215e-01	-1.40771819e-01	-7.16705256e-01	7.23370490e-01
6.81278251e-01	1.21563319e+00	4.01372591e-01	7.37034417e-01
-2.65018369e-01	9.95015917e-01	-1.47968187e-01	1.23357454e+00
-6.40926623e-02	-9.68834627e-01	5.16499781e-01	-2.52210028e-01
7.62228741e-01	-5.08174000e-01	-5.23127766e-01	9.56494021e-01
4.28547588e-01	-5.04876032e-01	-1.53992348e+00	2.92892036e-01

```

-3.13792864e-01  9.85778756e-01 -3.39982130e-01 -1.03908126e+00
 2.48963460e-01 -5.12403527e-01 -6.04943263e-01 -1.06906704e+00
-7.12532026e-01  4.93181775e-01 -4.06755614e-02  5.00654552e-01
-2.51896247e-01 -2.74471589e-01 -4.76638020e-01  5.29176189e-01
 8.70435489e-01 -1.00935828e+00  1.47056814e+00 -4.79589049e-01
-6.08300098e-03  2.74773953e+00  2.10107133e+00  1.35313166e+00
 9.36268224e-01  1.67241205e+00 -2.30487533e+00  4.33011096e-01
-4.46887449e-01  1.79843924e+00 -9.29143952e-01 -8.49582883e-01
-1.77899023e+00 -5.28075454e-01  1.85034930e+00  2.64617296e+00
 3.08885058e+00  9.51193342e-01  4.14852278e-01  9.91304438e-01
-2.14988413e+00 -6.35633481e-01 -3.71097019e-01  9.62871232e-01
-5.55387035e-01 -1.01475683e+00  7.23566195e-01 -3.62306541e-01
 5.18194555e-01  6.41147728e-01  8.33590197e-01  1.14903964e+00
 6.48755308e-01 -8.54505008e-01 -1.00491024e+00 -2.32059694e-01
-1.18410199e-01  5.72220565e-01  1.75659385e+00 -1.41370831e+00
 2.11380867e+00 -8.98546778e-01 -9.94325296e-01  7.77728717e-01
 7.92617732e-01  7.84769619e-01  2.33137508e-01  6.17098377e-01
 1.24384487e-01  6.23756673e-01 -1.96414801e-01  6.53762940e-01
-1.80193824e-01 -1.60865155e+00 -1.39008308e-01 -1.40666137e+00
 5.48117685e-01 -4.38277102e-02  3.79511055e-01  2.20749916e+00
 2.28283548e+00  2.93510102e-02 -1.33286145e+00 -1.45671335e+00
 2.25391005e-02  1.24999153e+00 -5.96352148e-02 -1.40862901e+00
 5.38992780e-01 -8.28709321e-01 -3.23730639e-01  1.96189603e-01
 1.05305517e+00 -4.06708703e-01  3.80258175e-01  4.69194461e-01
-4.96294390e-01  6.69530568e-01  7.42922476e-01 -7.97539233e-01
-7.92640094e-01  3.21075477e-01  2.82194258e-01  1.19847585e+00
 4.00000000e+00]]

```

```

In [55]: print("Originals:")
unique_orig, counts_orig = np.unique(y, return_counts=True)
for label, count in zip(unique_orig, counts_orig):
    print(f"Label {label}: {count} samples")

print("\nSynthetic:")
unique_syn, counts_syn = np.unique(y_synthetic_all, return_counts=True)
for label, count in zip(unique_syn, counts_syn):
    print(f"Label {label}: {count} samples")

print("\nTOTAL (Originals + Synthetic):")
y_combined = np.hstack((y, y_synthetic_all))
unique_comb, counts_comb = np.unique(y_combined, return_counts=True)
for label, count in zip(unique_comb, counts_comb):
    print(f"Label {label}: {count} samples")

```

Originals:
Label 1.0: 1144 samples
Label 2.0: 656 samples
Label 3.0: 638 samples
Label 4.0: 1074 samples

Synthetic:
Label 2.0: 488 samples
Label 3.0: 506 samples
Label 4.0: 70 samples

TOTAL (Originals + Synthetic):
Label 1.0: 1144 samples
Label 2.0: 1144 samples
Label 3.0: 1144 samples
Label 4.0: 1144 samples

5.3 - Generating three new samples of activity 4 of participant 3

```
In [56]: participant_mask = (participants == 3)
X_participant3 = X_zscore_participants[participant_mask]
y_participant3 = y[participant_mask]

X_synthetic3_smote, y_synthetic3_smote = smote(
    X_participant3,
    y_participant3,
    target_class=4,
    n_new=3,
    k=5,
    random_state=42
)

print(f"Generated {len(X_synthetic3_smote)} synthetic samples for activity 4.")
print(f"Synthetic samples shape: {X_synthetic3_smote.shape}.")
print(f"Synthetic labels: {y_synthetic3_smote}.")
print(f"\nFirst synthetic sample:\n{X_synthetic3_smote[0]}")
```

Generated 3 synthetic samples for activity 4.
Synthetic samples shape: (3, 140).
Synthetic labels: [4 4 4].

First synthetic sample:

```
[ 3.00000000e+00 -5.42528811e-01 -4.08075951e-01  2.99792781e+00
 3.54906576e+00  6.88612545e-01  2.32002795e+00 -9.67826871e-01
 6.97717831e-02  2.84580859e+00  5.57483043e-03 -1.02926308e+00
 4.20774665e-01 -5.98076469e-02  3.51939449e-01  5.22908874e-01
 4.85064727e-01  2.12200236e+00  1.22036488e+00  5.77837788e-01
-1.61632032e+00  4.24040474e-01 -2.50114193e-01  2.07404468e+00
-6.40926623e-02 -1.32160613e+00  1.59363705e+00 -2.39463917e-01
 4.81702499e-01 -5.82764061e-01 -5.22366149e-01  3.83956071e+00
 6.43084866e+00 -3.32309372e-01  5.30771951e-01 -2.50608240e+00
 8.49139079e-01  2.72065788e+00  1.80243908e+00 -1.01237978e+00
 2.97004907e+00 -3.84607398e-01 -5.73744261e-01 -5.42409895e+00
-4.16275274e+00  2.63508153e+00  2.58365734e+00  2.89413918e+00
 1.32826598e+00 -6.25771175e-01 -4.16329141e-01  2.55281384e+00
 9.62657841e-01 -9.49654896e-01  1.11191209e+00 -4.02041167e-01
 3.58412567e+00  2.83049285e+00  2.82044674e+00  3.20542109e+00
 4.93187878e+00  3.14914808e+00 -4.01101874e+00  4.09566348e-01
 1.17265013e-01  2.22683700e+00 -8.82975605e-01 -7.16772212e-01
-3.55065417e-01  2.83462957e-01  4.85077983e+00  1.36548984e+00
-1.61401084e-01  2.28239024e+00  2.61349744e+00  2.27012312e+00
-1.29036517e+00  3.66498633e-01 -2.74056021e-01  1.65601260e+00
 2.63248168e-01 -8.75755599e-01  4.98374587e-01  3.09957478e-02
 2.57190141e+00 -1.94030151e-01 -3.90465078e-01  1.32428481e+00
 8.61520814e-01  1.48730269e-01  1.02137268e-01  1.06195779e+00
-8.45189973e-02  6.85526354e-01  1.90476814e-01 -1.44334340e+00
 7.42079859e-01 -8.98546778e-01 -2.08216945e-01  5.51239659e-01
 6.01287809e-01  1.98212858e+00  1.54532022e+00  1.96268216e-01
 1.26063175e+00 -6.55135947e-02 -1.57330969e-01  1.58119197e+00
-1.80193824e-01 -1.69451268e+00  3.71449707e-01 -1.40666137e+00
-6.80200167e-02 -7.98403382e-01 -7.07815790e-01  1.16656576e+00
 6.43781674e-01 -9.88470050e-01 -1.07066110e+00 -1.88415903e-01
-9.97736479e-02  6.17131427e-01  5.39212634e-02 -1.22766443e+00
 1.12589674e+00 -8.28709321e-01 -9.84538348e-01  1.86575853e+00
 3.54582869e+00  9.33645302e-01  1.84776098e+00  5.10019563e+00
-1.03269810e+00  4.42033486e-02 -4.77830149e-01 -3.89739993e+00
-7.51173870e-01  2.16027457e+00  1.54175915e+00  4.17817004e+00].
```

2D Scatter Plot (Only Two features)

- X_zscore_participants: all data without the synthetic (3512,140)
- y_col: activity label for all data without the synthetic (3512,)
- combines_data_features (X_zscore_participants + y_col)
- X_synthetic_all: all synthetic data (1064,140)

- y_synthetic_all: activity label for the synthetic data (1064,)
- combines_data_features_synthetic (X_synthetic_all + y_synthetic_all)

```
In [57]: participant_ids_original = X_zscore_participants[:,0].reshape(-1,1)
X_features_original = X_zscore_participants[:,1:]
```

```
In [58]: X_original_with_ids = np.hstack((participant_ids_original, X_features_original)

print("Shape of X_original_with_ids:", X_original_with_ids.shape)
print("Shape of X_synthetic_all:", X_synthetic_all.shape)

y_original_col = y.reshape(-1, 1)
y_synthetic_all_col = y_synthetic_all.reshape(-1, 1)

combined_original_data = np.hstack((X_original_with_ids, y_original_col))
combined_synthetic_data = np.hstack((X_synthetic_all, y_synthetic_all_col))
```

Shape of X_original_with_ids: (3512, 140)
Shape of X_synthetic_all: (1064, 140)

```
In [59]: feature_x_col = 14
feature_y_col = 15
target_col = -1

colors = {
    1: 'blue',
    2: 'green',
    3: 'red',
    4: 'purple'
}

unique_activities = np.sort(np.unique(combined_original_data[:, target_col]))

plt.figure(figsize=(12, 9))

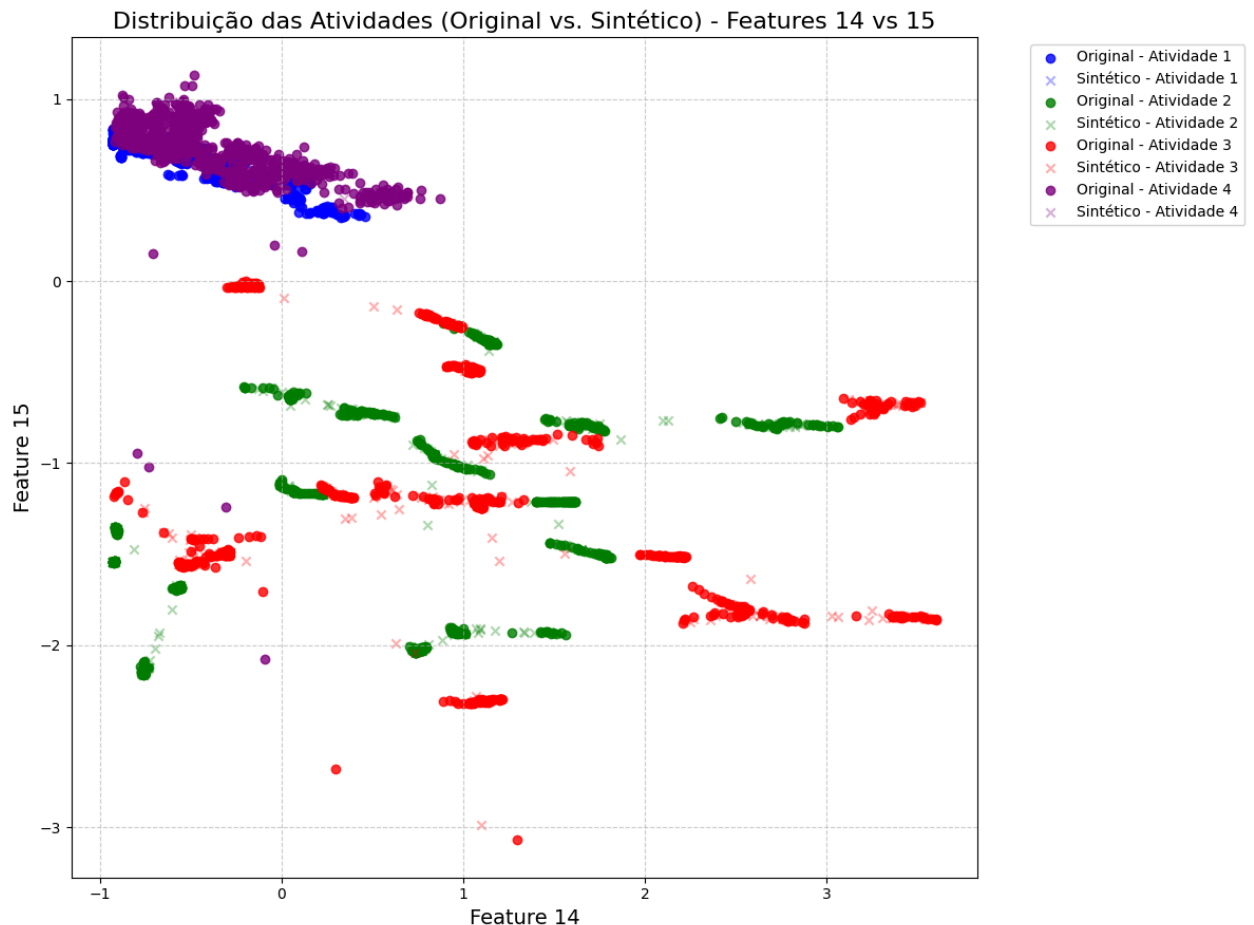
for activity_id in unique_activities:
    mask_original = combined_original_data[:, target_col] == activity_id
    original_data_activity = combined_original_data[mask_original]

    mask_synthetic = combined_synthetic_data[:, target_col] == activity_id
    synthetic_data_activity = combined_synthetic_data[mask_synthetic]

    plt.scatter(
        original_data_activity[:, feature_x_col],
        original_data_activity[:, feature_y_col],
        color=colors[activity_id],
        marker='o',
        alpha=0.8,
        label=f'Original - Atividade {int(activity_id)}'
    )
```

```
plt.scatter(
    synthetic_data_activity[:, feature_x_col],
    synthetic_data_activity[:, feature_y_col],
    color=colors[activity_id],
    marker='x',
    alpha=0.3,
    label=f'Sintético - Atividade {int(activity_id)}'
)

plt.title(f'Distribuição das Atividades (Original vs. Sintético) - Features {f
plt.xlabel(f'Feature {feature_x_col}', fontsize=14)
plt.ylabel(f'Feature {feature_y_col}', fontsize=14)
plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left') #
plt.grid(True, linestyle='--', alpha=0.6)
plt.tight_layout()
plt.show()
```



6 - Data Splitting

- Merged data (original + synthetic)

```
In [60]: merged_data = np.vstack((combined_original_data, combined_synthetic_data))

print("Shape of merged_full_data (with IDs and target):", merged_data.shape)
print("First 5 rows of merged_full_data:\n", merged_data[:5])
print("Last 5 rows of merged_full_data (should be synthetic):\n", merged_data[
```

Shape of merged_full_data (with IDs and target): (4576, 141)

First 5 rows of merged_full_data:

```
[[ 0.00000000e+00  6.71004380e-01  6.65697019e-01 -5.37778581e-01
-4.28326684e-01 -7.54865691e-01  2.02034806e-01 -3.19978904e-01
-1.84473052e-01 -5.30547118e-01 -3.43669644e-01 -7.26423695e-01
-4.30124356e-01 -5.30567384e-01 -7.07269212e-01  6.97687529e-01
 7.28512905e-01 -5.60651075e-01 -2.75516290e-01  6.81049583e-01
 2.36962501e-02 -1.43306864e+00 -2.65941026e-01 -5.08910035e-01
-6.40926623e-02 -6.33530677e-01 -4.14973460e-01 -7.89238006e-01
 6.96778460e-01 -5.54674622e-01 -5.81391582e-01 -3.98382647e-01
-4.67781210e-01 -5.91645716e-01 -2.14002211e-01  1.15801566e+00
-5.09796537e-01 -2.95016077e-01 -3.39982130e-01 -1.03363335e+00
-4.96115865e-01 -5.76301591e-01 -6.38668014e-01  1.08054005e-01
 3.37008405e-02 -4.91666782e-01 -3.83524977e-01 -5.23168179e-01
 1.09508004e-02 -2.12706536e+00  1.08563617e+00 -5.15041597e-01
 1.23327143e+00 -4.22105694e-01  9.59174101e-01 -5.82691223e-01
-4.00536500e-01 -1.15881283e-01 -9.24868404e-02 -5.99145233e-01
-5.05502825e-01 -6.06498195e-01  2.29918733e-02 -3.53089523e-01
-4.05421166e-01 -5.75652293e-01 -3.32502929e-01 -3.88308398e-01
-2.13796226e-02 -4.20337509e-01 -5.36293196e-01 -1.45418003e-02
-1.20981740e-02 -6.06283066e-01 -5.16823300e-01 -6.11668079e-01
-1.45341244e-02 -4.35634922e+00  2.68638404e+00 -5.90690330e-01
 2.13831831e-01  2.83974419e-02  1.42889938e+00 -1.29451400e-01
-5.21222803e-01  7.38649633e-02  6.64937376e-02 -5.47237125e-01
-4.84555945e-01 -8.03606477e-01  1.02832281e-01  6.33122798e-01
-1.28936486e-01 -5.03297090e-01 -1.96125955e-01 -1.62010708e-01
-5.46800056e-01 -8.98546778e-01 -7.73821305e-01  5.50168093e-01
 5.36907964e-01 -4.90924565e-01 -3.86232730e-01  3.71640443e-02
-3.11321153e-02 -2.42507429e-02 -1.08878635e-01 -5.01531419e-01
-1.80193824e-01 -1.82361109e-01 -3.20935806e-01 -1.40666137e+00
-1.63032554e-01  9.62840308e-01  9.44474798e-01 -5.99400929e-01
-4.80865547e-01  1.01261722e+00  2.36540824e-02  3.88840158e-01
-8.89363197e-02 -5.45640451e-01 -2.17190372e-01  2.79950670e-01
-5.62984872e-01 -8.28709321e-01  1.00131456e+00 -6.82610470e-01
-5.52740554e-01 -4.66221476e-01 -2.83615065e-01 -5.14352627e-01
-6.30200258e-01  5.25095728e-01  6.68842819e-01  5.26647501e-02
-7.09707646e-01 -6.74897088e-01 -6.33991637e-01 -5.51311067e-01
 1.00000000e+00]
[ 0.00000000e+00  6.88744125e-01  6.67421810e-01 -3.46532924e-01
-3.95214358e-01 -7.69852484e-01  5.05211027e-01  1.10636703e+00
 7.51589840e-01 -5.07930421e-01 -3.43669644e-01 -1.41512461e-01
-3.63173273e-01  2.20693614e-01 -7.14899537e-01  6.98718040e-01
 7.28067775e-01 -5.65322841e-01 -2.75679542e-01  6.82071586e-01
 1.04732342e-01 -9.23054761e-01 -3.02119877e-01 -5.08590165e-01
-6.40926623e-02  5.12086474e-02 -4.15163164e-01 -7.89238006e-01
 6.98409805e-01 -5.47087958e-01 -5.54201683e-01 -4.21129951e-01
-4.72476537e-01 -5.84145235e-01 -3.63473734e-01  3.90791272e-01
-5.15507979e-01 -3.38061501e-01 -3.39982130e-01 -1.10077139e+00
-4.99387052e-01 -5.76301591e-01 -6.34994020e-01  1.54021818e-01
 3.65784020e-02 -5.12666030e-01 -3.84907119e-01 -5.47073682e-01
-1.27789584e-02 -4.11231320e-01 -3.26643036e-01 -5.00667916e-01
 2.48196486e+00 -6.22186026e-01  1.02214351e+00 -7.35340154e-01
-4.02534529e-01 -1.29438853e-01 -1.07708934e-01 -4.79345795e-01
-4.94740853e-01 -4.98651315e-01 -3.46813357e-02 -2.68442279e+00]
```

2.32156080e+00	-5.33162489e-01	-5.53266367e-01	-6.52223846e-01
9.65274169e-01	-3.12599563e-01	-5.28193918e-01	4.28026467e-02
7.26135894e-03	-6.08233181e-01	-5.16877819e-01	-6.08842940e-01
-1.61638763e-02	1.74143106e-01	6.63294997e-01	-5.85171987e-01
9.83050697e-01	3.59854732e-01	9.45597254e-01	-2.74200291e-03
-5.20988300e-01	1.15113083e-01	1.14271719e-01	-5.37130433e-01
-4.83706600e-01	-8.73061906e-01	1.44149004e-01	1.27094020e-01
-1.17590611e-01	-4.88574732e-01	-1.96125955e-01	-8.14325533e-01
-5.38457603e-01	-7.98066097e-01	-8.17157980e-01	5.49559198e-01
5.36907964e-01	-5.09024786e-01	-3.89297620e-01	3.54707087e-02
-2.20187203e-01	4.34791326e-02	-9.82286317e-02	-4.97045172e-01
-1.80193824e-01	2.98098167e-01	-3.22157775e-01	-1.40666137e+00
-1.64816290e-01	9.71897546e-01	9.54566708e-01	-6.02501167e-01
-4.81020315e-01	1.02427796e+00	-4.17737580e-02	4.24654481e-01
-8.50236152e-02	-5.46115728e-01	-2.17190372e-01	5.68171204e-01
-5.63073307e-01	-8.28709321e-01	1.01818177e+00	-6.59482151e-01
-5.51765959e-01	-4.71463253e-01	-2.83344483e-01	-4.90977865e-01
-1.03737768e-01	5.31455784e-01	6.73381451e-01	2.20291786e-01
-4.19444079e-01	-6.55075241e-01	-6.14876899e-01	-5.47145277e-01
1.00000000e+00]			
[0.00000000e+00	7.39554373e-01	7.38650264e-01	-3.37032412e-01
-3.93030544e-01	-8.22469621e-01	8.48862132e-02	-3.24128384e-01
5.53082923e-01	-3.72786391e-01	-3.43669644e-01	-8.36094551e-01
-3.50468610e-01	-5.30567384e-01	-7.38092345e-01	7.00618745e-01
7.29307780e-01	-5.78494648e-01	-2.76094148e-01	6.83951459e-01
3.05002238e-02	-1.29330144e+00	-1.29722314e-02	-5.38092058e-01
-6.40926623e-02	4.13717702e-01	-4.16140157e-01	1.40985835e+00
7.01416177e-01	-5.31531206e-01	-5.31787445e-01	-5.22229529e-01
-4.89210874e-01	-5.69080990e-01	2.98106286e-01	-8.92700112e-01
1.99468752e-01	-5.85890619e-01	-3.39982130e-01	5.10541599e-01
-5.18547382e-01	-5.12403527e-01	-6.27441981e-01	2.28039760e-02
-5.20708918e-02	-5.43255549e-01	-3.86477878e-01	-5.52001929e-01
-1.09611026e-02	-9.86577027e-01	4.04745302e-01	-5.56712277e-01
6.08924712e-01	4.58247768e-01	3.48971187e-01	-5.82691223e-01
-4.01653469e-01	-1.03357555e-01	-7.05500422e-02	-4.83081291e-01
-4.95228561e-01	-5.03443407e-01	2.53611365e-02	-3.02847720e+00
2.73307487e+00	-5.65368107e-01	-3.32502929e-01	-4.32294306e-01
1.09594933e+00	-3.12599563e-01	-5.28856683e-01	1.44808029e-02
-9.27057875e-03	-6.08191174e-01	-5.16876674e-01	-6.11227544e-01
3.00033500e-02	4.55769124e-01	6.96377065e-01	-5.87979455e-01
2.74126525e+00	1.75711793e-01	1.32458439e+00	1.23967395e-01
-5.21142582e-01	1.76377562e-01	1.85965029e-01	-5.24916248e-01
-4.82587739e-01	-9.76191965e-01	4.77660808e-02	-1.75220043e-01
-1.24953821e-01	-4.59202901e-01	-1.96125955e-01	-6.27949869e-01
-5.29095432e-01	-8.98546778e-01	-8.77735280e-01	5.52254731e-01
5.40503243e-01	-5.55140099e-01	-3.96388140e-01	4.12408693e-02
-1.30738715e-03	-1.02846576e-01	-9.10409770e-02	-5.14726263e-01
-1.80193824e-01	8.03844774e-01	-3.26565239e-01	7.12479740e-01
-1.57886161e-01	9.87467988e-01	9.69368177e-01	-6.03450998e-01
-4.81066457e-01	1.04433950e+00	9.61552029e-02	-3.57159043e-02
-1.33899589e-01	-5.46115728e-01	-2.17190372e-01	5.68171204e-01
-5.63072178e-01	-8.28709321e-01	1.04734274e+00	-6.51906601e-01
-5.52097168e-01	-5.00705900e-01	-2.83041635e-01	-5.08138160e-01
8.24612870e-02	5.50149291e-01	7.21030746e-01	-1.77674318e-02

```

1.19616832e-01 -6.48678938e-01 -6.06514683e-01 -5.47420741e-01
1.00000000e+00]
[ 0.00000000e+00 7.60981432e-01 7.51208894e-01 -5.96856036e-01
-4.34376768e-01 -8.49552393e-01 -8.21590030e-02 2.71124834e-01
-2.45963606e-01 -5.53302851e-01 -3.43669644e-01 1.14386203e-01
-4.38441466e-01 -5.30567384e-01 -7.48493847e-01 7.05503560e-01
7.29530345e-01 -5.81967506e-01 -2.76192230e-01 6.88826685e-01
1.37911269e-01 1.47307334e+00 2.30914418e-01 -5.49115284e-01
-6.40926623e-02 5.34554053e-01 -4.16459451e-01 -7.89238006e-01
7.09186297e-01 -5.42672189e-01 -5.31194845e-01 -4.05222382e-01
-4.69228939e-01 -5.79498694e-01 -6.37892291e-01 -3.45788869e+00
6.82861355e-01 -5.75601337e-01 -3.39982130e-01 -8.99357265e-01
-4.94400596e-01 -5.76301591e-01 -6.32773242e-01 2.39490393e-01
-7.92297893e-03 -5.22642017e-01 -3.85477057e-01 -5.63947729e-01
-1.79269616e-02 1.92051455e+00 2.42469963e-02 -5.54157201e-01
2.96751354e-01 -9.02298491e-01 7.48387974e-01 -7.35340154e-01
-4.03958828e-01 -5.46978316e-02 -5.37188342e-02 -5.98502386e-01
-5.05471955e-01 -6.07063783e-01 1.16572501e-01 1.05149012e+00
1.90328804e-02 -5.94281312e-01 -1.11739490e-01 -2.12364766e-01
1.90439233e-01 -9.71236719e-02 -5.36361489e-01 -1.54941672e-02
-2.18652886e-02 -6.29050176e-01 -5.17286714e-01 -6.30328541e-01
1.28693407e-02 -4.25700768e-01 -1.28153278e-01 -5.94383863e-01
2.52148843e+00 9.49112138e-01 -2.74993525e-02 -7.62998388e-01
-5.21678603e-01 2.13540241e-01 2.09854020e-01 -5.97895789e-01
-4.87769838e-01 -1.04245383e+00 -4.40412710e-02 -1.91215548e-01
-7.60153069e-02 -5.41870395e-01 -1.96125955e-01 8.16461530e-01
-5.59448616e-01 1.11106685e+00 -9.13274788e-01 5.66847558e-01
5.51271194e-01 -5.38336690e-01 -3.93923953e-01 7.58937730e-02
1.57873429e-01 4.54434126e-01 -7.51488800e-02 -5.32407354e-01
-1.80193824e-01 6.52120792e-01 -3.25481259e-01 -1.40666137e+00
-1.18175601e-01 9.95535677e-01 9.84393910e-01 -5.72399652e-01
-4.79248337e-01 1.05486086e+00 -1.73042047e-01 -1.21230517e+00
1.72069099e-01 -5.45640451e-01 -2.17190372e-01 8.04133775e-02
-5.62221665e-01 -8.28709321e-01 1.06261885e+00 -6.55060993e-01
-5.53548196e-01 -5.38599704e-01 -2.85498505e-01 -5.14995190e-01
-9.13339690e-01 5.69902910e-01 7.68723338e-01 9.14637554e-02
1.19616832e-01 -6.51583125e-01 -6.08068231e-01 -5.52072090e-01
1.00000000e+00]
[ 0.00000000e+00 7.26448039e-01 7.22318654e-01 -4.41795080e-01
-4.14291675e-01 -8.11211726e-01 -1.68393384e-01 6.69170011e-01
8.85444895e-01 -5.61691421e-01 -3.43669644e-01 5.16512677e-01
-3.95683250e-01 -5.30567384e-01 -7.32737464e-01 7.12303151e-01
7.35539599e-01 -5.15399898e-01 -2.73496008e-01 6.95741012e-01
-1.26170927e-02 1.07838788e+00 1.41198313e+00 -4.99289318e-01
-6.40926623e-02 -7.14088245e-01 -4.10206510e-01 -7.89238006e-01
7.20126038e-01 -5.91121552e-01 -5.80223811e-01 -2.41147080e-01
-4.25982423e-01 -6.26673871e-01 -1.78203433e-01 -4.73230705e-01
-5.44403930e-01 -4.78806178e-02 -3.39982130e-01 -1.20147845e+00
-4.39797912e-01 -5.76301591e-01 -6.55654210e-01 1.13571272e-01
-3.88929415e-02 -4.68760544e-01 -3.81735249e-01 -5.02829900e-01
5.29150084e-02 4.94762465e-02 -3.05963293e-01 -4.47562171e-01
1.70153147e+00 -8.22266358e-01 1.41579823e+00 -5.82691223e-01
-3.98933797e-01 -4.94084208e-02 -4.21923287e-02 -5.47979479e-01
-5.02138698e-01 -5.61492155e-01 -6.16808211e-02 7.76780536e-02

```

```
-6.72372412e-02 -5.62226908e-01 -3.76655616e-01 -5.64252030e-01
5.71598367e-01 -4.20337509e-01 -5.33860882e-01 -8.74818744e-02
-2.96216689e-02 -5.79632419e-01 -5.15799862e-01 -5.88838212e-01
-5.96636935e-02 -3.09403993e+00 2.46140688e+00 -5.88109417e-01
7.63273878e-01 1.02054618e-01 2.03499885e+00 -7.62998388e-01
-5.20429612e-01 1.54317759e-01 1.56755668e-01 -5.26589143e-01
-4.82746959e-01 -9.38928179e-01 -1.63370341e-01 4.35703444e-02
-1.47789116e-01 -4.59202901e-01 -1.96125955e-01 -9.54107282e-01
-5.31704697e-01 -8.98546778e-01 -8.56405714e-01 5.87875022e-01
5.69229704e-01 -5.11827942e-01 -3.89758065e-01 1.25868131e-01
2.17572428e-01 5.89980252e-01 -1.60977809e-01 -5.14726263e-01
-1.80193824e-01 -4.35234413e-01 -3.25380951e-01 -1.40666137e+00
-5.99829341e-02 9.66304144e-01 9.46941709e-01 -5.24617423e-01
-4.75202642e-01 1.01747496e+00 -6.15199981e-02 3.81445712e-01
-2.00246509e-01 -4.40128962e-01 -2.17190372e-01 -6.29052552e-01
-5.60681196e-01 -8.28709321e-01 1.00804525e+00 -6.68634292e-01
-5.47983311e-01 -5.74807139e-01 -2.75240323e-01 -4.96019756e-01
7.88391197e-02 5.56086914e-01 7.39856682e-01 1.64479603e-01
-4.60910303e-01 -6.62289510e-01 -6.25503410e-01 -5.49433323e-01
1.00000000e+00]]
```

Last 5 rows of merged_full_data (should be synthetic):

```
[[ 5.00000000e+00 7.16814572e-01 7.40192792e-01 1.24752653e+00
6.85000356e-01 -6.86657035e-01 2.25889821e-01 1.71740284e+00
6.18504315e-01 9.83444811e-01 5.11729215e-01 -8.12520023e-01
9.13708824e-01 -2.54286317e-01 -7.01594271e-01 7.98335035e-01
7.26222607e-01 1.66898422e+00 7.70528774e-01 8.27197391e-01
1.18005655e-01 1.23713710e+00 -1.44308541e-02 1.41134448e+00
-6.40926623e-02 -9.87002929e-01 9.38606560e-01 -3.18003073e-01
8.97311413e-01 -8.72749658e-01 -8.92136949e-01 9.49139937e-01
4.21442338e-01 -8.52805563e-01 -6.90634826e-01 1.62716611e+00
3.47078133e-01 9.44033606e-01 -7.76354786e-02 -7.78099981e-01
1.32525297e+00 -4.27902449e-01 -7.55556337e-01 -1.30477743e+00
-1.28977681e+00 4.51817648e-01 -6.56687455e-02 4.82400647e-01
1.43565468e-01 -4.10815159e-03 -5.40428384e-01 4.54370005e-01
9.76528933e-01 -7.15120765e-01 1.31085147e+00 -2.77393360e-01
-4.19171519e-03 1.05165815e+00 1.13372089e+00 1.26825441e+00
8.13577257e-01 1.17892346e+00 -1.01204879e+00 -3.81044077e-01
-6.73943775e-02 9.79719522e-01 -6.85724430e-01 -6.67844505e-01
1.98040550e-02 -2.04861618e-01 6.54863968e-01 -4.31659630e-01
-5.22230226e-01 1.35831498e+00 9.49825782e-01 1.34242631e+00
1.57745525e+00 3.95463805e-02 -2.75808897e-01 9.96392760e-01
5.08065120e-01 -7.20049164e-01 5.73277237e-01 -3.94551072e-02
9.14672392e-01 2.02043846e+00 2.13402421e+00 9.00519567e-01
3.77700419e-01 -1.49310157e-01 -1.74422006e+00 -5.24326822e-01
-7.77930178e-02 5.66825530e-01 2.01736775e-02 -1.28026469e+00
9.02507666e-01 -8.49948949e-01 -4.15516438e-01 1.09491374e+00
1.12166895e+00 7.10164900e-01 1.92918499e-01 1.35542823e+00
-1.15406193e+00 -1.89765701e-01 -8.75409527e-02 6.78996449e-01
-1.80193824e-01 -1.38232606e+00 -2.00839290e-01 -1.35541488e+00
1.70145603e+00 -6.76866583e-01 -8.44534351e-01 1.59899501e+00
1.27248903e+00 -6.93527535e-01 2.17456157e+00 7.05897918e-01
-2.08276477e-01 1.76426058e+00 9.76292899e-02 -1.29272720e+00
1.08151537e+00 -8.28709321e-01 -8.04935928e-01 3.06359063e-01
1.84767070e+00 -8.74774067e-01 6.91161861e-01 6.47678920e-01
```

-2.53992862e-01	6.78230523e-01	6.85800046e-01	-8.16279704e-01
-7.92640094e-01	5.37224707e-01	4.94284956e-01	6.97620013e-01
4.00000000e+00]			
[6.00000000e+00	3.82770010e-01	3.62888896e-01	1.27475028e+00
7.27676801e-01	-3.63388906e-01	8.77293573e-01	-4.37072466e-02
-3.26934856e-01	1.97365720e+00	-3.43669644e-01	-1.16422295e+00
2.28673379e-01	-3.05189084e-01	-5.18998277e-01	8.92518894e-01
8.79820505e-01	1.12041658e+00	3.34592544e-01	9.01567030e-01
-9.89329829e-02	1.41102395e-01	-3.77896251e-01	1.30872552e+00
-6.40926623e-02	-7.39585667e-01	3.91951005e-01	-2.39463917e-01
1.03857457e+00	-1.05829188e+00	-1.06316501e+00	8.93341843e-01
3.59790414e-01	-1.02119436e+00	-4.69000181e-01	-1.87993403e-01
-2.61391579e-01	7.79540589e-01	2.91457934e+00	-6.84812201e-01
2.11738212e+00	-3.53431089e-01	-8.05380258e-01	-2.11099596e+00
-1.70800895e+00	8.61380301e-01	2.27732072e-01	9.37376088e-01
-3.45190778e-02	-2.73497548e-01	-4.99640388e-01	9.02249178e-01
1.15711565e+00	-7.83218125e-01	9.58151804e-01	2.58723892e-01
3.89356885e-01	2.28560515e+00	3.00559233e+00	2.60888776e+00
3.26096122e+00	2.53709619e+00	9.07189783e-01	-2.56293150e-01
-4.12642290e-01	2.80235303e+00	-1.17247188e+00	-1.09101908e+00
-1.59225369e+00	-2.04861618e-01	3.16226209e+00	8.95515330e-01
2.40253846e+00	2.21541484e+00	2.47271434e+00	2.19861015e+00
3.00639669e-01	-3.45612994e-01	-4.65444762e-01	2.76249285e+00
-7.18890762e-01	-1.13125770e+00	-6.16590417e-01	-3.82870196e-01
2.41364486e+00	-2.22082262e-01	-3.35079482e-01	1.25578593e+00
8.04363751e-01	1.80425026e-01	9.55609912e-02	8.61833517e-01
-1.20929705e-01	8.18840393e-01	2.24368236e-01	-1.37401599e+00
7.06381481e-01	-8.98546778e-01	-1.66456304e-01	6.68297322e-01
7.17583507e-01	1.24256344e+00	6.43677686e-01	3.92580904e-01
8.99587843e-01	-3.25066874e-01	-1.16665813e-01	7.96582077e-01
-1.80193824e-01	-1.40849083e+00	2.91189831e-02	-1.24772579e+00
2.22712289e-01	-9.59659214e-01	-9.99834411e-01	1.69225354e+00
1.40109537e+00	-9.64857999e-01	-2.26913457e+00	7.57106555e-01
-1.81284041e-01	1.72692357e+00	5.33284478e-01	-1.38286010e+00
2.02732832e+00	-8.28709321e-01	-1.00460781e+00	2.28630883e+00
8.38974057e-01	-6.21699357e-01	2.97990155e-01	6.98411141e-01
-9.05209230e-01	9.70681055e-01	3.47084439e-01	-1.86311533e+00
-7.51173870e-01	2.12542499e+00	2.20627402e+00	2.68493616e+00
4.00000000e+00]			
[7.00000000e+00	2.85597766e-01	3.06926810e-01	1.32477254e+00
7.73740209e-01	-2.65376311e-01	-1.26342786e+00	-6.54509301e-01
-5.98792334e-02	9.06886828e-01	-3.43669644e-01	-1.00119341e+00
2.23261717e-01	-1.54936885e-01	-4.54656739e-01	7.29801565e-01
7.31807771e-01	5.83735181e-01	2.15328602e-02	7.26440053e-01
9.78363558e-01	-1.29176113e-01	-2.80696365e-01	5.52415916e-01
-6.40926623e-02	-1.17144688e+00	3.33490217e-02	-3.96542228e-01
7.58851220e-01	-8.57655983e-01	-8.54730424e-01	3.67307727e-01
-1.09005320e-01	-8.74022575e-01	2.20539944e-01	4.11545229e-01
3.74405711e-02	2.73324769e-01	-5.12803885e-02	-7.96485618e-01
3.38150983e-01	-4.48505463e-01	-7.56743564e-01	-2.39191012e+00
-2.12562937e+00	2.82930659e-01	-1.57823366e-01	4.80681692e-01
8.24006583e-02	-3.38319985e-01	-3.67673325e-01	2.70815404e-01
8.55649546e-01	-9.17794564e-01	5.64183033e-01	-4.64520182e-01
7.92793635e-02	2.02060283e+00	1.98210074e+00	1.30796770e+00

8.71439247e-01	1.41747921e+00	3.57222712e-01	-1.04918491e-01
-3.15603772e-01	1.29881580e+00	-9.50640557e-01	-1.01688222e+00
-1.38107442e+00	-3.12599563e-01	1.19911957e+00	4.71579174e-01
7.58922908e-01	1.11819197e+00	6.24815769e-01	1.10410359e+00
5.92722366e-01	-2.60366201e-01	-4.48444230e-01	1.48176420e+00
-4.45498626e-01	-1.06339241e+00	1.48206521e-01	-5.09579593e-01
5.99256347e-01	8.82490156e-01	1.31396363e+00	2.49951924e+00
2.90836243e+00	5.51883493e-02	-3.14388078e+00	6.53734119e-02
-1.95901898e-01	3.10662570e+00	-8.97155807e-02	-1.46664036e+00
1.55041357e+00	-8.98546778e-01	-6.58471284e-01	8.86199811e-01
8.58891782e-01	1.62797815e+00	1.07572391e+00	9.22271381e-01
-1.82760498e+00	3.62371877e-01	-1.88212588e-01	1.58729406e+00
-1.80193824e-01	-1.40757596e+00	1.63997769e-02	-1.40666137e+00
9.51564272e-01	-2.86904943e-01	9.27918357e-02	2.26448567e+00
2.41561614e+00	-2.13896170e-01	2.11927806e+00	-3.00966991e-01
-2.42957178e-01	3.00928728e+00	3.03156586e-01	-1.40360130e+00
7.26700554e-01	-8.28709321e-01	-5.54672611e-01	5.12897964e-03
8.23390220e-02	-5.03169650e-01	3.85880384e-02	4.29617896e-01
1.21911602e-01	4.46843246e-01	3.32097049e-01	-2.02099389e+00
-8.34106318e-01	9.80046103e-03	-4.19750255e-02	8.88759484e-01
4.00000000e+00]			
[8.00000000e+00	-4.95141724e-01	-4.71225224e-01	5.22012841e-01
1.72460375e-02	4.84968064e-01	5.81570507e-01	-7.96896019e-01
-1.22339152e-01	6.02693331e-01	-3.43669644e-01	-9.17141162e-01
-3.11115417e-01	-1.54936885e-01	2.14646458e-01	5.88554361e-01
5.84909695e-01	3.98888393e-01	-6.21169804e-02	5.82084475e-01
-8.17880789e-01	9.00895946e-01	-6.99238531e-03	2.95579216e-01
-6.40926623e-02	-1.16540622e+00	-3.87278706e-02	7.46927048e-02
5.34367285e-01	-9.54199450e-01	-9.48314543e-01	7.86650851e-01
2.51548488e-01	-9.39235118e-01	-8.79484513e-01	2.42597941e-01
-3.25678206e-01	8.35750623e-01	7.97138884e-01	-1.11452502e+00
1.14188539e+00	-2.56811270e-01	-7.81740825e-01	3.75839613e+00
3.60769143e+00	5.41553950e-01	-2.91926670e-04	7.99031575e-01
-4.51310131e-01	7.47106609e-02	-9.98141851e-02	5.57921811e-01
4.84813319e-01	-8.54570686e-01	6.99585563e-01	2.79045028e-02
3.80010762e-01	-2.43097342e+00	-2.69023959e+00	5.88846132e-01
5.24586806e-02	9.90198565e-01	-9.54659725e-02	3.52970521e-01
-3.15500716e-01	8.06808143e-01	-1.07458969e+00	-1.04809702e+00
-2.18462798e+00	-3.12599563e-01	7.83589325e-01	-1.70119094e+00
-1.11099280e+00	5.66353924e-01	2.06160398e-02	5.75910857e-01
-1.16775157e+00	5.43700350e-01	-1.59270800e-01	2.72465611e-01
4.34942683e-01	-6.85987743e-01	6.89939187e-01	-1.29451400e-01
4.44921881e-02	1.62054901e-01	-3.41039163e-01	1.92811269e+00
1.79179469e+00	6.40189413e-02	-1.70124460e+00	8.02867405e-01
-1.57119976e-01	1.70288496e+00	-8.97155807e-02	-1.44811592e+00
1.48826886e+00	-8.98546778e-01	-4.49829144e-01	8.75513313e-01
8.77529452e-01	2.22884537e+00	1.88022176e+00	9.47778600e-01
-2.56629704e+00	7.64924929e-01	-2.10405125e-01	2.21526954e+00
-1.80193824e-01	-1.80075025e+00	1.46076262e-01	-1.40666137e+00
9.54931105e-01	-1.53466923e+00	-1.59226943e+00	1.82411074e+00
1.59425067e+00	-1.04945073e+00	-1.29451387e-01	5.26415055e-01
-2.23423979e-01	2.16548056e+00	1.35583881e-01	-1.40503091e+00
1.54133739e+00	-8.28709321e-01	-1.06142408e+00	-1.22902068e-01
8.08659970e-03	-1.71932625e-02	-5.72217210e-02	1.78685458e-01

```

-1.52051963e+00 -1.47708092e-01 -5.35932352e-01 3.18110260e+00
-6.84223837e-01 -1.11411405e-01 -1.20644300e-01 5.03879178e-01
4.00000000e+00]
[ 9.00000000e+00 7.48867883e-01 7.28476828e-01 1.07525571e+00
5.27605277e-01 -7.35317273e-01 -7.48349634e-01 4.37574883e-01
-2.25533313e-01 1.25700161e+00 5.11729215e-01 -7.98577590e-01
6.95326215e-01 -1.40771819e-01 -7.16705256e-01 7.23370490e-01
6.81278251e-01 1.21563319e+00 4.01372591e-01 7.37034417e-01
-2.65018369e-01 9.95015917e-01 -1.47968187e-01 1.23357454e+00
-6.40926623e-02 -9.68834627e-01 5.16499781e-01 -2.52210028e-01
7.62228741e-01 -5.08174000e-01 -5.23127766e-01 9.56494021e-01
4.28547588e-01 -5.04876032e-01 -1.53992348e+00 2.92892036e-01
-3.13792864e-01 9.85778756e-01 -3.39982130e-01 -1.03908126e+00
2.48963460e-01 -5.12403527e-01 -6.04943263e-01 -1.06906704e+00
-7.12532026e-01 4.93181775e-01 -4.06755614e-02 5.00654552e-01
-2.51896247e-01 -2.74471589e-01 -4.76638020e-01 5.29176189e-01
8.70435489e-01 -1.00935828e+00 1.47056814e+00 -4.79589049e-01
-6.08300098e-03 2.74773953e+00 2.10107133e+00 1.35313166e+00
9.36268224e-01 1.67241205e+00 -2.30487533e+00 4.33011096e-01
-4.46887449e-01 1.79843924e+00 -9.29143952e-01 -8.49582883e-01
-1.77899023e+00 -5.28075454e-01 1.85034930e+00 2.64617296e+00
3.08885058e+00 9.51193342e-01 4.14852278e-01 9.91304438e-01
-2.14988413e+00 -6.35633481e-01 -3.71097019e-01 9.62871232e-01
-5.55387035e-01 -1.01475683e+00 7.23566195e-01 -3.62306541e-01
5.18194555e-01 6.41147728e-01 8.33590197e-01 1.14903964e+00
6.48755308e-01 -8.54505008e-01 -1.00491024e+00 -2.32059694e-01
-1.18410199e-01 5.72220565e-01 1.75659385e+00 -1.41370831e+00
2.11380867e+00 -8.98546778e-01 -9.94325296e-01 7.77728717e-01
7.92617732e-01 7.84769619e-01 2.33137508e-01 6.17098377e-01
1.24384487e-01 6.23756673e-01 -1.96414801e-01 6.53762940e-01
-1.80193824e-01 -1.60865155e+00 -1.39008308e-01 -1.40666137e+00
5.48117685e-01 -4.38277102e-02 3.79511055e-01 2.20749916e+00
2.28283548e+00 2.93510102e-02 -1.33286145e+00 -1.45671335e+00
2.25391005e-02 1.24999153e+00 -5.96352148e-02 -1.40862901e+00
5.38992780e-01 -8.28709321e-01 -3.23730639e-01 1.96189603e-01
1.05305517e+00 -4.06708703e-01 3.80258175e-01 4.69194461e-01
-4.96294390e-01 6.69530568e-01 7.42922476e-01 -7.97539233e-01
-7.92640094e-01 3.21075477e-01 2.82194258e-01 1.19847585e+00
4.00000000e+00]]

```

6.1 - TVT split 60%-20%20%

- 60% will be used to train the model
 - 20% will be used for tuning the hyperparameters and preventing overfitting during training
 - 20% will be used the final model evaluation
-
- Isolate data by participant, so all participant data is present on each set

For each participant:

|0%-----Train-----60%|-----Validation-----80%|-----Test-----100%|

- Check the cutoffs
- Train-Validation-Test

```
In [61]: def get_participant_indices(participant_ids):
    participants_indices = {}
    unique_pids = np.sort(np.unique(participant_ids))

    for pid in unique_pids:
        indices_for_pid = np.where(participant_ids == pid)[0]
        if indices_for_pid.size > 0:
            start_idx = indices_for_pid.min()
            end_idx = indices_for_pid.max()
            participants_indices[int(pid)] = (start_idx, end_idx)

    return participants_indices

def get_participant_cutoffs(participant_ids):
    participants_cutoffs = {}
    unique_pids = np.sort(np.unique(participant_ids))

    for pid in unique_pids:
        participant_data_block_indices = np.where(participant_ids == pid)[0]
        Ni = len(participant_data_block_indices)

        if Ni == 0:
            continue

        cutoff_60_relative = int(np.ceil(0.60 * Ni))
        cutoff_80_relative = int(np.ceil(0.80 * Ni))

        absolute_idx_60_cutoff = (
            participant_data_block_indices[cutoff_60_relative - 1]
            if cutoff_60_relative > 0 else -1
        )
        absolute_idx_80_cutoff = (
            participant_data_block_indices[cutoff_80_relative - 1]
            if cutoff_80_relative > 0 else -1
        )

        participants_cutoffs[int(pid)] = {
            "Ni": Ni,
            "60_percent_cutoff_idx": absolute_idx_60_cutoff,
            "80_percent_cutoff_idx": absolute_idx_80_cutoff,
        }
```

```

return participants_cutoffs

def split_within_subject(participant_ids):
    participants_indices = get_participant_indices(participant_ids)
    participants_cutoffs = get_participant_cutoffs(participant_ids)
    unique_pids = np.sort(np.unique(participant_ids))

    train_indices, val_indices, test_indices = [], [], []

    for pid in unique_pids:
        pid = int(pid)
        info = participants_cutoffs[pid]
        start_idx, end_idx = participants_indices[pid]
        idx_60 = info['60_percent_cutoff_idx']
        idx_80 = info['80_percent_cutoff_idx']

        # Train = from start to 60%
        train_indices.extend(range(start_idx, idx_60 + 1))
        # Validation = from 60% to 80%
        val_indices.extend(range(idx_60 + 1, idx_80 + 1))
        # Test = from 80% to end
        test_indices.extend(range(idx_80 + 1, end_idx + 1))

    return np.array(train_indices), np.array(val_indices), np.array(test_indices)

```

6.2 - 9 subjects for training, 3 subjects for validation and 3 subjects for test

- It tests your model's ability to generalize to unseen participants, not just unseen time segments.

15 Participants: [P1,P2,P3,P4,P5,P6,P7,P8,P9,P10,P11,P12,P13,P14,P15]

After Shuffling (example): [P10,P5,P1,P12,P3,P9,P2,P14,P7,P15,P6,P11,P4,P8,P13]

Applying (9,3,3) split:

- Train: [P10,P5,P1,P12,P3,P9,P2,P14,P7]
- Validation: [P15,P6,P11]
- Test: [P4,P8,P13]

```

In [62]: def split_by_subject(participant_ids, subject_split=(9, 3, 3), random_seed=42):
    participants_indices = get_participant_indices(participant_ids)
    unique_pids = np.sort(np.unique(participant_ids))

```

```

np.random.seed(random_seed)
np.random.shuffle(unique_pids)

n_train, n_val, n_test = subject_split
train_pids = unique_pids[:n_train]
val_pids = unique_pids[n_train:n_train + n_val]
test_pids = unique_pids[n_train + n_val:n_train + n_val + n_test]

train_indices, val_indices, test_indices = [], [], []

for pid in train_pids:
    start_idx, end_idx = participants_indices[int(pid)]
    train_indices.extend(range(start_idx, end_idx + 1))
for pid in val_pids:
    start_idx, end_idx = participants_indices[int(pid)]
    val_indices.extend(range(start_idx, end_idx + 1))
for pid in test_pids:
    start_idx, end_idx = participants_indices[int(pid)]
    test_indices.extend(range(start_idx, end_idx + 1))

return np.array(train_indices), np.array(val_indices), np.array(test_indices)

```

6.3 - Function for True labels and Predicted labels + Classification Metrics

```

In [63]: def compute_classification_metrics(y_true, y_pred, average='weighted', labels=

y_true = np.array(y_true)
y_pred = np.array(y_pred)

cm = confusion_matrix(y_true, y_pred, labels=labels)
acc = accuracy_score(y_true, y_pred)
prec = precision_score(y_true, y_pred, average=average, zero_division=0)
rec = recall_score(y_true, y_pred, average=average, zero_division=0)
f1 = f1_score(y_true, y_pred, average=average, zero_division=0)
report = classification_report(y_true, y_pred, labels=labels, zero_division=0)

metrics = {
    "accuracy": acc,
    "precision": prec,
    "recall": rec,
    "f1_score": f1,
    "confusion_matrix": cm,
    "classification_report": pd.DataFrame(report).transpose()
}

return metrics

```

7 - KNN Implementation

```
In [64]: def euclidean_distance(x1, x2):
          return np.sqrt(np.sum((x1 - x2) ** 2))

def manhattan_distance(x1, x2):
    return np.sum(np.abs(x1 - x2))

def knn_predict(X_train, y_train, X_test, k=3, distance_metric='euclidean'):
    y_pred = []
    for x_test in X_test:
        if distance_metric == 'euclidean':
            distances = np.sqrt(np.sum((X_train - x_test) ** 2, axis=1))
        elif distance_metric == 'manhattan':
            distances = np.sum(np.abs(X_train - x_test), axis=1)
        else:
            raise ValueError("Unsupported distance metric: choose 'euclidean'")

        k_indices = np.argsort(distances)[:k]

        k_labels = y_train[k_indices]
        most_common = Counter(k_labels).most_common(1)[0][0]
        y_pred.append(most_common)

    return np.array(y_pred)

def knn_score(X_train, y_train, X_test, y_test, k=3, distance_metric='euclidean'):
    y_pred = knn_predict(X_train, y_train, X_test, k, distance_metric)
    return np.mean(y_pred == y_test)
```

8 - Evaluation

8.1 a) Using all features

- within subjects

```
In [65]: train_idx, val_idx, test_idx = split_within_subject(participant_ids_original)
          print(f"Train size: {len(train_idx)} | Val size: {len(val_idx)} | Test size: {len(test_idx)}")
```

Train size: 2113 | Val size: 702 | Test size: 697

```
In [66]: # Train set
          X_train, y_train = X[train_idx], y[train_idx]

          # Validation set
          X_val, y_val = X[val_idx], y[val_idx]
```

```
# Test set
X_test, y_test = X[test_idx], y[test_idx]
```

```
In [67]: k = 3
distance_metric = 'euclidean'

y_val_pred = knn_predict(X_train, y_train, X_val, k=k, distance_metric=distance_metric)
y_test_pred = knn_predict(X_train, y_train, X_test, k=k, distance_metric=distance_metric)

val_accuracy = knn_score(X_train, y_train, X_val, y_val, k=k, distance_metric=distance_metric)
test_accuracy = knn_score(X_train, y_train, X_test, y_test, k=k, distance_metric=distance_metric)

print(f"Validation Accuracy: {val_accuracy:.3f}")
print(f"Test Accuracy: {test_accuracy:.3f}")
```

Validation Accuracy: 0.980

Test Accuracy: 0.994

- 9/3/3 subjects split

```
In [68]: train_idx, val_idx, test_idx = split_by_subject(participant_ids_original, subject_ids)

print(f"Train size: {len(train_idx)} | Val size: {len(val_idx)} | Test size: {len(test_idx)}")
```

Train size: 2106 | Val size: 696 | Test size: 710

```
In [69]: # Train set
X_train, y_train = X[train_idx], y[train_idx]

# Validation set
X_val, y_val = X[val_idx], y[val_idx]

# Test set
X_test, y_test = X[test_idx], y[test_idx]
```

```
In [70]: k = 3
distance_metric = 'euclidean'

y_val_pred = knn_predict(X_train, y_train, X_val, k=k, distance_metric=distance_metric)
y_test_pred = knn_predict(X_train, y_train, X_test, k=k, distance_metric=distance_metric)

val_accuracy = knn_score(X_train, y_train, X_val, y_val, k=k, distance_metric=distance_metric)
test_accuracy = knn_score(X_train, y_train, X_test, y_test, k=k, distance_metric=distance_metric)

print(f"Validation Accuracy: {val_accuracy:.3f}")
print(f"Test Accuracy: {test_accuracy:.3f}")
```

Validation Accuracy: 0.917

Test Accuracy: 0.799

8.1 b) PCA-Transformed features

```
In [71]: X_pca_75, _, _ = pca(X_zscore, dimensions_needed)
print(X_pca_75.shape)
# smoothed_X_pca = gaussian_filter1d(X_pca_75_variance, sigma=1.0, axis=0)

(3512, 17)
```

- within subjects

```
In [72]: train_idx, val_idx, test_idx = split_within_subject(participant_ids_original)

print(f"Train size: {len(train_idx)} | Val size: {len(val_idx)} | Test size: {len(test_idx)}")

Train size: 2113 | Val size: 702 | Test size: 697
```

```
In [73]: # Train set
X_train, y_train = X_pca_75[train_idx], y[train_idx]

# Validation set
X_val, y_val = X_pca_75[val_idx], y[val_idx]

# Test set
X_test, y_test = X_pca_75[test_idx], y[test_idx]
```

```
In [74]: k = 3
distance_metric = 'euclidean'

y_val_pred = knn_predict(X_train, y_train, X_val, k=k, distance_metric=distance_metric)
y_test_pred = knn_predict(X_train, y_train, X_test, k=k, distance_metric=distance_metric)

val_accuracy = knn_score(X_train, y_train, X_val, y_val, k=k, distance_metric=distance_metric)
test_accuracy = knn_score(X_train, y_train, X_test, y_test, k=k, distance_metric=distance_metric)

print(f"Validation Accuracy: {val_accuracy:.3f}")
print(f"Test Accuracy: {test_accuracy:.3f}")
```

Validation Accuracy: 0.964
Test Accuracy: 0.990

- 9/3/3

```
In [75]: train_idx, val_idx, test_idx = split_by_subject(participant_ids_original, subject_ids)

print(f"Train size: {len(train_idx)} | Val size: {len(val_idx)} | Test size: {len(test_idx)}")

Train size: 2106 | Val size: 696 | Test size: 710
```

```
In [76]: # Train set
X_train, y_train = X_pca_75[train_idx], y[train_idx]
```

```
# Validation set
X_val, y_val = X_pca_75[val_idx], y[val_idx]

# Test set
X_test, y_test = X_pca_75[test_idx], y[test_idx]
```

```
In [77]: k = 3
distance_metric = 'euclidean'

y_val_pred = knn_predict(X_train, y_train, X_val, k=k, distance_metric=distance_metric)
y_test_pred = knn_predict(X_train, y_train, X_test, k=k, distance_metric=distance_metric)

val_accuracy = knn_score(X_train, y_train, X_val, y_val, k=k, distance_metric=distance_metric)
test_accuracy = knn_score(X_train, y_train, X_test, y_test, k=k, distance_metric=distance_metric)

print(f"Validation Accuracy: {val_accuracy:.3f}")
print(f"Test Accuracy: {test_accuracy:.3f}")
```

Validation Accuracy: 0.869
Test Accuracy: 0.868

8.1 c) Top 10 features selected by ReliefF

```
In [78]: top_10_features_names = df_relieff['Feature'].head(10).tolist()
print("Top 10 Features selecionadas:", top_10_features_names)

top_10_feature_indices = [feature_names.index(name) for name in top_10_features_names]
print("Índices das Top 10 Features no array X:", top_10_feature_indices)

X_relief = X[:,top_10_feature_indices]

y_relief = y

print(f"\nShape original de X: {X.shape}")
print(f"Shape de X_relief: {X_relief.shape}")
print(f"Shape de y original: {y.shape}")
print(f"Shape de y_relief: {y_relief.shape}")

print("\nPrimeiras 5 linhas de X_relief:")
print(X_relief[:5])
print("\nPrimeiras 5 linhas de y_relief:")
print(y_relief[:5])
```

Top 10 Features selecionadas: ['ay_energy', 'az_median', 'az_rms', 'az_mean', 'az_energy', 'my_mean', 'my_median', 'ay_rms', 'mz_energy', 'ay_mean']
Índices das Top 10 Features no array X: [27, 29, 32, 28, 41, 98, 99, 18, 125, 14]

Shape original de X: (3512, 139)
Shape de X_relief: (3512, 10)
Shape de y original: (3512,)
Shape de y_relief: (3512,)

Primeiras 5 linhas de X_relief:

```
[[2.19147370e+04 2.47910000e+00 2.54482975e+00 2.54020360e+00
 1.61609910e+03 5.97767560e-01 6.00400000e-01 9.36275542e+00
 2.62626329e+02 9.36251600e+00]
[2.19300425e+04 2.55710000e+00 2.56598220e+00 2.56201640e+00
 1.64352553e+03 5.97430080e-01 6.00400000e-01 9.36601251e+00
 2.64660298e+02 9.36579680e+00]
[2.19582487e+04 2.62140000e+00 2.60846558e+00 2.60674440e+00
 1.69990163e+03 5.98924080e-01 6.02410000e-01 9.37200360e+00
 2.68176736e+02 9.37184800e+00]
[2.20311492e+04 2.62310000e+00 2.57908613e+00 2.57471240e+00
 1.66010367e+03 6.07012160e-01 6.08430000e-01 9.38754078e+00
 2.70018837e+02 9.38739960e+00]
[2.21337874e+04 2.48245000e+00 2.44604523e+00 2.43541320e+00
 1.48929685e+03 6.18666640e-01 6.18470000e-01 9.40957650e+00
 2.63437964e+02 9.40904720e+00]]
```

Primeiras 5 linhas de y_relief:

```
[1. 1. 1. 1. 1.]
```

- within subjects

```
In [79]: train_idx, val_idx, test_idx = split_within_subject(participant_ids_original)
print(f"Train size: {len(train_idx)} | Val size: {len(val_idx)} | Test size: {
```

Train size: 2113 | Val size: 702 | Test size: 697

```
In [80]: # Train set
X_train, y_train = X_relief[train_idx], y_relief[train_idx]

# Validation set
X_val, y_val = X_relief[val_idx], y_relief[val_idx]

# Test set
X_test, y_test = X_relief[test_idx], y_relief[test_idx]
```

```
In [81]: k = 3
distance_metric = 'euclidean'

y_val_pred = knn_predict(X_train, y_train, X_val, k=k, distance_metric=distance_metric)
```

```

y_test_pred = knn_predict(X_train, y_train, X_test, k=k, distance_metric=distance_metric)

val_accuracy = knn_score(X_train, y_train, X_val, y_val, k=k, distance_metric=distance_metric)
test_accuracy = knn_score(X_train, y_train, X_test, y_test, k=k, distance_metric=distance_metric)

print(f"Validation Accuracy: {val_accuracy:.3f}")
print(f"Test Accuracy: {test_accuracy:.3f}")

```

Validation Accuracy: 0.798
Test Accuracy: 0.694

- 9/3/3

```

In [82]: train_idx, val_idx, test_idx = split_by_subject(participant_ids_original, subject)

print(f"Train size: {len(train_idx)} | Val size: {len(val_idx)} | Test size: {len(test_idx)}")

```

Train size: 2106 | Val size: 696 | Test size: 710

```

In [83]: # Train set
X_train, y_train = X_relief[train_idx], y_relief[train_idx]

# Validation set
X_val, y_val = X_relief[val_idx], y_relief[val_idx]

# Test set
X_test, y_test = X_relief[test_idx], y_relief[test_idx]

```

```

In [84]: k = 3
distance_metric = 'euclidean'

y_val_pred = knn_predict(X_train, y_train, X_val, k=k, distance_metric=distance_metric)
y_test_pred = knn_predict(X_train, y_train, X_test, k=k, distance_metric=distance_metric)

val_accuracy = knn_score(X_train, y_train, X_val, y_val, k=k, distance_metric=distance_metric)
test_accuracy = knn_score(X_train, y_train, X_test, y_test, k=k, distance_metric=distance_metric)

print(f"Validation Accuracy: {val_accuracy:.3f}")
print(f"Test Accuracy: {test_accuracy:.3f}")

```

Validation Accuracy: 0.759
Test Accuracy: 0.375

8.2 - Hypparameter Tunning

- Using the 8.1. a) (only within subject)

```

In [85]: train_idx, val_idx, test_idx = split_within_subject(participant_ids_original, subject)

print(f"Train size: {len(train_idx)} | Val size: {len(val_idx)} | Test size: {len(test_idx)}")

```

```

# Train set
X_train, y_train = X[train_idx], y[train_idx]

# Validation set
X_val, y_val = X[val_idx], y[val_idx]

# Test set
X_test, y_test = X[test_idx], y[test_idx]

```

Train size: 2113 | Val size: 702 | Test size: 697

```

In [86]: k_values = [1, 3, 5, 7, 9, 11, 13, 15]

best_k = -1
best_val_accuracy = -1.0
distance_metric = 'euclidean'

print("--- Hyperparameter Tune (k) ---")
for k in k_values:
    val_accuracy = knn_score(X_train, y_train, X_val, y_val, k=k, distance_metric=distance_metric)
    print(f"k = {k}: Validation Accuracy = {val_accuracy:.3f}")

    if val_accuracy > best_val_accuracy:
        best_val_accuracy = val_accuracy
        best_k = k

print(f"\nBest k on validation set: {best_k} (Accuracy: {best_val_accuracy:.3f})")

X_train_combined = np.vstack((X_train, X_val))
y_train_combined = np.concatenate((y_train, y_val))

print(f"\nShape of X_train_combined: {X_train_combined.shape}")
print(f"Shape of y_train_combined: {y_train_combined.shape}")

print("\n--- Final evaluation on test set ---")
final_test_accuracy = knn_score(X_train_combined, y_train_combined, X_test, y_test, distance_metric=distance_metric)
print(f"Final accuracy on test set with best_k={best_k}: {final_test_accuracy:.3f}")

```

```

--- Hyperparameter Tune (k) ---
k = 1: Validation Accuracy = 0.976
k = 3: Validation Accuracy = 0.980
k = 5: Validation Accuracy = 0.981
k = 7: Validation Accuracy = 0.983
k = 9: Validation Accuracy = 0.983
k = 11: Validation Accuracy = 0.983
k = 13: Validation Accuracy = 0.981
k = 15: Validation Accuracy = 0.981

Best k on validation set: 7 (Accuracy: 0.983)

Shape of X_train_combined: (2815, 139)
Shape of y_train_combined: (2815,)

--- Final evaluation on test set ---
Final accuracy on test set with best_k=7: 0.994

```

8.3 - Results and Discussions

```

In [87]: feature_sets = ['All Features\nWithin Subject', 'All Features\n9/3/3',
                        'PCA Features\nWithin Subject', 'PCA Features\n9/3/3',
                        'ReliefF Top 10\nWithin Subject', 'ReliefF Top 10\n9/3/3']

validation_acc = [0.980, 0.917, 0.964, 0.869, 0.798, 0.759]
test_acc = [0.994, 0.799, 0.990, 0.868, 0.694, 0.375]

x = np.arange(len(feature_sets))
width = 0.35

fig, ax = plt.subplots(figsize=(14, 7))

bars1 = ax.bar(x - width/2, validation_acc, width, label='Validation Accuracy',
               color='#3498db', edgecolor='black', linewidth=1.2)
bars2 = ax.bar(x + width/2, test_acc, width, label='Test Accuracy',
               color='#e74c3c', edgecolor='black', linewidth=1.2)

for bars in [bars1, bars2]:
    for bar in bars:
        height = bar.get_height()
        ax.text(bar.get_x() + bar.get_width()/2., height,
                f'{height:.3f}',
                ha='center', va='bottom', fontsize=10, fontweight='bold')

ax.set_xlabel('Feature Set and Split Strategy', fontsize=12, fontweight='bold')
ax.set_ylabel('Accuracy', fontsize=12, fontweight='bold')
ax.set_title('KNN Classifier (k=3) Performance Comparison', fontsize=14, fontw
ax.set_xticks(x)
ax.set_xticklabels(feature_sets, fontsize=11)
ax.set_ylim(0, 1.05)

ax.grid(axis='y', alpha=0.3, linestyle='--')

```

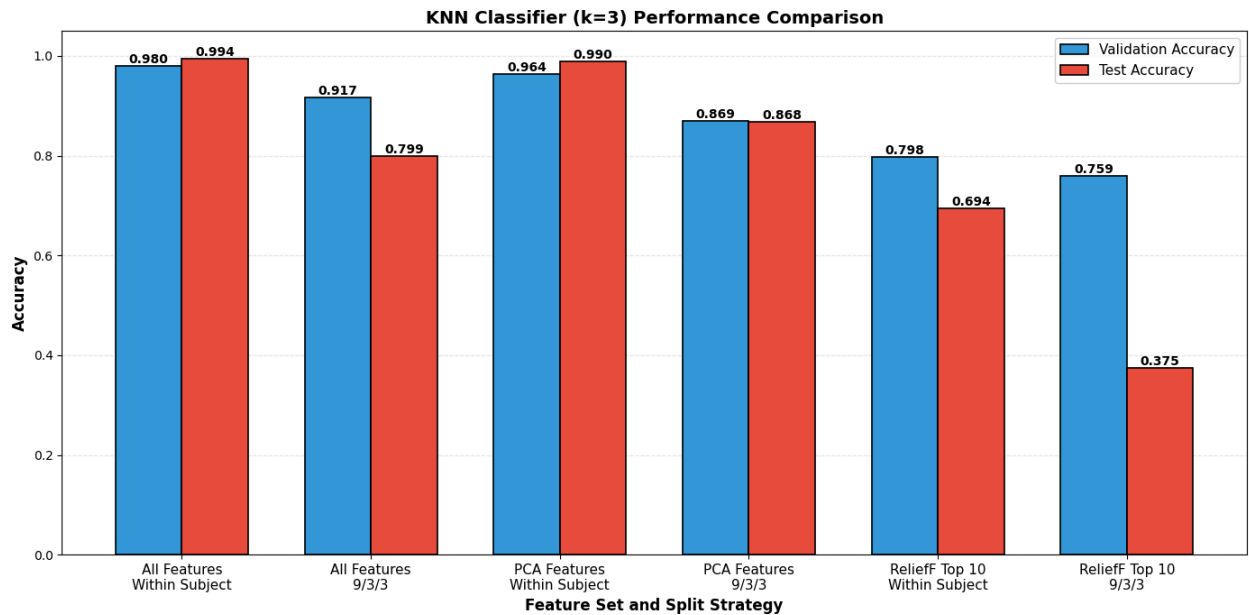
```

ax.set_axisbelow(True)

ax.legend(fontsize=11, loc='upper right', framealpha=0.9)

plt.tight_layout()
plt.show()

```



- Based on the analysis of the best model is the one who was tested and validated using all features within a subject, and the other with the PCA features within subject. Using a within subject TVT is important on this scenario, where i'm working with time-serie data (the order of observations within a participant is important). By spiting the data for each participant in train, validation and test sets based on the order their observations, 0-60% for training, 60-80% for validation and 80-100% for testing, i ensure that the model is always trained with earlier data points and evaluated with later data points for the same individual. Also by having a training, validation and test set for each participant my model gets exposure to the individual characteristics and variability of every participant during training, leading to a more robust model capable of generalize better to new observations.
- The split by subject approach is used to evaluate the model's ability to generalize to new participants that it has never seen before. Assigning entire participants data exclusive to either training, validation or test set, the evaluation will be more realistic. These lead to the performance on the test set directly reflects the model's ability to generalize completely novel individuals.

- Within Subject Split using all features slightly outperforms using PCA features, though both achieve excellent results. The benefit of PCA isn't as critical when the model sees some of the subject's data during training.
- 9/3/3 by subject split PCA features significantly outperforms all features in terms of test accuracy (0.868 vs 0.799). This is a strong indicator that PCA is helping the model generalize better to unseen subjects and it's likely reducing noise and potentially multicollinearity among the original features, leading to a more robust model when faced with entirely new participants.
- Using ReliefF had worst performance than the others because i selected (as the assignment asks) only the top ten features, discarding all other features, being more probable that more than ten features contain valuable information for classification.

9

- The number of data points (3512 rows) is low and when splitting within subjects, the models might be learning very specific patterns or even noise present in the limited samples rather than truly generalizable. In my case, the models performance was great, considering the number of features extracted helped the model a lot to generalize to new participants even without a few data points.
- A larger dataset would allow the models to learn more robust and generalize patterns reducing the risk of overfitting to specific instances, and would capture a wider range of individual differences.